UNIVERSAL CODE SCANNER

Documentación de sistema

Todos los documentos en un solo archivo

Versión analizada: **1.1.0+2**
Commit: **6d5e0b7**
Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. Documentación de sistema · Universal Code Scanner
2. 01 · Descripción general del sistema
3. 02 · Instalación y ejecución
4. 03 · Arquitectura
5. 04 · Mapa completo del código
6. 05 · Referencia técnica
7. 06 · Explicación profunda del código
8. 07 · Base de datos y persistencia
9. 08 · Flujo de datos
10. 09 · APIs e integraciones
11. 10 · Configuración
12. 11 · Seguridad
13. 12 · Pruebas y calidad
14. 13 · Despliegue y operación
15. 14 · Solución de problemas
16. 15 · Riesgos y deuda técnica
17. 16 · Glosario
18. 17 · Resumen ejecutivo
19. 18 · Guía para nuevos desarrolladores
20. 19 · Matriz de trazabilidad

Documentación de sistema · Universal Code Scanner

Portada e índice de la documentación técnica, funcional, arquitectónica y operativa del repositorio. Se escribió recorriendo el código fuente completo, no resumiendo la documentación existente: cada afirmación apunta a un archivo, una función, una prueba o un comando concreto de este repositorio.

El sistema en una frase

Universal Code Scanner es una **aplicación Flutter que lee códigos QR y de barras en el propio dispositivo**, interpreta su contenido en campos legibles, evalúa señales locales de riesgo y **exige confirmación explícita antes de ejecutar cualquier acción externa**. No consulta ningún servidor propio.

Para quién es esta documentación

Perfil	Empieza por	Después
Persona no técnica	Descripción general	Resumen ejecutivo
Desarrollador que se incorpora	Guía para nuevos desarrolladores	Instalación → Arquitectura → Mapa del código
Desarrollador con experiencia	Referencia técnica	Explicación profunda
Auditor técnico o de seguridad	Arquitectura	Seguridad → Riesgos y deuda técnica → Trazabilidad
Operación y soporte	Despliegue y operación	Solución de problemas
Otro agente de IA	Mapa del código	Referencia técnica + Trazabilidad

Tabla de contenidos

#	Documento	Contenido	Estado
01	Descripción general del sistema	Qué es, qué problema resuelve, actores, flujo, tecnologías y límites	Completo
02	Instalación y ejecución	Requisitos, dependencias, generación de plataformas, ejecución y pruebas	Completo
03	Arquitectura	Capas, patrones, diagramas, estado, errores, persistencia y seguridad	Completo
04	Mapa completo del código	Inventario jerárquico de directorios, archivos, clases y funciones	Completo
05	Referencia técnica	Catálogo de constantes, tipos, funciones, claves, almacenes y errores	Completo

06	Explicación profunda del código	Flujo interno, decisiones y casos límite de los módulos importantes	Completo
07	Base de datos	Motor, almacenes, diccionario de datos, migraciones y cifrado	Completo
08	Flujo de datos	Origen, validación, transformación, almacenamiento y salida de cada dato	Completo
09	APIs e integraciones	Contratos JSON, interfaces de plataforma y ausencia deliberada de red	Completo
10	Configuración	Preferencias, banderas, límites y consecuencias de cada valor	Completo
11	Seguridad	Cifrado, análisis de enlaces, validación de entrada y superficie de ataque	Completo
12	Pruebas y calidad	Suite existente, cobertura observable, huecos y propuesta priorizada	Completo
13	Despliegue y operación	CI/CD, artefactos, publicación, PWA, mantenimiento y reversión	Completo
14	Solución de problemas	Síntoma, causa, diagnóstico, solución y riesgo	Completo
15	Riesgos y deuda técnica	Hallazgos clasificados por severidad, impacto, evidencia y prioridad	Completo · informativo
16	Glosario	Términos técnicos, del dominio y del producto	Completo
17	Resumen ejecutivo	Presentación para decisión, sin detalle innecesario	Completo
18	Guía para nuevos desarrolladores	Itinerario, entorno, convenciones y primeras tareas	Completo
19	Matriz de trazabilidad	De la funcionalidad al código, a los datos y a su prueba	Completo

Los mismos documentos en PDF están en `pdf/`. Se generan desde estos Markdown con `python tool/build_system_documentation_pdf.py`; el procedimiento está en 13-deployment-and-operations.md.

Datos del análisis

Dato	Valor

Fecha del análisis	27 de agosto de 2026
Versión analizada	`1.1.0+2` (`pubspec.yaml`)
Commit analizado	`c1f9878`
Rama	`main`
Toolchain usado para verificar	Flutter 3.44.7 · Dart 3.12.2
Archivos versionados en el repositorio	171 (`git ls-files`)
Archivos Dart en `lib/`	59
Líneas de Dart en `lib/`	6 821
Archivos de prueba Dart	20 (19 en `test/`, 1 en `integration_test/`)
Casos de prueba declarados	58 · **57 los ejecuta `flutter test`**
Herramientas en `tool/`	12
Dependencias directas de producción	19 paquetes de pub + 2 del SDK
Señales locales de riesgo de URL	16
Tipos de contenido interpretados (`ContentKind`)	17
Almacenes Sembast	5

Convenciones

Estos marcadores aparecen a lo largo de los documentos y significan siempre lo mismo:

Marcador	Significado
Comprobado	Se leyó el código, se ejecutó el comando o se verificó el archivo en este repositorio
Inferencia	Conclusión razonada a partir del código, no una afirmación literal del repositorio
No identificado	Se buscó y no existe evidencia en ningún sentido
No documentado en el repositorio	Existe en el código pero ninguna documentación previa lo explica
Requiere validación	Solo puede confirmarse con hardware, red o credenciales que este análisis no tiene

Otras convenciones:

- las rutas se escriben relativas a la raíz del repositorio;
- los nombres de clases, funciones, almacenes y claves se conservan **literales**, para que puedan buscarse con `grep`;
- ningún ejemplo contiene secretos, credenciales ni datos personales reales; se usan dominios reservados (`example.com`) y valores ficticios;

- los diagramas Mermaid siempre van acompañados de una explicación en texto: el diagrama nunca es el único portador de la información.

Qué se verificó ejecutando, y qué no

Ejecutado en la máquina de análisis con Flutter 3.44.7 sobre el commit `c1f9878`:

Comando	Resultado
`flutter pub get`	Resuelve · `pubspec.lock` versionado y sin cambios
`flutter analyze --fatal-infos`	**No issues found!** (100,8 s)
`flutter test`	Registrado en 12-testing-and-quality.md
`python tool/validate_structure.py`	Registrado en 12-testing-and-quality.md

Pendiente de validar

Lo que esta documentación **no** pudo comprobar en el entorno de análisis:

1. **Comportamiento en dispositivo físico.** Cámara real, biometría, almacén seguro del sistema, PDF grandes, TalkBack y VoiceOver solo pueden confirmarse en hardware. La matriz obligatoria vive en `../quality/DEVICE_TEST_MATRIX.md`.
2. **Compilaciones de iOS y macOS.** Requieren hardware Apple; la evidencia proviene del trabajo `apple` de la CI, no de esta máquina.
3. **Carpetas nativas.** `android/`, `ios/`, `web/` y `macos/` **no se versionan**: las genera `tool/bootstrap.py`. Lo que esta documentación describe de `ios/` y `macos/` proviene de leer ese generador, no de inspeccionar el resultado. `android/` y `web/` sí existen en la máquina de análisis, generados previamente, y se leyeron directamente.
4. **La lectura en un teléfono real después de las correcciones de 1.1.0.** Es la prueba que originó esas correcciones y sigue declarada pendiente en `../../VALIDATION.md`.
5. **La ejecución del catálogo de regresión de `test_assets/`.** Ninguna prueba Dart lo lee; ver 12-testing-and-quality.md y el hallazgo R-01 de 15-risks-and-technical-debt.md.

Relación con la documentación previa

Este directorio **no sustituye** la documentación existente del repositorio; la complementa y la enlaza. La documentación previa sigue siendo la fuente normativa en estas áreas:

Área	Fuente normativa
Estado de verificación de la versión publicada	`../../VALIDATION.md`
Qué está implementado y qué no	`../../IMPLEMENTATION_STATUS.md`
Historial de cambios por versión	`../../CHANGELOG.md`
Compromisos que no pueden romperse	`../quality/COMPATIBILITY_CONTRACT.md`
Comportamiento del escáner estado por estado	`../quality/SCANNER_UX.md`

Política de anclaje de dependencias	`../quality/LOCKFILE.md`
Modelo de amenazas y lista MASVS	`../security/THREAT_MODEL.md` · `../security/MASVS_CHECKLIST.md`
Lista de publicación en tiendas	`../RELEASE.md`

01 · Descripción general del sistema

Qué es

Universal Code Scanner es una **aplicación Flutter de lectura de códigos QR y de barras** publicada bajo licencia MIT. El manifiesto la describe en una línea (`pubspec.yaml`, campo `description`):

> Lector, intérprete, generador e inventario seguro de códigos QR y barras.

La versión analizada es `1.1.0+2`. El nombre del paquete Dart es `universal_code_scanner` y el identificador de organización usado al generar los proyectos nativos es `dev.vladimiracuna` (`tool/bootstrap.py`, argumento `--org` de `flutter create`).

Qué problema resuelve

Un código QR es una instrucción opaca: nadie puede leer a simple vista si lleva a un banco o a una imitación del banco. El comportamiento habitual de un lector es resolver esa ambigüedad **abriendo el enlace de inmediato**.

Este proyecto invierte el orden. El flujo, verificable en `lib/models/scan_record.dart` y `lib/features/result/scan_result_sheet.dart`, es:

1. leer el código;
2. interpretarlo en campos con nombre (`ContentInterpreter`);
3. evaluarlo con 16 señales locales de riesgo (`ScanSecurityAnalyzer`);
4. mostrar el resultado;
5. **esperar una confirmación explícita** antes de abrir nada (`_openRecord` en `scan_result_sheet.dart` líneas 214-246).

Ninguna rama del código llama a `launchUrl` sin pasar antes por ese diálogo cuando `confirmBeforeOpen` está activo o el riesgo no es `RiskLevel.low` —**comprobado**: `launchUrl` aparece una sola vez en todo `lib/`, dentro de `_openRecord`.

Qué hace, en cinco capacidades

Capacidad	Dónde vive	Resumen
Leer	`lib/features/scanner/`	Cámara con marco de lectura, zoom, linterna, enfoque por toque, cambio de cámara y varios códigos por imagen. Además, lotes de hasta 20 imágenes de galería y 50 páginas de PDF
Interpretar	`lib/services/content_interpreter.dart`	20 ramas de reconocimiento que producen 17 tipos de contenido (`ContentKind`), desde una URL hasta un Swiss QR Bill
Evaluar	`lib/core/security/scan_security_analyzer.dart`	16 señales locales sobre la URL más el bloqueo de esquemas de aplicación no permitidos
Inventariar	`lib/features/inventory/`	Sesiones de conteo continuo con cantidad, notas y exportación

Generar	`lib/features/generator/`	10 simbologías de salida a PNG y SVG, s in enviar los datos a ningún servidor

Qué NO hace

Estas ausencias son deliberadas y están escritas en el código, no solo en la documentación:

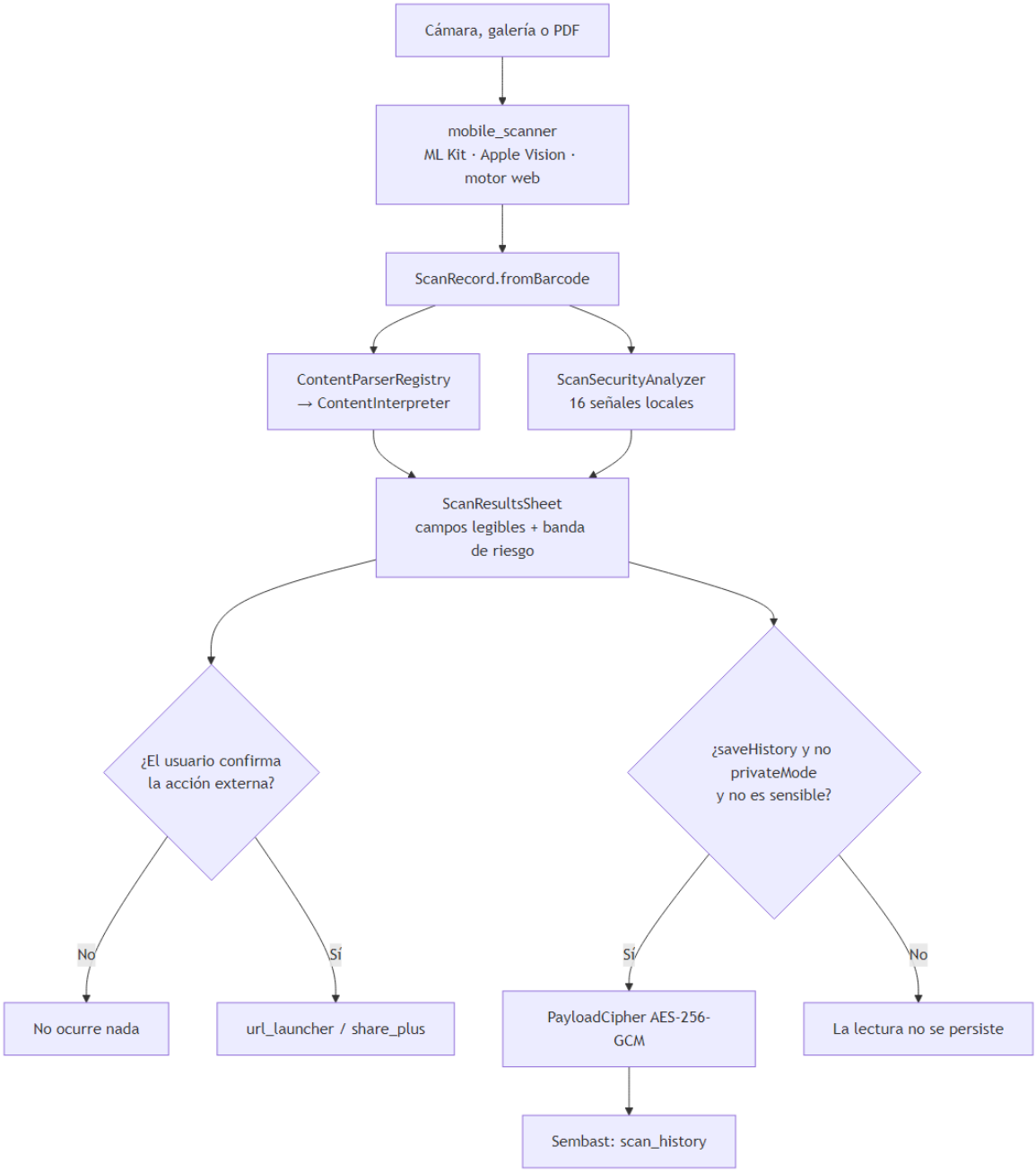
- **No hay red saliente propia.** El único paquete de red presente es `url_launcher`, que delega en el sistema operativo la apertura de un enlace que el usuario ya confirmó. No existe ningún cliente HTTP en `lib/` —**comprobado**: no hay `import 'dart:io'` con `HttpClient`, ni `package:http` entre las dependencias de `pubspec.yaml`. Ver 09-apis-and-integrations.md.
- **No hay cuentas, publicidad ni analítica.** Ninguna dependencia de telemetría en `pubspec.yaml`.
- **No consulta reputación remota de dominios.** `ScanSecurityAnalyzer` es íntegramente local y nunca declara que un enlace *sea seguro*: su nivel más bajo se llama `Sin señales evidentes` (`scan_security_analyzer.dart`, `SecurityAssessment.label`).
- **No hay firma de publicación.** Los APK publicados van firmados con la clave de depuración de Android; está declarado en `README.md` y en `docs/RELEASE.md`.
- **No hay motor nativo para Windows ni Linux.** `SUPPORTED_PLATFORMS` en `tool/bootstrap.py` es `("android", "ios", "web", "macos")`. Esas dos plataformas se cubren mediante la PWA.
- **La interfaz solo está en español.** `AppLocalizations.supportedLocales` contiene `es_CL` y `es`; el delegado rechaza `en` explícitamente (`_AppLocalizationsDelegate.isSupported`). Ver `../quality/ACCESSIBILITY.md`.

Actores

Actor	Interactúa mediante	Puede
Persona que escanea	Las cinco pestañas de `HomeShell` (`lib/app.dart`)	Leer, interpretar, inventariar, generar, exportar, importar y borrar
Sistema operativo	Cámara, almacén seguro, biometría, portapapeles, selector de archivos, compartir	Conceder o denegar cada permiso; ejecutar la acción externa confirmada
Aplicación externa	`url_launcher` y `share_plus`	Recibir un enlace, un contacto, un evento o un archivo exportado
Persona que publica	`tool/`, `.github/workflows/`	Generar plataformas, validar, compilar, firmar SBOM y publicar

No existe un actor «servidor»: no hay backend en este repositorio ni fuera de él.

Flujo principal, de extremo a extremo



Lo que el diagrama muestra: que la interpretación y el análisis de riesgo ocurren *antes* de que exista cualquier posibilidad de acción, y que la persistencia es una rama independiente de la acción.

Lo que el diagrama no muestra: el inventario continuo, que tiene su propio camino (`InventoryScreen._onDetect` → `InventoryStore.addScan`) y **no aplica el filtro de contenido sensible** —ver el hallazgo R-03 de 15-risks-and-technical-debt.md. Tampoco muestra el modo temporal, ni el Centro de recuperación, ni la degradación del tono de lectura. Esos caminos están en 08-data-flow.md.

Tecnologías

Capa	Tecnología	Versión fijada
Lenguaje	Dart	`>=3.12.0 <4.0.0` (`pubspec.yaml`)
Framework	Flutter	`>=3.44.0`; el toolchain se fija en `3.44.7` (`.fvmrc`)

Motor de lectura	`mobile_scanner`	`7.4.0`
Persistencia	`sembast` / `sembast_web`	`3.8.9+1` / `2.4.5+1`
Cifrado	`cryptography` (AES-GCM 256)	`2.9.0`
Almacén de llaves	`flutter_secure_storage`	`10.3.1`
Preferencias	`shared_preferences`	`2.5.5`
Generación de códigos	`barcode_widget`	`2.0.4`
PDF	`pdfrx`	`2.4.5` (anclado)
Hojas de cálculo	`excel`	`4.0.6` (anclado)
Compartir	`share_plus`	`12.0.2` (anclado)
Audio	`audioplayers`	`6.8.1`
Biometría	`local_auth`	`3.0.2`

Las tres versiones marcadas como *ancladas* no pueden subirse sin romper la resolución de dependencias; el motivo exacto está en `../quality/LOCKFILE.md` y replicado en `.github/dependabot.yml`.

Plataformas y su grado de verificación

Plataforma	Estado declarado en 1.1.0	Evidencia
Android (API 24+)	APK compilado y ejecutado en emulador API 36	`VALIDATION.md`, `RELEASE_SOURCE.json`
Web / PWA	Compila y se ejecuta en release; publicada en GitHub Pages	`.github/workflows/deploy-landing.yml`
iOS 15.5+	Compila en CI con `--no-codesign`; sin ejecución verificada	Trabajo `apple` de `flutter-ci.yml`
macOS	Compila en CI; sin ejecución verificada	Trabajo `apple` de `flutter-ci.yml`
Windows · Linux	Sin motor nativo; se cubren vía PWA	`SUPPORTED_PLATFORMS` en `tool/bootstrap.py`

El mínimo de Android se fija en `minSdk = 24` por sustitución de texto en `tool/bootstrap.py` (función `patch_android`), y el de iOS en `15.5` en tres archivos distintos (`Podfile`, `project.pbxproj`, `AppFrameworkInfo.plist`).

Una decisión que explica el resto del diseño

El repositorio **no versiona las carpetas nativas**. `.gitignore` excluye `/android/`, `/ios/`, `/web/` y `/macos/`, y `tool/bootstrap.py` las regenera de forma reproducible aplicando permisos, `minSdk`, entitlements, descripciones de uso, la cadena de compilación de Gradle y los iconos.

La consecuencia práctica —y la sorpresa número uno de quien clona— es que **`flutter run` no funciona sobre un clon recién hecho**: primero hay que ejecutar `python tool/bootstrap.py`. El procedimiento completo está en 02-installation-and-execution.md.

La consecuencia de diseño es que el repositorio contiene solo fuente propia: 171 archivos versionados, de los cuales 150 entran en `SOURCE_MANIFEST.json` con su tamaño y su SHA-256.

Para una persona no técnica

Imagina que recibes un sobre cerrado con una instrucción escrita dentro. La mayoría de los lectores de códigos QR abre el sobre y **ejecuta la instrucción sin enseñártela**: si dentro decía «ve a esta página web», te lleva allí antes de que puedas mirar. Esto funciona muy bien hasta el día en que la instrucción la escribió alguien que no querías obedecer.

Esta aplicación hace lo contrario. Abre el sobre, **te lee la instrucción en voz alta y en tu idioma** —«esto es una dirección web; el dominio es este; el protocolo es este; la ruta es esta»— y después se queda quieta esperando a que tú decidas. Si algo en la instrucción le parece raro, te lo dice antes: que el nombre del sitio usa letras de otro alfabeto que se parecen a las nuestras, que el enlace está acortado y esconde su destino, que la dirección lleva un usuario y una contraseña pegados delante, o que lo que va a descargarse parece un programa ejecutable.

Todo esto ocurre **dentro de tu teléfono**. No hay ninguna consulta a internet para decidir si algo es sospechoso: la aplicación no le pregunta a nadie, mira la propia instrucción y saca conclusiones. Eso significa que nadie —ni siquiera quien hizo la aplicación— sabe qué códigos escaneas.

Además guarda un historial, **pero cifrado**: si alguien consiguiera copiar el archivo del teléfono, vería ruido. La llave para descifrarlo vive en la caja fuerte del sistema operativo, la misma donde el teléfono guarda las contraseñas. Y hay dos cosas que la aplicación decide **no** guardar nunca por su cuenta: las contraseñas de redes Wi-Fi y los códigos de verificación en dos pasos. Son demasiado sensibles para quedarse en un historial que quizá olvides que existe.

También sirve para lo contrario: **fabricar** un código. Puedes crear un QR con tu tarjeta de contacto, con la contraseña del Wi-Fi de tu casa o con un enlace, y guardarlo como imagen. Nada de lo que escribes sale del teléfono mientras se dibuja.

Y sirve para contar cosas. Si tienes que inventariar una bodega, abres una sesión y vas escaneando: cada lectura suma una unidad, puedes corregir cantidades a mano, añadir notas y exportar la lista a Excel.

Lo que esta aplicación no puede hacer

Conviene decirlo con la misma claridad:

- **No es un antivirus.** Puede avisarte de que un enlace tiene señales sospechosas, pero no puede saber si una página concreta es una estafa. Un enlace perfectamente normal a un sitio recién creado le parecerá normal.
- **No garantiza que un enlace sea seguro.** Su mensaje más tranquilizador dice «sin señales evidentes», y eso es exactamente lo que significa: no encontró nada raro *en el texto del enlace*.
- **No lee cualquier código.** Los códigos privados, cifrados o sin especificación pública no se pueden interpretar, y la aplicación lo dice en lugar de inventar.
- **Todavía no se ha probado en teléfonos reales de forma sistemática.** Se instaló y arrancó en un emulador de Android; la prueba en dispositivos físicos sigue declarada como pendiente.
- **Si desinstalas la aplicación, el historial se pierde para siempre.** Está cifrado con una llave que también se borra. No hay copia en ningún servidor porque no hay servidor. Si te importa conservarlo, expórtalo antes.
- **Los archivos que exportas ya no están cifrados.** Un CSV o un Excel con tu historial es un archivo normal: quien lo abra, lo lee.

Continúa en 02 · Instalación y ejecución o salta al 17 · Resumen ejecutivo.

02 · Instalación y ejecución

Lo primero: un clon no compila tal cual

Es el hecho de primer orden de este repositorio y conviene leerlo antes que nada. `.gitignore` contiene:

```
/android/
/ios/
/web/
/macos/
```

con el comentario `# Se generan con tool/bootstrap.py`. Las carpetas de plataforma **no están versionadas**. `flutter run` sobre un clon recién hecho falla porque no existe ningún proyecto nativo al que apuntar.

El paso obligatorio es:

```
python tool/bootstrap.py --platforms android,web
```

Ese script hace tres cosas, en este orden (`tool/bootstrap.py`, función `main`):

1. **Genera** las carpetas ausentes ejecutando `flutter create` en un directorio temporal y copiando el resultado (`generate_platforms`);
2. **Parchea** cada plataforma generada (`patch_android`, `patch_ios`, `patch_macos`, `patch_web`);
3. **Resuelve** dependencias con `flutter pub get`, salvo que se pase `--skip-pub-get`.

Requisitos

Requisito	Versión	Cómo se comprueba
Flutter	**3.44.7**	Fijado en `../../.fvmrc` y en la variable `FLUTTER_VERSION` de los dos workflows
Dart	`>=3.12.0 <4.0.0`	`environment.sdk` de `pubspec.yaml`; vie ne con el propio Flutter
Python	3 (la CI usa 3.12)	`actions/setup-python@v7` en `.github/ workflows/flutter-ci.yml`
`pyyaml`	cualquiera reciente	La CI hace `python3 -m pip install - -upgrade pyyaml`; lo importan `tool/val idate_structure.py`, `tool/generate_s bom.py` y `tool/bootstrap.py` no
JDK y Android SDK	Los que exija la cadena AGP 8.11.1 / Gra dle 8.14.3	Solo para compilar Android
Xcode	Solo en macOS	Solo para iOS y macOS
Pillow	opcional	Únicamente para **regenerar** los iconos c on `tool/generate_launcher_icons.py`. Compilar no lo necesita

Comprobado: `tool/bootstrap.py` no importa `yaml`; los que sí lo hacen son `tool/validate_structure.py` (línea 11) y `tool/generate_sbom.py` (línea 302).

Sobre `.fvmrc` y la versión exacta

`.fvmrc` contiene únicamente:

```
{
  "flutter": "3.44.7"
}
```

El repositorio **no incluye la configuración de FVM más allá de ese archivo**: no hay `.fvm/`, ni script que lo consuma, ni paso de CI que lo lea —la CI fija la versión con `subosito/flutter-action@v2` y su propia variable de entorno. En la práctica `.fvmrc` funciona como declaración de la versión de referencia. Si usas FVM, `fvm use` la respetará; si no, basta con tener 3.44.7 en el `PATH`.

Una versión distinta de Flutter **no está bloqueada por ningún gate**: nada en `tool/validate_structure.py` compara la versión instalada con `.fvmrc`. *Inferencia*: compilar con 3.44.6 o 3.45.x probablemente funcione, pero `pubspec.lock` fue resuelto con 3.44.7 y las plantillas de `flutter create` cambian entre versiones menores, así que el resultado de `bootstrap.py` puede diferir. **Requiere validación.**

Instalación paso a paso

1 · Clonar y resolver dependencias

```
git clone https://github.com/vladimiracunadev-create/universal-code-scanner.git
cd universal-code-scanner
flutter pub get
```

`pubspec.lock` **está versionado** (es una aplicación, no un paquete; la política está en `../quality/LOCKFILE.md`), así que `pub get` reproduce exactamente el grafo con el que se validó la versión.

`flutter pub outdated` mostrará ~30 paquetes con versiones más nuevas. **Tres de ellos no deben subirse**: `share_plus`, `pdfrx` y `excel`. El motivo está en el propio `.github/dependabot.yml`, que los ignora con la razón escrita al lado.

2 · Generar las plataformas

```
python tool/bootstrap.py --platforms android,web
```

En Windows existe un envoltorio equivalente:

```
.\tool\bootstrap.ps1 --platforms android,web
```

`tool/bootstrap.ps1` prefiere el lanzador `py -3` y cae a `python` si no existe. En POSIX, `tool/bootstrap.sh` hace lo mismo con `python3`.

Sin `--platforms`, la lista por defecto depende del sistema operativo (`default_platforms`): en macOS son `android, ios, web, macos`; en cualquier otro, `android, web`.

Opciones:

Opción	Efecto
`--platforms a,b,c`	Lista explícita. Valores válidos: `android`, `ios`, `web`, `macos`. Cualquier otro aborta con `Plataformas no válidas`
`--force`	Borra y regenera las carpetas aunque ya existan

`--skip-pub-get`	Omite el `flutter pub get` final

Detalle que sorprende: si la carpeta ya existe y no se pasa `--force`, el script **no la regenera pero sí vuelve a parchearla**. Las funciones `patch_*` se ejecutan siempre que la carpeta exista (`main`, líneas 388-395). Los parches son idempotentes: `insert_before` comprueba si el contenido ya está presente antes de insertarlo, y las sustituciones de texto son estables al repetirlas.

3 · Verificar

```
python tool/validate_structure.py
flutter analyze --fatal-infos
flutter test
```

4 · Ejecutar

```
flutter run                 # dispositivo o emulador conectado
flutter run -d chrome       # PWA en local, sin --base-href
```

Qué aplica exactamente `bootstrap.py` a cada plataforma

Esta tabla es el contenido real de las funciones `patch_*`. Es la única documentación de esos valores: los archivos resultantes no se versionan, así que **no se pueden leer en el repositorio**.

Android (`patch_android`)

Cambio	Valor	Por qué
`uses-permission`	`android.permission.CAMERA`	Lectura con cámara
`uses-permission`	`android.permission.USE_BIOMETRIC`	Bloqueo de la aplicación
`uses-feature`	`android.hardware.camera.any` con `required="false"`	Permite instalar en dispositivos sin cámara
`android:allowBackup`	`false`	Evita que una llave del Keystore se restaure sin su material criptográfico compatible (`docs/SECURITY.md`)
`android:label`	`Universal Code Scanner`	La plantilla genera `universal_code_scanner`
`MainActivity`	`FlutterFragmentActivity` en vez de `FlutterActivity`	Requisito de `local_auth` para el diálogo biométrico
`minSdk`	`24`	Sustituye `flutter.minSdkVersion`
`LaunchTheme`	`Theme.AppCompat.DayNight.NoActionBar`	En `values/` y `values-night/`
Android Gradle Plugin	`8.11.1`	Ver el bloque siguiente
Kotlin Gradle Plugin	`2.2.20`	Mínimo que Flutter 3.44.7 no marca obsoleto
Distribución de Gradle	`8.14.3`	Compatible con AGP 8.11.1

`android.builtInKotlin`	`false`	Coherente con AGP 8
Iconos	Copiados de `assets/launcher/android/`	Heredados, adaptativos y monocromos

El anclaje de AGP tiene una explicación concreta escrita como comentario en el propio script (líneas 31-39): la plantilla de Flutter 3.44.7 genera AGP 9, cuyo valor por defecto es el Kotlin integrado; varios plugins de este conjunto —`file_picker` entre ellos— **dejan de aplicar el Kotlin Gradle Plugin en cuanto detectan AGP 9**, dando por hecho que el integrado está activo. Pero la misma plantilla entrega `android.builtInKotlin=false`, así que nadie compila el Kotlin de esos plugins y la compilación falla con `cannot find symbol: class FilePickerPlugin`. Activar el Kotlin integrado rompe el caso contrario. La serie 8 de AGP deja a todos los plugins en la ruta que todos soportan.

iOS (`patch_ios`)

Cambio	Valor
`platform :ios` del `Podfile`	`15.5`
`IPHONEOS_DEPLOYMENT_TARGET`	`15.5`
`MinimumOSVersion` de `AppFrameworkInfo.plist`	`15.5`
`NSCameraUsageDescription`	«La cámara se utiliza para leer códigos QR y códigos de barras.»
`NSPhotoLibraryUsageDescription`	«La galería se utiliza para analizar códigos presentes en imágen es seleccionadas.»
`NSFaceIDUsageDescription`	«Face ID protege el acceso a la aplicación cuando el usuario act iva el bloqueo.»
`Runner.entitlements`	`keychain-access-groups` vacío
`CODE_SIGN_ENTITLEMENTS`	Añadido al `project.pbxproj` junto a `CODE_SIGN_STYLE`

macOS (`patch_macos`)

Añade `NSCameraUsageDescription` y `NSPhotoLibraryUsageDescription` al `Info.plist`, y a **ambos** archivos de entitlements (`DebugProfile.entitlements` y `Release.entitlements`) las tres claves `com.apple.security.device.camera`, `com.apple.security.files.user-selected.read-only` y `keychain-access-groups`.

Web (`patch_web`)

Reescribe `web/manifest.json` con nombre, nombre corto, descripción, `display: standalone` y los colores `#006B66`; cambia el `<title>` y el `content` del `index.html`; y copia los iconos y el favicon desde `assets/launcher/web/`.

Comandos agrupados

El repositorio ya trae dos guiones que encadenan lo anterior.

`tool/run_quality_gate.sh` — puerta de calidad

```
python3 tool/validate_structure.py
flutter pub get
flutter analyze
flutter test --coverage
python3 tool/generate_sbom.py
python3 tool/generate_license_inventory.py
flutter pub outdated || true
```

Nota: usa `flutter analyze` **sin** `--fatal-infos`, a diferencia de la CI y de `README.md`. Ver el hallazgo R-06 de 15-risks-and-technical-debt.md.

`tool/finalize_stable.sh` — antes de etiquetar

Añade a lo anterior `flutter build apk --debug`, `flutter build web --release`, `validate_structure.py --require-lock`, `generate_source_manifest.py` y un `git diff -- pubspec.lock pubspec.yaml` final. Ambos guiones son `bash` y requieren `flutter` en el `PATH`; **no existe un equivalente PowerShell**.

Ejecución de las pruebas

```
flutter test                  # 57 casos en test/
flutter test --coverage       # además escribe coverage/lcov.info
```

La prueba de integración **no la ejecuta `flutter test`**: vive en `integration_test/` y necesita un dispositivo.

```
flutter test integration_test/app_launch_test.dart -d macos
```

La CI la omite a propósito, con el motivo escrito al final de `.github/workflows/flutter-ci.yml`: los runners macOS de GitHub no tienen sesión gráfica interactiva, así que la aplicación arranca sin llegar a establecer la conexión de depuración.

Compilaciones

Objetivo	Comando	Verificado en 1.1.0
Web release	`flutter build web --release`	Sí, local y CI
APK de depuración	`flutter build apk --debug`	Sí, CI
APK por arquitectura	`flutter build apk --release --split-per-abi`	Sí: 32,8 MB arm64-v8a · 26,8 MB armeabi-v7a · 35,3 MB x86_64
APK universal	`flutter build apk --release`	Sí: 88,1 MB
iOS sin firma	`flutter build ios --release --no-codesign`	CI, trabajo `apple`
macOS	`flutter build macos --release`	CI, trabajo `apple`
App Bundle (`.aab`)	`flutter build appbundle`	**No**: fuera del alcance de 1.1.0

Los tamaños provienen de `RELEASE_SOURCE.json`, campo `runtimeValidation.androidReleaseApk`.

Verificación ejecutada en este análisis

Sobre el commit `c1f9878`, con Flutter 3.44.7 y Dart 3.12.2:

Comando	Salida real
`flutter pub get`	`Got dependencies!` · `30 packages have newer versions in compatible with dependency constraints.`
`flutter analyze --fatal-infos`	`No issues found! (ran in 100.8s)` · código de salida `0`
`flutter test`	`00:24 +57: All tests passed!` · código de salida `0`

Problemas frecuentes al instalar

Síntoma	Causa	Solución
`flutter run` no encuentra dispositivo Android ni proyecto	Falta ejecutar `bootstrap.py`	`python tool/bootstrap.py --platforms android,web`
`cannot find symbol: class FilePickerPlugin`	Se está usando AGP 9 en vez del 8.11.1 que fija el script	Volver a ejecutar `bootstrap.py`, que reescribe `settings.gradle.kts`
`Falta pubspec.lock` al validar	Se ejecutó `validate_structure.py --require-lock` sin haber hecho `pub get`	`flutter pub get`
`ModuleNotFoundError: No module named 'yaml'`	Falta `pyyaml`	`python -m pip install --upgrade pyyaml`
`Ruta absoluta del entorno encontrada en …` al validar	Un archivo contiene una de las tres rutas de entorno que el validador prohíbe: dos directorios de trabajo de entornos de ejecución remotos y el prefijo del directorio de perfiles de Windows	Quitar la ruta; el validador la considera contaminación del entorno de origen. La lista literal está en la variable `bad` de `check_absolute_paths`
`flutter pub outdated` propone subir `share_plus`, `pdfrx` o `excel`	Anclajes deliberados	No subirlos; ver `../quality/LOCKFILE.md`

Más casos, ya en ejecución y no en instalación, en 14-troubleshooting.md.

Continúa en 03 · Arquitectura.

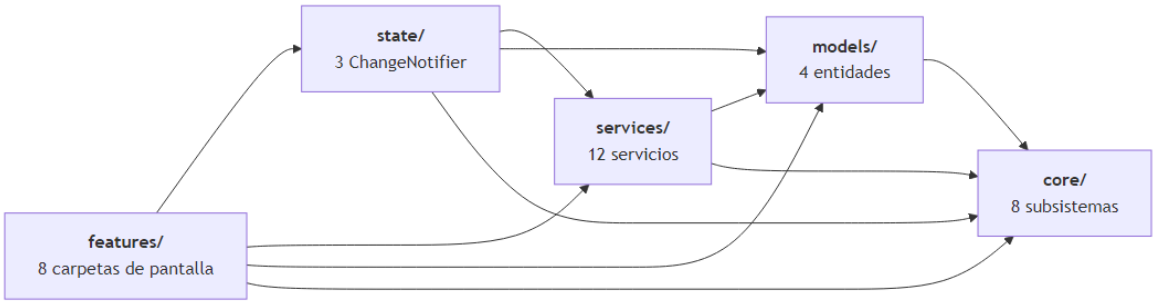
03 · Arquitectura

La conclusión primero

Universal Code Scanner es una **aplicación de un solo proceso, sin red propia, con estado en memoria y persistencia cifrada local**. Su arquitectura se apoya en cinco capas de dependencia unidireccional y en tres fronteras de extensión declaradas (`ScannerEngine`, `ContentParserRegistry`, `FeatureFlags`), de las cuales **solo dos están realmente conectadas al flujo de ejecución**: la tercera, `FeatureFlags`, se persiste y se restaura pero ningún código la consulta. Ver el hallazgo R-02 de 15-risks-and-technical-debt.md.

No hay inyección de dependencias por contenedor ni gestor de rutas: los servicios se construyen una vez en `AppBootstrapper.initialize` y se pasan por constructor hasta las pantallas. La navegación es un índice entero (`_HomeShellState._index`) sobre una lista de cinco widgets.

Las cinco capas



Capa	Contenido
`features/`	`scanner` · `inventory` · `generator` · `history` · `settings` · `result` · `recovery` · `formats`
`state/`	`ScanStore` · `InventoryStore` · `SettingsStore`
`services/`	`ContentInterpreter` · `ImportService` · `ExportService` · `HistoryRepository` · `InventoryRepository` · `SettingsRepository` · `ScanFeedback` · `BiometricService` · `ClipboardService` · `PdfPageRenderer`
`models/`	`ScanRecord` · `InventorySession` · `ParsedContent` · `AppSettings`
`core/`	`database` · `security` · `recovery` · `performance` · `theme` · `localization` · `diagnostics` · `feature_flags`

Lo que muestra: la dirección general de las dependencias. Ningún archivo de `lib/core` ni de `lib/models` importa `lib/state`, y ningún archivo de `lib/core` importa `lib/services` ni `lib/features` —comprobado con `grep -rn "import 'package:universal_code_scanner/state" lib/core lib/models lib/services`, que no devuelve nada.

Lo que no muestra: las dos excepciones reales, ambas verificadas por `grep`.

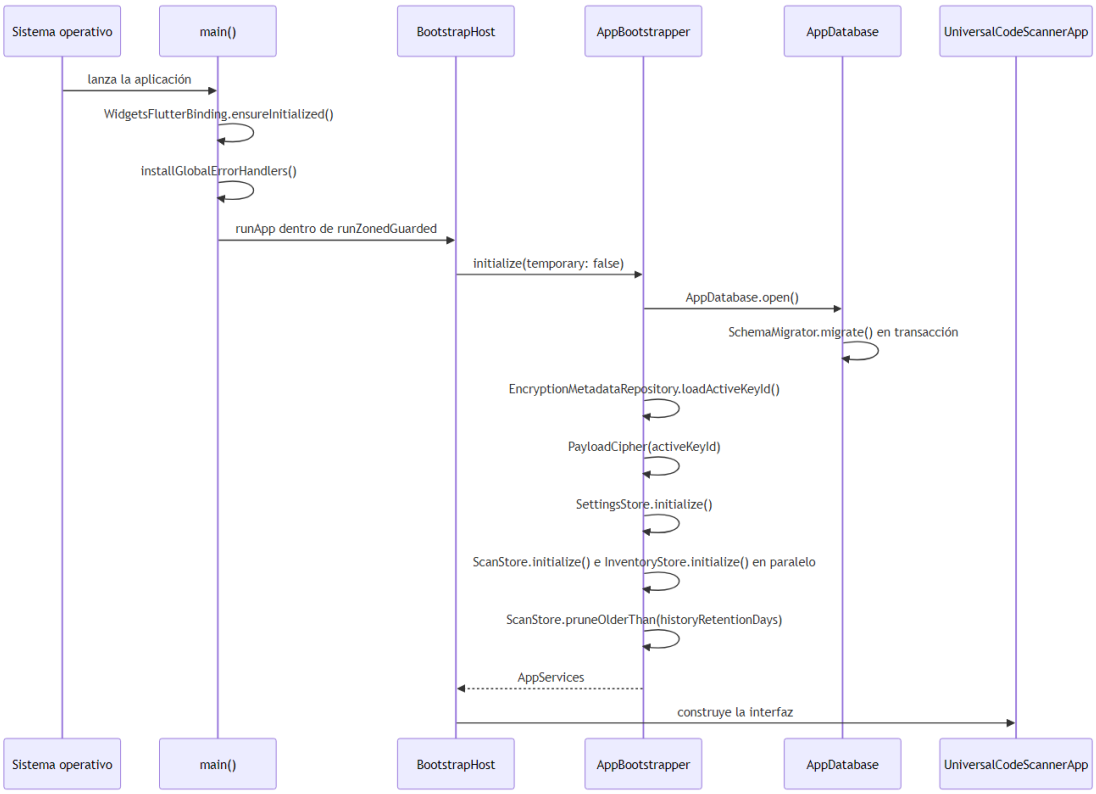
1. `lib/models/scan_record.dart` (línea 9) importa `lib/features/formats/domain/content_parser_registry.dart` —es decir, un modelo importa una carpeta de `features`. El registro de parsers está físicamente en `features/formats/domain/` pero conceptualmente pertenece a `services`. Es una inconsistencia de ubicación, no un ciclo: `content_parser_registry.dart` solo

importa `ParsedContent` de `models`. Registrado como R-07.

2. `lib/core/recovery/recovery_service.dart` (líneas 8-9) importa `InventorySession` y `ScanRecord`, mientras que `lib/models/scan_record.dart` importa `core/security/scan_security_analyzer.dart` y `core/utils/barcode_labels.dart`. La arista entre `core` y `models` es por tanto **bidireccional a nivel de paquete**, aunque no haya ciclo entre archivos concretos. `RecoveryService` necesita los modelos para verificar que una carga descifrada es reconstruible antes de reinsertarla (`ScanRecord.fromJson(clear)` en `retry`), lo cual convierte un fallo de forma en un `false` en vez de en un registro corrupto reinsertado.

Tampoco muestra que `features/` contiene ocho subcarpetas, no cinco: además de las cuatro pestañas hay `result/` (la hoja de resultados, compartida por escáner e historial), `recovery/` (el Centro de recuperación, accesible desde Ajustes), `formats/` (la pantalla de formatos compatibles y el registro de parsers) y `settings/`.

El arranque, paso a paso



Si cualquier paso lanza, `AppBootstrapper` cierra la base de datos que hubiera abierto y relanza (`bootstrap.dart`, líneas 72-75). `_BootstrapHostState` captura ese error, lo registra en `AppDiagnostics` bajo el área `bootstrap` y construye `_StartupRecovery`, la pantalla de **inicio seguro**, con cuatro salidas:

Acción	Efecto
«Reintentar»	Repite `initialize(temporary: false)`
«Abrir sin historial ni inventarios persistentes»	`initialize(temporary: true)`: base Sembast **en memoria** y l lave también en memoria
«Restablecer preferencias visuales»	`SettingsRepository.resetNonSensitive()` y reintento
«Copiar diagnóstico privado»	`AppDiagnostics.exportJson()` al portapapeles

El texto bajo los botones enumera literalmente lo que el diagnóstico **no** incluye: «códigos escaneados, enlaces, contraseñas, secretos OTP, notas ni productos».

Modo temporal: qué cambia exactamente

`AppBootstrapper.initialize(temporary: true)` altera tres cosas y solo tres:

1. `AppDatabase.openTemporary()` abre una base `databaseFactoryMemory` con nombre `ucs_temporary_<microsegundos>` en lugar del archivo real;
2. `PayloadCipher` recibe un `MemoryEncryptionKeyProvider` en vez del almacén seguro, y el `activeKeyId` es la constante `PayloadCipher.currentKeyId` sin consultar la base;
3. `SettingsStore.initialize()` se envuelve en `try/catch`: **el modo temporal sigue disponible aunque las preferencias no puedan leerse**.

La interfaz lo señala con un `MaterialBanner` permanente sobre el contenido (`_HomeShellState.build`, línea 254) y con una entrada extra en Ajustes. Además, la rotación de llave se deshabilita en modo temporal (`SettingsScreen`, `enabled: !widget.temporaryMode && !_rotatingKey`).

Gestión del estado

El patrón es `ChangeNotifier` + `AnimatedBuilder`/`addListener`. No hay Riverpod, Bloc, Provider ni GetIt —**comprobado**: ninguno aparece en `pubspec.yaml`.

Store	Notifica cuando	Lo consume
`SettingsStore`	Cambia cualquier preferencia	`UniversalCodeScannerApp` vía `AnimatedBuilder`; todas las pantallas leen `.value` directamente
`ScanStore`	Se carga, importa, actualiza, poda, elimina o limpia el historial	`HistoryScreen` (`addListener`), `SettingsScreen` (lectura directa)
`InventoryStore`	Cambia cualquier sesión o ítem	`InventoryScreen` (`addListener`)

Consecuencia de diseño: **el tema y el idioma se recalculan en el nivel más alto del árbol**, porque `AnimatedBuilder` sobre `settingsStore` envuelve al `MaterialApp` completo. Cambiar «Alto contraste» reconstruye toda la aplicación. Es correcto para un ajuste que afecta al `ThemeData`, y es la razón por la que `SettingsScreen` no necesita escuchar el store: su propio `setState` no existe, la reconstrucción llega desde arriba.

Serialización de escrituras

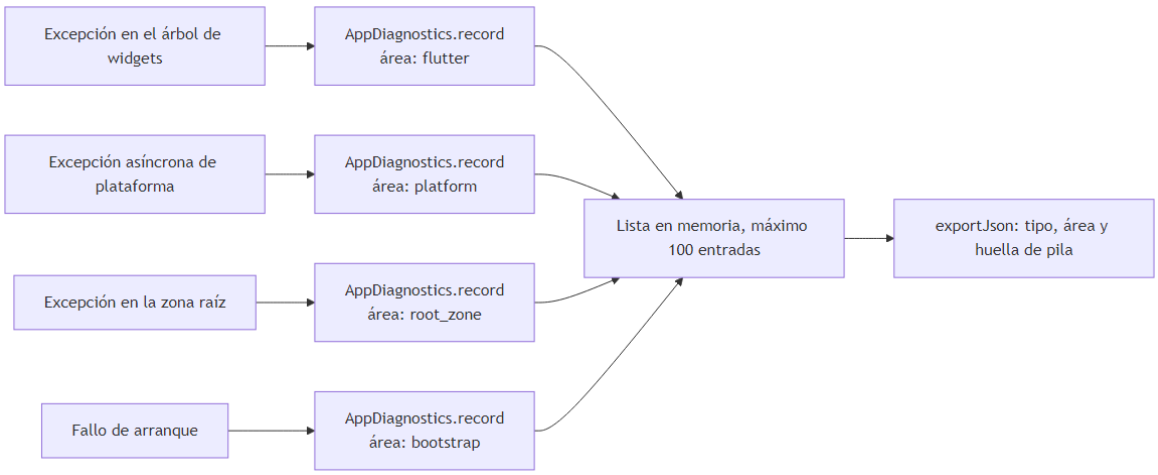
`InventoryStore` protege **todas** sus mutaciones con una cola serial (`AsyncWriteQueue`, `lib/core/performance/async_write_queue.dart`). Nueve métodos pasan por `_writes.run<void>(...)`. La razón: el inventario recibe lecturas continuas de la cámara mientras el usuario edita cantidades y notas, y dos escrituras concurrentes sobre el mismo `InventorySession` inmutable se pisarían.

`ScanStore` **no** usa la cola. Sus escrituras vienen de acciones puntuales del usuario o de un lote ya cerrado, no de un flujo continuo. Es una asimetría deliberada pero **no documentada en el repositorio**.

La cola tiene una propiedad que merece nombrarse: **un fallo no envenena la cola**. `AsyncWriteQueue.run` captura la excepción dentro del `then` y la entrega al `Completer` del llamante, de modo que la cadena `_tail` siempre resuelve. Hay una prueba dedicada a ese comportamiento (`test/core/async_write_queue_test.dart`, «a failed write does not block later writes»).

Manejo de errores

Hay cuatro niveles, y ninguno de ellos escribe nunca la carga escaneada.

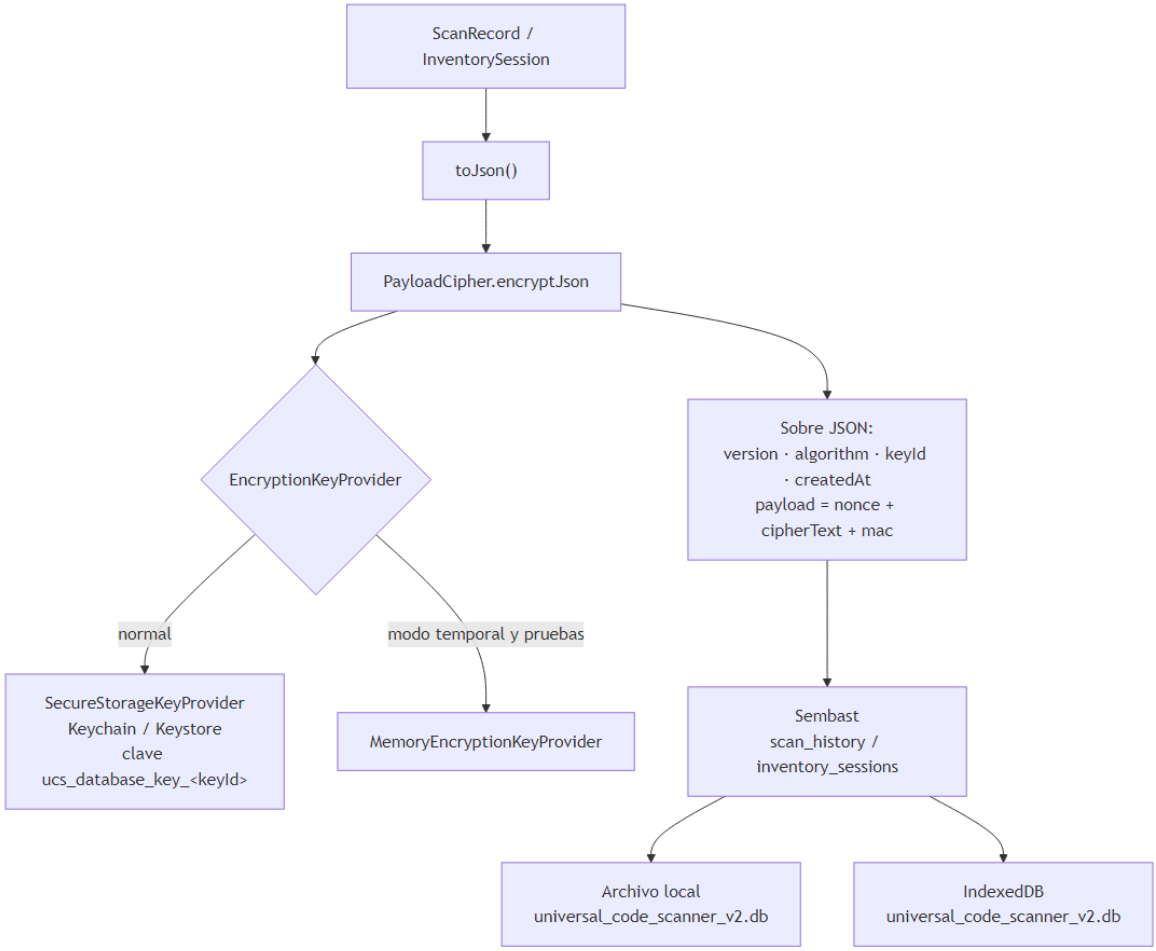


`AppDiagnostics.record` guarda exactamente cuatro campos: instante UTC, área saneada, **nombre del tipo** del error y una huella hexadecimal de 8 dígitos calculada sobre `runtimeType` más las cuatro primeras líneas de la pila. El mensaje del error **no se guarda**, y hay una prueba que lo verifica pasando un `StateError` cuyo mensaje contiene `password=`, `otp=` y un dominio, y comprobando que ninguno aparece en la exportación (`test/core/diagnostics_test.dart`).

`_sanitize` reemplaza cualquier carácter fuera de `[a-zA-Z0-9_.-]` por `_` y recorta a 80 caracteres. La lista se recorta a `maxEntries = 100` insertando siempre al principio, de modo que se conservan los 100 errores **más recientes**.

Los diagnósticos **viven solo en memoria**: no hay ninguna escritura a disco. Se pierden al cerrar la aplicación. *Inferencia*: es coherente con la política de privacidad, pero significa que un fallo de arranque intermitente no deja rastro entre sesiones.

Persistencia y cifrado



Cada registro persistido guarda **tres campos en claro** —`id`, `scannedAt` (o `createdAt`) y `payload`— y todo lo demás dentro del `payload` cifrado. Los dos campos en claro existen porque Sembast necesita ordenar y filtrar sobre ellos: `Finder(sortOrders: [SortOrder('scannedAt', false)])` en `HistoryRepository.load` y `Filter.lessThan('scannedAt', cutoff)` en `pruneOlderThan`.

La consecuencia, que conviene tener presente en una auditoría: **el momento de cada lectura es visible sin la llave**. El contenido no.

El detalle completo del esquema, los sobres y la rotación está en 07-database.md.

Fronteras de extensión

Frontera	Archivo	Implementaciones en 1.1.0	¿Conectada?
`ScannerEngine`	`lib/features/scanner/domain/scanner_engine.dart`	Una: `MobileScannerEngine`	Sí, pero **solo en `ScannerScreen`**
`ContentParser` / `ContentParserRegistry`	`lib/features/formats/domain/`	Una: `LegacyContentParser`, prioridad `-1000`	Sí
`FeatureFlags`	`lib/core/feature_flags/feature_flags.dart`	Nueve banderas, todas `false`	**No**

`ScannerEngine`

Interfaz de ocho miembros que envuelve `MobileScannerController`. Su objetivo declarado en `docs/ARCHITECTURE.md` es permitir un segundo motor —por ejemplo sobre ZXing-C++ para Windows y Linux— sin

tocar `ScanRecord`, `ContentInterpreter` ni los repositorios.

Hallazgo: `InventoryScreen` no usa `ScannerEngine`. Construye su propio `MobileScannerController` en `_createController()` y llama al controlador directamente (`_controller.start()`, `_controller.toggleTorch`, …). Un segundo motor requeriría también reescribir esa pantalla. Registrado como R-04.

`ContentParserRegistry`

Singleton con una lista ordenada por prioridad descendente. `register` sustituye por `id` y reordena; `unregister` elimina por `id` **salvo `builtin-v2`**, que no puede quitarse (`content_parser_registry.dart`, línea 18).

`parse` recorre la lista y devuelve el primero cuyo `canParse` acepte. Como `LegacyContentParser.canParse` devuelve siempre `true` y no puede desregistrarse, **la línea de reserva final `return ContentInterpreter.parse(rawValue)` es inalcanzable**. Es defensa redundante, no un error de comportamiento; registrada como R-08.

`FeatureFlags`

Nueve banderas: `experimentalParsers`, `secondaryScannerEngine`, `ocr`, `nfc`, `urlReputation`, `productLookup`, `encryptedSync`, `enterpriseApi`, `kioskMode`. Se serializan a la preferencia `feature_flags` y se restauran al arrancar. **Comprobado con `grep` sobre las nueve**: ninguna se lee fuera de `feature_flags.dart`. No hay condicional en `lib/` que dependa de una bandera, ni control en Ajustes para cambiarlas. Son, en 1.1.0, una declaración de intención persistida. R-02.

Ciclo de vida de la cámara

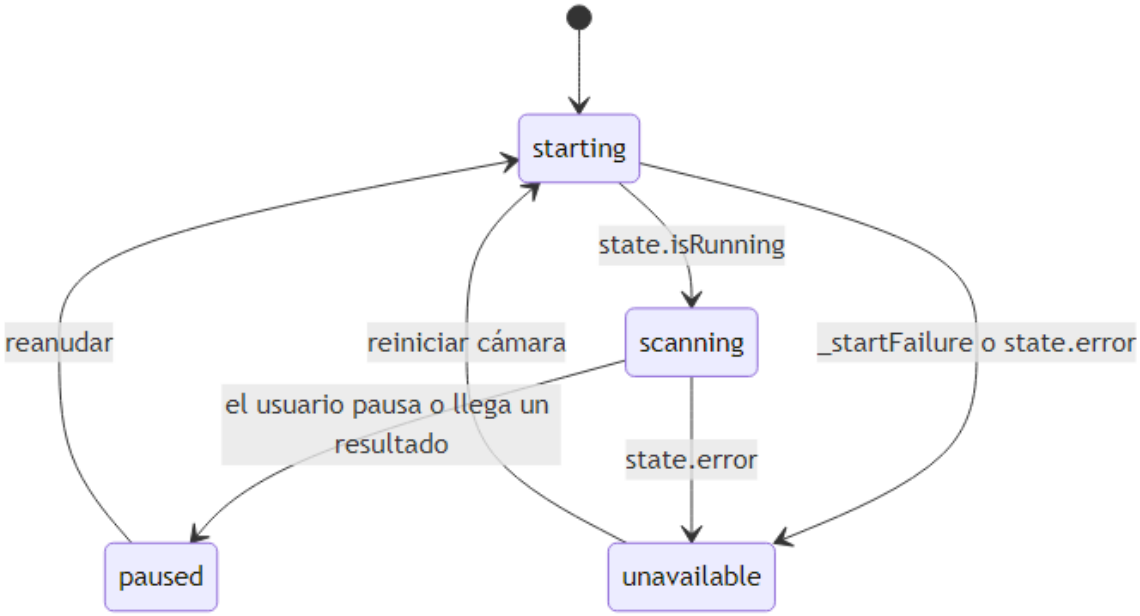
Es la parte más delicada del código y la que originó la versión 1.1.0. El resumen técnico:

`MobileScanner` **solo gestiona el ciclo de vida cuando él mismo crea el controlador**. Estas dos pantallas le pasan el suyo, así que la gestión les corresponde. Ambas implementan `WidgetsBindingObserver`:

Estado del ciclo de vida	`ScannerScreen`	`InventoryScreen`
`resumed`	`_startCamera()` si no está en pausa ni mostrando resultado	`_startCamera()` si hay sesión activa y n o está en pausa
`inactive`	`_stopCamera()`	`_stopCamera()`
`hidden`, `paused`, `detached`	Nada	Nada

Y ambas ofrecen «Reiniciar cámara», que **libera el controlador antiguo antes de crear el nuevo**. El orden importa y está comentado en el código (`scanner_screen.dart`, líneas 120-122): ambos comparten una única sesión de cámara de plataforma, y desechar el viejo *después* derribaría la sesión que el nuevo acaba de reclamar.

El estado visible se reduce a cuatro fases (`ScanPhase`) derivadas del estado del controlador y de dos banderas locales:



`ScanStatusBar` nombra cada fase, y su barra horizontal **solo se mueve mientras el motor analiza cuadros de verdad** (`_ScanProgressBar`: `value: null` es la barra indeterminada; un valor fijo la dibuja quieta). Con movimiento reducido activado, la barra se dibuja llena y quieta y el marco no barre la línea. El razonamiento de producto completo está en `../quality/SCANNER_UX.md`.

Concurrencia y trabajo fuera del hilo de interfaz

Trabajo	Mecanismo	Archivo
Codificación CSV	`compute(_encodeCsv, rows)`	`export_service.dart`
Codificación XLSX	`compute(_encodeXlsx, request)`	`export_service.dart`
Escrituras de inventario	`AsyncWriteQueue` (serial, mismo isolate)	`inventory_store.dart`
Lotes de imágenes y PDF	Bucle con `await Future.delayed(Duration.zero)` entre elementos	`scanner_screen.dart`
Cifrado	Síncrono en el isolate principal, pero **pre calculado antes de la transacción**	`history_repository.dart`, `data_maintenance_service.dart`

El `Future.delayed(Duration.zero)` entre elementos de un lote cede el turno al bucle de eventos, de modo que el diálogo de progreso pueda repintarse y el botón «Cancelar» responda. No es paralelismo: **el análisis de 20 imágenes bloquea el isolate principal por tramos**.

El cifrado se precalcula fuera de la transacción de forma deliberada (`_migrateRecordsAtomically`, `upsertAll`, `replaceAll`, `rotateEncryptionKey`): AES-GCM sobre miles de registros dentro de una transacción de Sembast mantendría la base bloqueada mucho más tiempo del necesario.

Cancelación

`CancellationToken` es un `ChangeNotifier` con una bandera y dos métodos. Se propaga a dos niveles distintos en el caso del PDF:

1. el bucle del lote llama a `token.throwIfCancelled()` entre páginas;

2. el propio renderizador recibe un `PdfPageRenderCancellationToken` que se cancela mediante un *listener* registrado sobre el token de la aplicación (`pdf_page_renderer_io.dart`, líneas 48-53).

El *listener* se retira en un `finally` para no acumular suscripciones página a página. Al cancelar, `PdfPageRenderer.cleanup` borra las imágenes ya escritas y el directorio temporal, y el historial **no se modifica**: el mensaje al usuario lo dice literalmente («Análisis cancelado sin modificar el historial»).

Lo que está bien y conviene no romper

Un informe que solo enumera problemas da una imagen falsa. Estas propiedades del diseño son sólidas y alguien las romperá si nadie dice que importan:

1. **Ninguna acción externa sin confirmación.** `launchUrl` aparece una sola vez en todo `lib/`, y está detrás de un diálogo.
2. **Una llave ausente nunca se regenera en silencio.** `PayloadCipher.decryptJson` registra el `keyId` en `_missingKeyIds` y a partir de ahí **también bloquea el cifrado** con ese identificador. Sin esa doble barrera, un descifrado fallido seguido de una escritura generaría una llave nueva y dejaría los datos antiguos ilegibles para siempre. Hay prueba dedicada.
3. **La rotación de llave limpia su propia basura.** Si falla, el `catch` llama a `forgetKey(keyId)` para que la llave recién creada no pueda confundirse después con una llave activa. Hay prueba dedicada.
4. **La migración del historial verifica antes de borrar.** `_migrateRecordsAtomically` comprueba `actual.containsAll(expected)` **dentro de la transacción** y lanza si falta algo; solo después se elimina la clave antigua.
5. **La importación se valida completa antes de tocar la base.** Tamaño, profundidad, número de nodos, procedencia y versión de esquema se comprueban sobre el JSON decodificado; la vista previa se muestra y la estrategia se elige antes de escribir.
6. **El diagnóstico es estructuralmente incapaz de filtrar datos.** No acepta una cadena de mensaje: acepta un `Object` y guarda su `runtimeType`.
7. **El error de audio no puede romper una lectura.** `ScanFeedback` degrada al sonido del sistema y marca `_toneUnavailable` para no volver a intentarlo.

Continúa en 04 · Mapa completo del código.

04 · Mapa completo del código

Inventario jerárquico de todo el código fuente versionado. Cada elemento lleva ubicación, responsabilidad, dependencias, quién lo usa y **estado aparente**:

Estado	Significado
Activo	Participa en el flujo de ejecución de 1.1.0
Reservado	Existe y funciona, pero ningún camino de ejecución lo alcanza en 1.1.0
Sin uso	Declarado y nunca invocado
Plataforma	Se compila condicionalmente según la plataforma destino

Vista general del repositorio

Directorio	Archivos versionados	Contenido
`lib/`	59 `.dart` · 6 821 líneas	Todo el código de la aplicación
`test/`	19 `.dart` · 57 casos	Pruebas unitarias y de widget
`integration_test/`	1 `.dart` · 1 caso	Prueba de arranque, no la ejecuta `flutter test`
`tool/`	12 guiones (8 Python, 3 Bash, 1 PowerShell)	Generación, validación y empaquetado
`docs/`	18 `.md` previos + 2 SVG + 5 capturas + SBOM	Documentación anterior a este directorio
`landing/`	`index.html`, `style.css`, `favicon.svg`, `README.md`	Sitio publicado en GitHub Pages
`assets/`	`sounds/scan_success.wav` + iconos PNG	Recursos empaquetados y fuente de iconos
`test_assets/`	8 PNG + `manifest.json` + 7 `README.md`	Catálogo de regresión visual
`.github/`	2 workflows + `dependabot.yml`	Automatización
`android/`, `web/`	**No versionados**	Generados por `tool/bootstrap.py`

`lib/` — punto de entrada

`lib/main.dart` · 14 líneas · Activo

Aspecto	Detalle
Responsabilidad	Inicializar el binding, instalar los manejadores globales de error y lanzar `BootstrapHost` dentro de `runZonedGuarded`

Depende de	`bootstrap_host.dart`, `core/diagnostics/app_diagnostics.dart`
Lo usa	El sistema operativo
Nota	Los errores que escapan de la zona se registran con área `root_zone`

`lib/bootstrap.dart` · 77 líneas · Activo

Aspecto	Detalle
Clases	`AppServices` (contenedor inmutable de siete servicios), `AppBootstrapper` (`abstract final class` con un único método estático)
Responsabilidad	Construir el grafo completo de servicios en el orden correcto
Depende de	`core/database`, `core/recovery`, `core/security`, `services/*_repository`, `state/*`
Lo usa	`bootstrap_host.dart`
Riesgo al modificar	**Alto.** El orden importa: la llave activa debe leerse antes de construir `PayloadCipher`, y los stores antes de podar el historial

`lib/bootstrap_host.dart` · 133 líneas · Activo

Aspecto	Detalle
Widgets	`BootstrapHost` (`StatefulWidget`), `_StartupRecovery` (pantalla de inicio seguro)
Responsabilidad	Ejecutar el arranque, mostrar el indicador de progreso y, si falla, la pantalla de recuperación con cuatro salidas
Lo usa	`main.dart`
Nota	`_retry` guarda el fallo en `_failure` **y vuelve a lanzar**, de modo que el `FutureBuilder` también entra en estado de error

`lib/app.dart` · 277 líneas · Activo

Elemento	Tipo	Responsabilidad
`UniversalCodeScannerApp`	`StatelessWidget`	`MaterialApp` reconstruido por `AnimatedBuilder` sobre `SettingsStore`: tema, idioma y delegados
`HandheldFrame`	`StatelessWidget`	Limita la interfaz a `maxWidth = 560` en pantallas anchas. **Por debajo del umbral no envuelve nada**: el árbol en un móvil es idéntico al que habría sin esta capa
`BiometricLockGate`	`StatefulWidget` + `WidgetsBindingObserver`	Bloquea la aplicación al pasar a `inactive`, `paused`, `hidden` o `detached` si `biometricLock` está activo

`HomeShell`	`StatefulWidget`	Las cinco pestañas y el banner de modo t emporal

`HandheldFrame` tiene tres pruebas de widget dedicadas (`test/core/handheld_frame_test.dart`), incluida la del caso exacto del umbral.

`lib/core/` — infraestructura transversal

`core/app_info.dart` · 18 líneas · Activo

Dos constantes: `appVersion = '1.1.0'` y `appName = 'Universal Code Scanner'`.

Este archivo existe por un defecto real: el número de versión estaba escrito como literal en `settings_screen.dart` y se quedó en `1.0.0` al publicar 1.1.0. `tool/validate_structure.py::check_interface_version` compara ahora esta constante con `pubspec.yaml` y **falla la validación si divergen**. La documentación del propio archivo explica también por qué es una constante y no una consulta en tiempo de ejecución: leer los metadatos reales del paquete añadiría una dependencia de plugin para mostrar una cadena ya conocida al compilar.

`core/database/` · 5 archivos · Activo y Plataforma

Archivo	Líneas	Estado	Responsabilidad
`app_database.dart`	25	Activo	Abre la base persistente o un a en memoria; ejecuta `Schema Migrator`
`database_opener.dart`	3	Plataforma	`export` condicional por `dart. library.io` / `dart.library. js_interop`
`database_opener_io.dart`	11	Plataforma	`getApplicationSupportDir ectory()` + `universal_code _scanner_v2.db`, versión 2
`database_opener_web.dart`	5	Plataforma	`databaseFactoryWeb` con el mismo nombre lógico, versión 2
`database_opener_stub.dar t`	5	Plataforma	Lanza `UnsupportedError` co n mensaje en español
`schema_migrator.dart`	42	Activo	`currentVersion = 4`; migrac iones idempotentes en una tr ansacción

Detalle no obvio: hay **dos números de versión distintos**. Sembast recibe `version: 2` al abrir el archivo; `SchemaMigrator` mantiene su propio contador en el almacén `_schema_meta`, actualmente en `4`. No están relacionados. Ver 07-database.md.

`core/diagnostics/` · 2 archivos · Activo

Archivo	Líneas	Contenido
`app_diagnostics.dart`	71	`DiagnosticEntry`, `AppDiagnostics` (sin gleton, `maxEntries = 100`), `installGlo balErrorHandlers()`

`startup_failure.dart`	16	`StartupFailure` con tipo, mensaje fijo e n español y huella de pila

`core/feature_flags/feature_flags.dart` · 70 líneas · Reservado

Nueve banderas booleanas, todas `false` por defecto, con `copyWith`, `toJson` y `fromJson`. Se persisten en la preferencia `feature_flags`.

Ningún código de `lib/` las consulta. Comprobado con `grep` sobre los nueve nombres: cero apariciones fuera de este archivo. Tampoco hay control en Ajustes que permita cambiarlas. R-02.

`core/localization/app_localizations.dart` · 47 líneas · Activo

Doce cadenas con doble forma español/inglés, un `LocalizationsDelegate` que **rechaza en** y la extensión `AppStringsContext` que expone `context.strings`.

`supportedLocales` es `[Locale('es','CL'), Locale('es')]`. Cuatro pruebas protegen esta decisión (`test/core/localization_test.dart`), incluida una que verifica que las claves en inglés siguen existiendo para la traducción pendiente.

Consumidores reales de `context.strings`: `app.dart` (cinco etiquetas de navegación), `scanner_screen.dart` (título y subtítulo) y `settings_screen.dart` (el título «Ajustes»). El resto de las pantallas usa literales en español.

`core/performance/` · 2 archivos · Activo

Archivo	Líneas	Contenido	Lo usa
`async_write_queue.dart`	19	`AsyncWriteQueue`: cola serial que no se envenena ante un fallo	Solo `InventoryStore`
`cancellation_token.dart`	39	`OperationCancelledException`, `CancellationToken`, `BatchProgress`	`ScannerScreen`, `PdfPageRenderer`

`core/recovery/` · 3 archivos · Activo

Archivo	Líneas	Contenido
`recovery_issue.dart`	53	`RecoveryEntityType` (5 valores), `RecoveryIssueState` (3 valores), `RecoveryIssue`
`recovery_repository.dart`	60	Almacén `recovery_issues`; el `id` es un SHA-256 truncado a 24 caracteres de `tipo|entidad|código`
`recovery_service.dart`	84	`retry`, `discard`, `exportBundle`

Detalle no obvio: como el `id` se deriva del contenido, **una misma incidencia detectada dos veces no crea dos registros**: la segunda sobrescribe a la primera y reinicia su estado a `unresolved`. Es intencional para no llenar la lista, pero significa que marcar «revisado» un problema que vuelve a ocurrir lo devuelve a la lista.

De los cinco valores de `RecoveryEntityType`, solo `history`, `inventory` y `migration` llegan a crearse: los repositorios registran los dos primeros y `HistoryRepository._migrateLegacyHistory` el tercero. `database` y `startup` están declarados y **nunca se emiten** (comprobado: no aparecen en ninguna llamada a `record`). Su

`switch` en `retry` y `discard` devuelve `false` o no hace nada. R-09.

`core/security/` · 4 archivos · Activo

Archivo	Líneas	Contenido	Riesgo al modificar
`payload_cipher.dart`	168	`EncryptionKeyProvider`, `SecureStorageKeyProvider`, `MemoryEncryptionKeyProvider`, `CipherEnvelopeInfo`, `PayloadCipher`	**Crítico**: un cambio incompatible deja ilegibles los datos existentes
`encryption_metadata_repository.dart`	30	Lee y escribe `encryption_active_key_id` en `_security_meta`, con migración desde preferencias	Alto
`data_maintenance_service.dart`	72	`EncryptionRotationResult`, `DataMaintenanceService.rotateEncryptionKey()`	**Crítico**
`scan_security_analyzer.dart`	165	`RiskLevel`, `SecurityAssessment`, `ScanSecurityAnalyzer` con 16 señales	Alto: es la única defensa antes de una acción externa

`core/theme/app_theme.dart` · 54 líneas · Activo

`ColorScheme.fromSeed` con semilla `#006B66` en claro y `#5BD7CD` en oscuro. Tres modificadores: `highContrast` (sube `contrastLevel` a 1.0 y dibuja borde en las tarjetas), `largeControls` (`VisualDensity(1, 1)`) y `reduceMotion` (sustituye las transiciones de página por `_NoTransitionsBuilder` en las seis plataformas objetivo).

`core/utils/barcode_labels.dart` · 86 líneas · Parcialmente sin uso

Miembro	Estado
`BarcodeLabels.format(BarcodeFormat)`	**Activo**: `ScanRecord.fromBarcode` lo llama
`BarcodeLabels.contentType(Barcode)`	**Sin uso**: cero llamadas en `lib/` y en `test/`
`BarcodeLabels._fallbackType`	Sin uso por transitividad

`contentType` fue reemplazado por `ParsedContent.title`, que `ScanRecord` toma del intérprete. Las 39 líneas de `contentType` y `_fallbackType` son código muerto. R-05.

`format` cubre los 23 valores de `BarcodeFormat` de `mobile_scanner 7.4.0`, uno de ellos —`BarcodeFormat.itf`— con `// ignore: deprecated_member_use`. Si el paquete añade un valor, este `switch` exhaustivo **fallará al compilar**, que es exactamente lo que se quiere.

`lib/models/` — entidades

Archivo	Líneas	Clases	Estado
`scan_record.dart`	161	`ScanRecord` con tres factorías (`fromBarcode`, `manual`, `fromJson`)	Activo

`inventory_session.dart`	94	`InventoryItem`, `InventorySession`	Activo
`parsed_content.dart`	54	`ContentKind` (17 valores), `ParsedContent`	Activo
`app_settings.dart`	89	`AppLanguage` (4 valores), `AppSettings` (16 campos + banderas)	Activo

Todas son inmutables (`final` en todos los campos) y todas serializan a JSON. Ninguna sobrescribe `==` ni `hashCode`: la identidad se compara por `id` allí donde importa.

Detalle no obvio en `ScanRecord`: el `id` se calcula como SHA-256 de `rawValue|microsegundos` truncado a 20 caracteres. Dos lecturas del mismo código en microsegundos distintos producen identificadores distintos, así que el historial **no deduplica lecturas repetidas del mismo código**; lo que sí deduplica es el mismo código dentro de una misma captura (`unique.putIfAbsent`) y las lecturas repetidas dentro de una ventana de 2 segundos (`_lastSignature` / `_lastDetectedAt`).

`AppLanguage` tiene cuatro valores, pero `AppSettings.locale` devuelve `Locale('en')` para `AppLanguage.en` y el delegado rechaza `en`. El desplegable de Ajustes ofrece solo tres y **traduce `en` a `system`** para no quedarse sin coincidencia si una versión futura lo hubiera guardado (comentario en `settings_screen.dart`, líneas 66-69).

`lib/services/` — servicios de aplicación y plataforma

Archivo	Líneas	Responsabilidad	Estado
`content_interpreter.dart`	402	20 ramas de reconocimiento → 17 `ContentKind`	Activo
`history_repository.dart`	223	Persistencia cifrada del historial, poda, recorte y migración desde preferencias	Activo
`import_service.dart`	194	Validación y vista previa de respaldos JSON	Activo
`export_service.dart`	164	JSON, CSV y XLSX de historial e inventario vía `share_plus`	Activo
`inventory_repository.dart`	58	Persistencia cifrada de sesiones	Activo
`settings_repository.dart`	77	17 claves de `SharedPreferencesAsync`	Activo
`scan_feedback.dart`	108	`ScanTonePlayer`, `AssetScanTonePlayer`, `ScanFeedback`	Activo
`pdf_page_renderer.dart`	2	`export` condicional	Plataforma
`pdf_page_renderer_io.dart`	115	Selección y renderizado de PDF a PNG con cancelación	Plataforma
`pdf_page_renderer_stub.dart`	20	Lanza `UnsupportedError` en web	Plataforma

`biometric_service.dart`	24	`isAvailable()` y `authenticate()`, ambos con `catch` que devuelve `false`	Activo
`clipboard_service.dart`	14	Copia con borrado programado condicional	Activo

Detalle no obvio en `ClipboardService.copy`: el borrado programado **comprueba antes que el portapapeles siga conteniendo el mismo valor**. Si el usuario copió otra cosa entre medias, no se le borra. El temporizador no se cancela nunca ni se guarda referencia: sobrevive al `dispose` del widget que lo inició.

Detalle no obvio en `ExportService`: `_encodeCsv` antepone el BOM UTF-8 (`0xEF 0xBB 0xBF`) y usa `\r\n` como separador de línea, y **entrecomilla todas las celdas sin excepción**. Es la combinación que hace que Excel en Windows abra el archivo con acentos correctos.

`lib/state/` — stores

Archivo	Líneas	Notifica	Cola serial
`scan_store.dart`	103	Carga, importación, actualización, poda, borrado	No
`inventory_store.dart`	145	Cualquier cambio de sesión o de ítem	**Sí** (`AsyncWriteQueue`)
`settings_store.dart`	22	Cualquier cambio de preferencia	No

`SettingsStore.update` **notifica antes de persistir** (`notifyListeners()` en la línea 19, `await _repository.save(value)` en la 20). La interfaz responde al instante y la escritura ocurre después. *Inferencia*: si la escritura fallara, la interfaz mostraría un valor que no está guardado. No hay manejo de ese caso.

`lib/features/` — pantallas

Carpeta	Archivo	Líneas	Estado
`scanner/`	`scanner_screen.dart`	638	Activo · el archivo más grande
`scanner/domain/`	`scanner_engine.dart`	16	Activo · interfaz de 8 miembros
`scanner/data/`	`mobile_scanner_engine.dart`	36	Activo · única implementación
`scanner/widgets/`	`scanner_overlay.dart`	145	Activo · `CustomPainter` del marco y la línea
`scanner/widgets/`	`scan_status_bar.dart`	137	Activo · `ScanPhase` y la barra de estado
`inventory/`	`inventory_screen.dart`	491	Activo · **no usa `ScannerEngine`**
`settings/`	`settings_screen.dart`	378	Activo

`result/`	`scan_result_sheet.dart`	318	Activo · `ScanResultsSheet`, `ScanRecordCard`, `_RiskBanner`
`generator/`	`generator_screen.dart`	310	Activo
`history/`	`history_screen.dart`	284	Activo
`formats/`	`formats_screen.dart`	170	Activo · **lista estática de 20 formatos, escrita a mano**
`formats/domain/`	`content_parser.dart`	8	Activo · interfaz de 4 miembros
`formats/domain/`	`content_parser_registry.dart`	38	Activo
`recovery/`	`recovery_screen.dart`	139	Activo

`FormatsScreen` mantiene tres grupos con 20 nombres de formato **escritos como literales**, sin ninguna relación con `BarcodeLabels.format` ni con lo que el motor declare. Si `mobile_scanner` añadiera o quitara una simbología, esta pantalla no lo reflejaría. R-10.

`ScanRecordCard` se reutiliza en dos sitios: dentro de `ScanResultsSheet` tras una lectura, y desde `HistoryScreen._showDetails`. Su parámetro `compact` —que oculta la banda de riesgo y los botones de acción— **nunca se pasa como `true`**: ambos llamantes usan el valor por defecto. R-11.

`tool/` — herramientas

Guion	Líneas	Qué hace	Cuándo se ejecuta
`bootstrap.py`	407	Genera y parchea `android`, `ios`, `web`, `macos`	Tras clonar; en ambos workflows
`validate_structure.py`	185	**Diez comprobaciones offline** (ver abajo)	CI, puerta de calidad, publicación
`generate_launcher_icons.py`	174	Dibuja los iconos por código con Pillow	Solo al cambiar la identidad visual
`generate_sbom.py`	92	SBOM CycloneDX 1.5; resuelto si hay Flutter, declarado si no	CI, puerta de calidad
`generate_scan_beep.py`	60	Sintetiza `assets/sounds/scan_success.wav` sin dependencias	Solo al cambiar el tono
`generate_license_inventory.py`	49	Inventario de licencias desde `.dart_tool/package_config.json`	CI, puerta de calidad
`generate_source_manifest.py`	45	`SOURCE_MANIFEST.json` con tamaño y SHA-256 de cada fuente	Publicación
`generate_checksums.py`	8	SHA-256 de los archivos que reciba por argumentos	CI, publicación

`run_quality_gate.sh`	10	Encadena validación, análisis, pruebas y SBOM	Manual
`finalize_stable.sh`	17	Lo anterior más compilaciones y manifiesto	Antes de etiquetar
`bootstrap.sh`	4	Envoltorio POSIX de `bootstrap.py`	Manual
`bootstrap.ps1`	10	Envoltorio Windows de `bootstrap.py`	Manual

Las diez comprobaciones de `validate_structure.py`

Son el **gate propio del repositorio** y revelan qué considera irrompible:

Función	Comprueba
`check_yaml`	Que `pubspec.yaml`, `analysis_options.yaml` y `flutter-ci.yml` sean YAML válido
`check_imports`	Que todo `import 'package:universal_code_scanner/…'` apunte a un archivo existente
`check_absolute_paths`	Que ningún archivo contenga una de las tres rutas de entorno prohibidas: dos directorios de trabajo de entornos de ejecución remotos y el prefijo del directorio de perfiles de Windows. La lista literal está en la variable `bad` de esa función
`check_version`	Que `version` de `pubspec.yaml` sea `X.Y.Z+N`
`check_interface_version`	Que `appVersion` de `app_info.dart` coincida con `pubspec.yaml`
`check_sensitive_logging`	Que ningún `print`/`debugPrint`/`log` reciba `rawValue`, `password`, `secret`, `otp` o `payload`
`check_test_assets`	Que `test_assets/manifest.json` exista, tenga `schemaVersion: 1` y que **cada archivo declarado exista**
`check_json_files`	Que todo `.json` del repositorio sea JSON válido
`check_markdown_links`	Que **todo enlace relativo de todo `.md`** resuelva a un archivo existente
`check_source_sbom`	Que `docs/security/sbom-source.cdx.json` exista y declare `CycloneDX`

`--require-lock` añade la exigencia de que `pubspec.lock` exista.

`check_markdown_links` es el validador de enlaces del repositorio y cubre también esta documentación: los directorios excluidos son solo `.git`, `.dart_tool`, `build`, `coverage` y las carpetas de plataforma.

`check_test_assets` verifica **existencia y forma**, nunca el contenido: no decodifica ninguna imagen ni compara con `expectedPayload`. Ver R-01.

Archivos de nivel raíz

Archivo	Contenido	Estado
`pubspec.yaml` / `pubspec.lock`	Manifiesto y grafo resuelto y versionado	Activo
`analysis_options.yaml`	`flutter_lints` + 4 reglas: `always_declare_return_types`, `avoid_print`, `prefer_final_locals`, `use_build_context_synchronously`	Activo
`.fvmrc`	Solo `{"flutter": "3.44.7"}`	Declarativo
`.gitattributes`	`* text=auto eol=lf` y binarios explícitos. **Los hashes de `SOURCE_MANIFEST.json` dependen de esto**	Activo
`.gitignore`	Excluye artefactos y las cuatro carpetas de plataforma	Activo
`SOURCE_MANIFEST.json`	150 archivos con tamaño y SHA-256	Regenerable
`RELEASE_SOURCE.json`	Qué se validó al publicar 1.1.0, con banderas por comprobación	Manual
`VALIDATION.md`, `IMPLEMENTATION_STATUS.md`, `CHANGELOG.md`, `README.md`	Documentación previa	Activo
`LICENSE`	MIT	Activo

Resumen de elementos sin uso o reservados

Elemento	Ubicación	Clasificación	Hallazgo
Las nueve banderas de `FeatureFlags`	`core/feature_flags/`	Reservado, sin lector	R-02
`BarcodeLabels.contentType` y `_fallbackType`	`core/utils/`	Sin uso	R-05
Reserva final de `ContentParserRegistry.parse`	`features/formats/domain/`	Inalcanzable	R-08
`RecoveryEntityType.database` y `.startup`	`core/recovery/`	Declarados, nunca emitidos	R-09
Parámetro `compact` de `ScanRecordCard`	`features/result/`	Nunca `true`	R-11
Filtro `paths: web/**` de `deploy-landing.yml`	`.github/workflows/`	Inerte: `web/` está en `.gitignore`	R-12
Seis carpetas de `test_assets/` con solo `README.md`	`test_assets/`	Reservadas a propósito y declaradas	Documentado

Continúa en 05 · Referencia técnica.

05 · Referencia técnica

Catálogo de consulta. Para cada elemento relevante: firma, propósito, parámetros, retorno, excepciones, efectos secundarios, quién lo llama, a quién llama y **riesgo al modificarlo**.

La explicación narrativa de los mismos módulos está en 06-deep-code-explanation.md.

Constantes públicas

Constante	Valor	Archivo	Consecuencia de cambiarla
`appVersion`	`'1.1.0'`	`core/app_info.dart`	`validate_structure.py` fall a si no coincide con `pubspec.yaml`
`appName`	`'Universal Code Scanner'`	`core/app_info.dart`	Solo texto de interfaz
`SchemaMigrator.currentVersion`	`4`	`core/database/schema_migrator.dart`	Bajarla deja bases existentes por delante del código
`PayloadCipher.currentVersion`	`2`	`core/security/payload_cipher.dart`	**Crítico.** Un sobre con versión mayor se rechaza con `FormatException`
`PayloadCipher.currentAlgorithm`	`'AES-256-GCM'`	idem	**Crítico.** Un sobre con otro algoritmo se rechaza
`PayloadCipher.currentKeyId`	`'v2'`	idem	**Crítico.** Cambiarla busca una llave que no existe
`PayloadCipher.legacyKeyId`	`'v2'`	idem	Idéntica a la anterior; ver la nota más abajo
`SecureStorageKeyProvider._prefix`	`'ucs_database_key_'`	idem	**Crítico.** Cambiarlo pierde el acceso a las llaves guardadas
`AppDiagnostics.maxEntries`	`100`	`core/diagnostics/app_diagnostics.dart`	Solo memoria
`HistoryRepository.maxItems`	`5000`	`services/history_repository.dart`	Recorte del historial
`ImportService.currentSchemaVersion`	`2`	`services/import_service.dart`	Un respaldo con versión mayor se rechaza
`ImportService.maxHistoryRecords`	`5000`	idem	Límite de registros por respaldo
`ImportService.maxImportBytes`	`25 * 1024 * 1024`	idem	25 MB
`ImportService.maxJsonDepth`	`32`	idem	Defensa contra JSON anidado
`ImportService.maxJsonNodes`	`100000`	idem	Defensa contra JSON masivo

`ImportService.maxInventoryItems`	`10000`	idem	Límite de productos por sesión
`AssetScanTonePlayer.assetPath`	`'sounds/scan_success.wav'`	`services/scan_feedback.dart`	Ruta relativa a `assets/`, tal como la resuelve `AssetSource`
`HandheldFrame.maxWidth`	`560`	`app.dart`	Umbral de centrado en pantallas anchas

Nota sobre `legacyKeyId`: vale `'v2'`, exactamente igual que `currentKeyId`. Un sobre «heredado» —el que no tiene campo `version`— se descifra por tanto con la misma llave que uno actual. Es correcto para este repositorio (el formato inicial usaba esa misma llave) pero las dos constantes son indistinguibles y cualquier cambio futuro de `currentKeyId` **rompería silenciosamente la lectura de los sobres heredados** si no se cambia también `legacyKeyId`. R-13.

Enumeraciones

Enum	Valores	Archivo
`RiskLevel`	`low`, `caution`, `high`	`core/security/scan_security_analyzer.dart`
`ContentKind`	`url`, `wifi`, `contact`, `event`, `email`, `phone`, `sms`, `geo`, `otp`, `gs1`, `isbn`, `product`, `payment`, `crypto`, `identity`, `binary`, `text` (17)	`models/parsed_content.dart`
`AppLanguage`	`system`, `esCl`, `es`, `en`	`models/app_settings.dart`
`ScanPhase`	`starting`, `scanning`, `paused`, `unavailable`	`features/scanner/widgets/scan_status_bar.dart`
`RecoveryEntityType`	`history`, `inventory`, `migration`, `database`, `startup`	`core/recovery/recovery_issue.dart`
`RecoveryIssueState`	`unresolved`, `recovered`, `deleted`	idem
`ImportStrategy`	`merge`, `skipDuplicates`, `replace`	`services/import_service.dart`
`_PayloadType` (privado)	`text`, `url`, `wifi`, `contact`, `event`, `email`, `phone`, `sms`, `geo`	`features/generator/generator_screen.dart`
`_CodeFormat` (privado)	`qr`, `dataMatrix`, `aztec`, `pdf417`, `code128`, `code39`, `ean13`, `ean8`, `upcA`, `gs128`	idem

`ContentKind`, `RiskLevel`, `RecoveryEntityType` y `RecoveryIssueState` se serializan **por nombre** (`.name` / `.byName`). Renombrar un valor rompe la lectura de los datos ya guardados y de los respaldos exportados.

Cifrado — `PayloadCipher`

```
Future<String> encryptJson(Map<String, dynamic> value, {String? keyId})
```

Aspecto	Detalle
Propósito	Cifrar un mapa JSON y devolver el sobre serializado

Parámetros	`value`: el objeto a cifrar. `keyId`: llave alternativa; por defecto la a ctiva
Retorno	JSON con `version`, `algorithm`, `keyId`, `createdAt` y `payload{nonce, cipherText, mac}`, todos en Base64
Excepciones	`StateError('encryption_key_missing:<keyId>')` si ese iden tificador está marcado como ausente
Efectos secundarios	**Crea la llave si no existe** (`readOrCreate`) y la escribe en el alm acén seguro
Lo llama	`HistoryRepository` (4 sitios), `InventoryRepository.save`, `Da taMaintenanceService.rotateEncryptionKey`, `upgradeEnvel ope`
Riesgo al modificar	**Crítico.** Cualquier cambio de forma del sobre invalida los datos existentes

La comprobación de `_missingKeyIds` **al cifrar** es la mitad menos evidente y más importante del diseño: sin ella, un descifrado fallido seguido de una escritura crearía una llave nueva bajo el mismo identificador y dejaría los datos antiguos ilegibles para siempre.

`Future<Map<String, dynamic>> decryptJson(String encoded)`

Aspecto	Detalle
Retorno	El mapa original
Excepciones	`FormatException` si la versión está fuera de `[1, currentVersi on]` o el algoritmo no es `AES-256-GCM`; `StateError('encryptio n_key_missing:<keyId>')` si la llave no está; `SecretBoxAuthe nticationError` de `cryptography` si el MAC no valida
Efectos secundarios	Añade o quita el `keyId` de `_missingKeyIds`
Lo llama	Ambos repositorios, `RecoveryService.retry`, `DataMaintenanc eService`, `upgradeEnvelope`
Riesgo al modificar	**Crítico**

Acepta dos formas: el sobre actual (con `version`) y el **heredado**, que es el objeto `payload` suelto, sin envoltura. La detección es `!envelope.containsKey('version')`.

`CipherEnvelopeInfo inspect(String encoded)`

Devuelve versión, algoritmo, `keyId` y si es heredado, **sin descifrar**. Los repositorios lo usan para decidir si toca actualizar el sobre tras una lectura correcta.

`Future<String> upgradeEnvelope(String encoded)`

Devuelve el mismo texto si ya está en la versión actual y con la llave activa; en caso contrario descifra y vuelve a cifrar. Es una operación **cara**: se llama solo cuando `inspect` confirma que hace falta.

`void activateKey(String keyId)` · `Future<void> forgetKey(String keyId)`

`activateKey` lanza `ArgumentError` con un identificador vacío. `forgetKey` borra la llave del almacén **y** la quita de `_missingKeyIds`; solo lo llama el manejador de fallo de la rotación.

Rotación — `DataMaintenanceService.rotateEncryptionKey()`

```
Future<EncryptionRotationResult> rotateEncryptionKey()
```

Aspecto	Detalle
Retorno	`EncryptionRotationResult(keyId, historyRecords, inventorySessions)`
Excepciones	`StateError('encryption_rotation_in_progress')` si ya hay una en curso; `StateError('history_payload_missing:<id>')` o `StateError('inventory_payload_missing:<id>')` si un registro no tiene `payload`; cualquier error de cifrado
Efectos secundarios	Crea una llave `v3_<epoch_ms>`; reescribe todos los `payload`; actualiza `encryption_active_key_id`; cambia la llave activa del cifrador en memoria
Lo llama	`SettingsScreen._rotateEncryptionKey`
Riesgo al modificar	**Crítico**

Secuencia exacta:

1. bandera `_rotationInProgress` para impedir reentrada;
2. leer todos los registros de ambos almacenes;
3. **precalcular** todos los sobres nuevos en memoria (descifrar con la llave vieja, cifrar con la nueva);
4. una sola transacción que escribe registros **y** metadatos;
5. `_cipher.activateKey(keyId)` solo después de que la transacción confirme;
6. si algo falla, `forgetKey(keyId)` dentro de un `try` anidado que no oculta el error original, y `rethrow`.

El paso 3 antes del 4 es deliberado: mantiene la base bloqueada el mínimo tiempo. El paso 6 es lo que impide dejar material criptográfico huérfano que después pudiera confundirse con una llave activa. Ambos tienen prueba dedicada en `test/core/data_maintenance_test.dart`.

Análisis de riesgo — `ScanSecurityAnalyzer`

`static SecurityAssessment analyze(String rawValue)`

Aspecto	Detalle
Retorno	`SecurityAssessment(level, reasons, canOpen, normalizedHost)`
Excepciones	Ninguna. Todas las rutas devuelven un valor
Efectos secundarios	Ninguno: función pura
Lo llama	`ScanRecord.fromBarcode`, `ScanRecord.manual`

Riesgo al modificar	**Alto**: es la única defensa antes de una acción externa

Orden de evaluación —importa, porque las primeras ramas devuelven de inmediato:

#	Condición	Resultado
1	Empieza por `otpauth:`, `bitcoin:`, `lightning:` o `ethereum:`	`caution`, `canOpen: false`, un motivo
2	Empieza por `mailto:`, `tel:`, `sms:`, `smsto:` o `geo:`	`low`, `canOpen: **true**`, sin motivos
3	Contenido estructurado conocido (`wifi:`, `begin:vcard`, `mecard:`, `matmsg:`, `begin:vevent`, `binary-base64:`, `000201`, `spc\n`, `bcd\n`)	`low`, `canOpen: false`
4	No es una URI `http`/`https` **y** empieza por un esquema	`high`, `canOpen: false`, «esquema de aplicación no permitido»
5	No es una URI `http`/`https` y no tiene esquema	`low`, `canOpen: false`
6	Es `http`/`https`	Se evalúan las 16 señales

Las 16 señales, en el orden en que se aplican:

#	Señal	Nivel
1	Esquema `http` sin TLS	`caution`
2	Host vacío	`high`
3	`userInfo` presente (credenciales o `@` engañoso)	`high`
4	Host contiene `xn--` (Punycode)	`high`
5	Host mezcla latino, cirílico o griego	`high`
6	Host con puntos de código > 127	`caution`
7	Host es una IP (IPv4 o contiene `:`)	`caution`
8	Host privado, `localhost` o `.local`	`high`
9	Host en la lista de 11 acortadores conocidos	`caution`
10	Más de 5 etiquetas en el host	`caution`
11	Puerto explícito distinto de 80 y 443	`caution`
12	Longitud total > 240 caracteres	`caution`
13	Contiene `%00` o un punto de código < 32	`high`
14	La ruta termina en una de 17 extensiones ejecutables o comprimidas	`high`

15	Ruta o consulta contienen `login`, `signin`, `verify`, `account`, `password`, `wallet` o `bank`	`caution`
16	Tres o más parámetros de seguimiento (`utm_*`, `gclid`, `fbclid`, `mc_cid`, `ref`)	`caution`

`caution` solo eleva desde `low`; `high` es absorbente. `canOpen` para una URI web es siempre `true` **independientemente del nivel**: un enlace de riesgo alto sigue siendo abrible, pero el diálogo de confirmación es obligatorio porque `_openRecord` fuerza `shouldConfirm` cuando `riskLevel != RiskLevel.low`.

Las 17 extensiones peligrosas: `.apk .appx .bat .cmd .com .dmg .exe .hta .iso .jar .js .msi .pkg .ps1 .scr .vbs .zip`.

Los 11 acortadores: `bit.ly tinyurl.com t.co goo.gl ow.ly buff.ly is.gd cutt.ly rebrand.ly shorturl.at tiny.one`.

`static Uri? normalizedActionUri(String rawValue)`

Devuelve la URI que debe abrirse. Para los esquemas de sistema devuelve `Uri.tryParse` tal cual; para el resto aplica `_toWebUri`, que **antepone `https://` si el valor empieza por `www.`**. Es la única normalización que hace el analizador.

Interpretación — `ContentInterpreter.parse(String rawValue)`

Aspecto	Detalle
Retorno	`ParsedContent(kind, title, fields, summary, sensitive)`
Excepciones	Ninguna: todas las ramas devuelven, y la última es `ContentKind.text`
Efectos secundarios	Ninguno
Lo llama	`ContentParserRegistry.parse` y `LegacyContentParser.parse`
Riesgo al modificar	Medio. `sensitive` decide qué **no** se persiste

Orden de reconocimiento (el primero que coincide gana):

#	Prefijo o patrón	`ContentKind`	`sensitive`
1	`binary-base64:`	`binary`	no
2	`wifi:`	`wifi`	**sí si `P` no está vacío**
3	`begin:vcard`	`contact`	no
4	`mecard:`	`contact`	no
5	`begin:vevent`	`event`	no
6	`mailto:` o `matmsg:`	`email`	no
7	`tel:`	`phone`	no

8	`sms:` o `smsto:`	`sms`	no
9	`geo:`	`geo`	no
10	`otpauth:`	`otp`	**sí, siempre**
11	`bitcoin:`, `lightning:`, `ethereum:`	`crypto`	**sí, siempre**
12	Empieza por `SPC\n`	`payment` (Swiss QR Bill)	**sí**
13	Empieza por `BCD\n`	`payment` (EPC/SEPA)	**sí**
14	Empieza por `000201` y contiene `6304`	`payment` (EMVCo)	**sí**
15	Empieza por `@` y contiene `ANSI`	`identity` (AAMVA)	**sí**
16	Empieza por `(NN)` o contiene `<GS>`	`gs1`	no
17	10 dígitos, o 13 empezando por `978`/`979`	`isbn`	no
18	URI `http`/`https` con host, o empieza por `www.`	`url`	no
19	8 a 14 dígitos	`product`	no
20	Cualquier otra cosa	`text`	no

Cuidado con el orden 17 antes que 19: un EAN-13 que empiece por `978` o `979` se clasifica como ISBN, no como producto. Es correcto —los prefijos 978 y 979 son precisamente el rango Bookland de ISBN— pero explica por qué un libro escaneado aparece como «ISBN» y no como «Código de producto».

Y **cuidado con la comprobación `value.length == 10`** de `_isIsbn`: cualquier cadena de exactamente 10 caracteres tras eliminar todo lo que no sea dígito o `X`/`x` se considera ISBN. Un código de producto de 10 dígitos se etiquetará como ISBN. R-14.

Funciones auxiliares relevantes

Función	Qué hace	Nota
`_segments(String)`	Divide por `;` respetando `\` como escape y separa clave/valor por el primer `:`	Es lo que permite interpretar `WIFI:S:Mi\;Red;P:clave\:segura;;`. Hay prueba dedicada
`_tlv(String)`	Recorre un TLV EMVCo: 2 caracteres de identificador, 2 de longitud, N de valor	Se detiene en el primer campo mal formado en vez de lanzar
`_aiLabel(String)`	Traduce 9 Application Identifiers GS1 a nombre legible	El resto se muestra como `AI NN`
`_aamvaLabel(String)`	Traduce 7 códigos AAMVA	Solo esos 7 se extraen

Registro de parsers — `ContentParserRegistry`

Método	Firma	Nota
`parse`	`ParsedContent parse(String rawValue)`	Recorre por prioridad descendente; devuelve el primero que acepte
`register`	`void register(ContentParser parser)`	Sustituye por `id` y reordena
`unregister`	`void unregister(String id)`	**No puede quitar `builtin-v2`**
`parsers`	`List<ContentParser> get`	Copia inmodificable

`ContentParser` exige `id`, `priority`, `canParse` y `parse`. `LegacyContentParser` tiene prioridad `-1000` para quedar siempre al final.

Historial — `HistoryRepository`

Método	Firma	Efectos	Riesgo
`initialize`	`Future<HistoryMigrationStatus> initialize()`	Ejecuta la migración desde preferencias	**Alto**
`load`	`Future<List<ScanRecord>> load({int limit = maxItems})`	Descifra; **actualiza sobres antiguos**; registra incidencias	Medio
`upsert`	`Future<void> upsert(ScanRecord)`	Escribe y llama a `_trim()`	Bajo
`upsertAll`	`Future<void> upsertAll(Iterable<ScanRecord>)`	Precifra, escribe en una transacción, `_trim()`	Medio
`replaceAll`	`Future<void> replaceAll(Iterable<ScanRecord>)`	Borra todo y escribe en una transacción. **No llama a `_trim()`**	**Alto**
`ids`	`Future<Set<String>> ids()`	Solo claves; no descifra	Bajo
`remove`	`Future<void> remove(String id)`	Borra un registro	Bajo
`pruneOlderThan`	`Future<void> pruneOlderThan(int days)`	No hace nada si `days <= 0`	Medio
`clear`	`Future<void> clear()`	Vacía el almacén	Alto

`load` tiene un efecto secundario que su nombre no anuncia: si el sobre es heredado o de versión anterior, lo reescribe con `upgradeEnvelope`. Es actualización perezosa deliberada, documentada en `../quality/MIGRATIONS.md`, pero significa que **leer el historial escribe en la base**.

Y otro: si un registro no descifra, `load` **no lanza**. Lo omite del resultado y crea una incidencia con `code: 'decrypt_<TipoDeError>'` y la carga cifrada guardada. Un registro dañado no impide cargar los demás.

`replaceAll` no llama a `_trim()`. En la práctica no puede desbordar, porque `ImportService.maxHistoryRecords` vale también `5000`, pero la protección depende de esa coincidencia entre dos constantes de archivos distintos. R-15.

`HistoryMigrationStatus`

```
const HistoryMigrationStatus({
 required bool completed,
 int migrated = 0,
 String? errorCode,
 bool backupPresent = false,
})
```

`errorCode` tiene la forma `legacy_<TipoDeError>`; es lo que se muestra como subtítulo en Ajustes cuando la migración quedó pendiente.

Importación — `ImportService`

Método	Firma	Excepciones
`pickHistoryJson`	`Future<HistoryImportPreview?> pickHistoryJson({required Set<String> existingIds})`	Las de `parseHistoryBytes`, más `FormatException` por tamaño en el propio selector
`parseHistoryBytes`	`HistoryImportPreview parseHistoryBytes(List<int> bytes, {required Set<String> existingIds, String fileName})`	`FormatException` con mensaje en español para: >25 MB, JSON demasiado profundo, demasiados nodos, procedencia incorrecta, esquema futuro, sin registros, >5 000 registros
`pickInventoryJson`	`Future<InventoryImportPreview?> pickInventoryJson()`	Idem
`parseInventoryBytes`	`InventoryImportPreview parseInventoryBytes(List<int> bytes, {String fileName})`	Idem, más «La sesión supera el límite de productos»

`parseHistoryBytes` **nunca escribe**. Devuelve un `HistoryImportPreview` con `valid`, `duplicates`, `rejected`, `schemaVersion`, `legacy` y `fileName`. La escritura la decide `ScanStore.importPreview` tras elegir estrategia.

Los registros individuales que no se pueden reconstruir se cuentan en `rejected` y **no abortan la importación**: solo un error de forma del archivo completo lanza.

`_validateJsonShape` recorre el árbol con una pila explícita, no por recursión. Es la defensa que evita un desbordamiento de pila con un JSON profundo; el límite de profundidad se comprueba **por rama**, y el de nodos, en total.

Exportación — `ExportService`

Método	Formato	Nombre de archivo	MIME
`shareHistoryJson`	Sobre con `application`, `schemaVersion: 2`, `exportedAt`, `type: history`, `records`	`historial-escaneos.json`	`application/json`
`shareHistoryCsv`	8 columnas	`historial-escaneos.csv`	`text/csv`
`shareHistoryXlsx`	9 columnas (añade `Notas`)	`historial-escaneos.xlsx`	`…spreadsheetml.sheet`
`shareInventoryJson`	Sobre con `type: inventory` y `session`	`inventario-<id>.json`	`application/json`

`shareInventoryCsv`	8 columnas	`inventario-<id>.csv`	`text/csv`
`shareInventoryXlsx`	8 columnas	`inventario-<id>.xlsx`	`…spreadsheetml.sheet`

Asimetría real: el CSV del historial tiene 8 columnas y **no incluye `notas`**; el XLSX tiene 9 y sí. Los dos del inventario tienen ambos 8 y los dos incluyen `notas`. R-16.

Todo lo exportado está **en claro**. Es una decisión explícita del usuario y está declarada en `docs/SECURITY.md`.

Confirmación de lectura — `ScanFeedback`

Miembro	Firma	Comportamiento
`success`	`Future<void> success({required bool sound, required bool vibration})`	Vibra y suena, cada canal según su ajuste. **Ningún fallo se propaga**
`dispose`	`Future<void> dispose()`	Libera el reproductor; tolera que nunca se haya creado
`toneUnavailable`	`bool get` (`@visibleForTesting`)	`true` tras el primer fallo del reproductor

`AssetScanTonePlayer` construye el `AudioPlayer` **en la primera reproducción**, no en el constructor: una pantalla que nunca lee un código no crea un reproductor de plataforma. Configura `mixWithOthers` (no interrumpe música ni llamadas), `PlayerMode.lowLatency`, `ReleaseMode.stop` y precarga la fuente. Antes de cada reproducción hace `seek(Duration.zero)`: sin eso, una segunda lectura llegada mientras el tono suena sería silenciosa.

Modelos — firmas clave

`ScanRecord`

Factoría	Firma	Nota
`fromBarcode`	`ScanRecord.fromBarcode(Barcode barcode, {required String source, DateTime? scannedAt})`	Ejecuta analizador e intérprete
`manual`	`ScanRecord.manual({required String rawValue, required String format, required String source})`	Igual, pero sin `Barcode`
`fromJson`	`ScanRecord.fromJson(Map<String, dynamic> json)`	**Reinterpreta** si falta `parsed` o `contentType`

Método estático	Firma	Nota
`payloadForBarcode`	`static String payloadForBarcode(Barcode)`	Devuelve el texto, o `binary-base64:<base64>`, o cadena vacía
`_decodedBytes`	`static Uint8List? _decodedBytes(BarcodeBytes?)`	Distingue `DecodedBarcodeBytes` de `DecodedVisionBarcodeBytes` con comprobación de tipo explícita

`isSensitive` delega en `parsed.sensitive`. Es el único predicado que decide qué queda fuera del historial automático.

`InventorySession` · `InventoryItem`

`InventorySession.items` está indexado por el **valor bruto del código**, no por un identificador propio. `totalUnits` suma las cantidades. `isOpen` es `closedAt == null`.

`InventoryItem.copyWith` acepta solo `quantity`, `lastScannedAt` y `notes`: el código, el formato, la etiqueta y la primera lectura son inmutables una vez creados.

Claves de `SharedPreferences`

Escritas por `SettingsRepository.save` (17):

`theme_mode`, `language`, `sound_enabled`, `vibration_enabled`, `save_history`, `private_mode`, `auto_torch`, `use_scan_window`, `confirm_before_open`, `hide_sensitive_values`, `biometric_lock`, `high_contrast`, `large_controls`, `reduce_motion`, `clear_clipboard_seconds`, `history_retention_days`, `feature_flags`.

Escritas por `HistoryRepository` (4): `scan_history_v1`, `scan_history_v1_backup`, `history_migrated_to_v2`, `history_migration_error`.

Leída y eliminada por `EncryptionMetadataRepository` (1, solo migración): `encryption_active_key_id`.

`resetNonSensitive()` borra 14 de las 17 primeras. **Deja intactas `save_history`, `private_mode` y `biometric_lock`**: restablecer las preferencias visuales desde la pantalla de inicio seguro no puede desactivar el bloqueo de la aplicación ni reactivar el guardado de historial que el usuario había apagado. Es una decisión deliberada y **no documentada en el repositorio** fuera del propio nombre del método.

Almacenes Sembast

Almacén	Tipo	Clave	Valor
`scan_history`	`stringMapStoreFactory`	`ScanRecord.id`	`{id, scannedAt, payload}`
`inventory_sessions`	`stringMapStoreFactory`	`InventorySession.id`	`{id, createdAt, payload}`
`recovery_issues`	`stringMapStoreFactory`	SHA-256 truncado a 24	`RecoveryIssue.toJson()`
`_schema_meta`	`StoreRef<String, Object?>`	Nombre de la marca	Valor escalar
`_security_meta`	`StoreRef<String, Object?>`	`encryption_active_key_id`	Cadena

Detalle en 07-database.md.

Códigos de error e identificadores internos

Código	Dónde se genera	Significado
`encryption_key_missing:<keyId>`	`PayloadCipher`	La llave no está en el almacén
`encryption_rotation_in_progress`	`DataMaintenanceService`	Rotación reentrante
`history_payload_missing:<id>`	idem	Un registro de historial sin `payload`
`inventory_payload_missing:<id>`	idem	Un registro de inventario sin `payload`

`migration_limit_exceeded`	`HistoryRepository`	La migración superaría 5000 registros
`migration_verification_failed`	idem	La transacción no contiene todo lo esperado
`missing_payload`	Ambos repositorios	Incidencia de recuperación
`decrypt_<TipoDeError>`	Ambos repositorios	Incidencia de recuperación
`legacy_<TipoDeError>`	`HistoryRepository`	Fallo de la migración desde preferencias
`legacy_not_list`	idem	El historial heredado no era una lista JSON

Las áreas que se pasan a `AppDiagnostics.record` son cinco: `flutter`, `platform`, `root_zone`, `bootstrap` y `bootstrap_temporary`. Las tres primeras las instala `installGlobalErrorHandlers()`; las dos últimas las emite `_BootstrapHostState._retry` según el modo con el que se reintentó el arranque.

Límites operativos

Límite	Valor	Dónde
Imágenes por lote de galería	20	`pickMultiImage(limit: 20)`
Páginas de PDF por lote	50	`PdfPageRenderer.pickAndRender(maxPages: 50)`
Escala de renderizado de página	2400 px en el lado mayor, acotada a `[1.5, 4.0]`	`_renderScale`
Registros de historial	5000	`HistoryRepository.maxItems`
Ventana antirrebote de la cámara	2 s por firma de captura	`ScannerScreen._handleCapture`
Ventana antirrebote del inventario	1200 ms por código	`InventoryScreen._onDetect`
Ancho máximo de la interfaz	560 px lógicos	`HandheldFrame.maxWidth`
Lado del marco de lectura	68 % del lado menor	`ScannerScreen.build`
Duración del barrido del marco	2200 ms, con inversión	`_ScannerOverlayState`
Entradas de diagnóstico	100	`AppDiagnostics.maxEntries`

Continúa en 06 · Explicación profunda del código.

06 · Explicación profunda del código

Este documento explica **flujo por flujo** qué ocurre dentro del sistema, por qué está escrito así y qué casos límite contempla. La referencia seca de firmas está en 05-technical-reference.md; aquí se explica el razonamiento.

Se explican en detalle los flujos que concentran las decisiones. Las pantallas de lista y formulario —historial, generador, formatos— se resumen: son voluminosas y poco densas en decisiones.

---

1 · Arranque: `main` → `AppServices`

1.1 `main.dart`

Catorce líneas que hacen tres cosas antes de nada más:

```
WidgetsFlutterBinding.ensureInitialized();
installGlobalErrorHandlers();
runZonedGuarded<void>(() => runApp(const BootstrapHost()), …);
```

El orden importa. `ensureInitialized()` debe ir primero porque `installGlobalErrorHandlers()` toca `PlatformDispatcher.instance`. Y `runZonedGuarded` envuelve a `runApp`, no al revés: una excepción asíncrona que escapa de un `Future` sin `catch` cae en la zona, no en `FlutterError.onError`.

Los tres manejadores capturan tres clases distintas de fallo, y por eso existen los tres:

Manejador	Captura	Área
`FlutterError.onError`	Errores del árbol de widgets, durante `build`, `layout` o `paint`	`flutter`
`PlatformDispatcher.instance.onError`	Errores asíncronos que llegan del motor	`platform`
El de `runZonedGuarded`	Todo lo demás que escape de la zona	`root_zone`

`FlutterError.onError` **vuelve a llamar a `FlutterError.presentError(details)`** después de registrar. Sin eso, se perdería el volcado rojo en depuración, que es donde un desarrollador espera verlo.

1.2 `AppBootstrapper.initialize`

El grafo se construye en un orden que no es arbitrario:

```
database = temporary ? await AppDatabase.openTemporary() : await AppDatabase.open();
final RecoveryRepository recovery = RecoveryRepository(database);
final EncryptionMetadataRepository encryptionMetadata = EncryptionMetadataRepository(database);
final String activeKeyId = temporary ? PayloadCipher.currentKeyId : await encryptionMetadata.loadActiveKey
final PayloadCipher cipher = PayloadCipher(keyProvider: temporary ? MemoryEncryptionKeyProvider() : null,
```

La llave activa se lee de la base antes de construir el cifrador. Si se hiciera al revés, el cifrador arrancaría con `'v2'` y la primera escritura tras una rotación previa crearía una llave nueva bajo un identificador equivocado. `loadActiveKeyId` es además el punto donde ocurre la migración desde preferencias: si el almacén `_security_meta` está vacío, lee la preferencia `encryption_active_key_id`, la escribe en la base y **la borra de preferencias**.

Después:

```
await Future.wait(<Future<void>>[scans.initialize(), inventory.initialize()]);
await scans.pruneOlderThan(settings.value.historyRetentionDays);
```

Historial e inventario se inicializan en paralelo porque no dependen entre sí. La poda va después porque necesita el valor de `historyRetentionDays`, que viene de las preferencias ya inicializadas.

El bloque completo está en un `try` cuyo `catch` hace:

```
} on Object {
 await database?.close();
 rethrow;
}
```

Es decir: **si algo falla a mitad, la base se cierra antes de propagar**. Sin eso, el reintento desde la pantalla de inicio seguro intentaría abrir un archivo ya bloqueado por la propia aplicación.

En modo temporal, `settings.initialize()` va dentro de su propio `try/catch` vacío. El comentario del código lo dice: «Temporary mode remains available even when preferences are unavailable». Es la última red de seguridad: si lo que falló fue el propio subsistema de preferencias, el modo temporal debe seguir arrancando con `const AppSettings()` por defecto.

1.3 La pantalla de inicio seguro

`_BootstrapHostState._retry` hace algo que parece redundante y no lo es:

```
_future = AppBootstrapper.initialize(temporary: temporary).catchError((error, stack) {
 AppDiagnostics.instance.record(error, stack, area: temporary ? 'bootstrap_temporary' : 'bootstrap');
 _failure = StartupFailure.from(error, stack);
 throw error;
});
```

Registra el fallo, guarda `_failure` **y vuelve a lanzar**. El relanzamiento mantiene el `Future` en estado de error, de modo que el `FutureBuilder` no se quede colgado en el indicador de progreso. `build` comprueba `_failure` primero —porque `_failure` se asigna fuera de un `setState`— y el `FutureBuilder` comprueba `snapshot.hasError && _failure != null` como segunda vía.

Nótese que `_failure = …` ocurre sin `setState`. Funciona porque el propio `setState` que envuelve la asignación de `_future` ya programó una reconstrucción, y `catchError` se resuelve en un microtask posterior que dispara la reconstrucción del `FutureBuilder`. **Es frágil**: si el orden cambiara, la pantalla podría quedarse en el indicador de progreso. R-17.

---

2 · Lectura con cámara

2.1 Por qué la pantalla gestiona el ciclo de vida

`MobileScanner` solo atiende el ciclo de vida cuando **él mismo** crea el controlador. Estas pantallas le pasan el suyo —lo necesitan para el zoom, la linterna y `analyzeImage`—, así que la gestión les corresponde. El comentario del código lo dice sin rodeos: sin esto, volver del segundo plano o del diálogo de permiso del sistema dejaba la vista previa congelada sin más salida que abandonar la pestaña.

```
case AppLifecycleState.resumed:
 if (!_paused && !_handlingResult) unawaited(_startCamera());
case AppLifecycleState.inactive:
 unawaited(_stopCamera());
```

Las dos condiciones de `resumed` importan. `!_paused` respeta la decisión explícita del usuario de pausar; `!_handlingResult` evita que volver del diálogo de compartir reanime la cámara mientras la hoja de resultados sigue abierta.

`hidden`, `paused` y `detached` no hacen nada porque `inactive` **siempre precede** a los tres en Android e iOS: parar dos veces sería redundante.

2.2 Toda llamada al controlador va envuelta

```
Future<void> _startCamera() async {
 try {
   await _engine.start();
   if (mounted) setState(() => _startFailure = null);
 } on MobileScannerException catch (error) {
   if (mounted) setState(() => _startFailure = _describe(error));
 } on Object {
   if (mounted) setState(() => _startFailure = 'No fue posible iniciar la cámara.');
 }
}
```

El comentario del código explica el porqué: «Every call into the controller can throw: it may be disposed, still starting, or not yet attached to a widget. A failure here must leave the screen in a state the user can act on, never in an exception.»

`_describe` traduce dos códigos concretos a instrucciones accionables (`permissionDenied` → «Actívalo en los ajustes del sistema»; `unsupported` → «Este dispositivo no permite leer códigos con la cámara») y el resto a «Toca "Reintentar"». El texto genérico existe porque `MobileScannerErrorCode` puede crecer con el paquete.

`_stopCamera` captura y **no hace nada**: parar una cámara ya parada o ya desechada no es un error en este contexto.

2.3 «Reiniciar cámara»: el orden que importa

```
await _stopCamera();
try { await _engine.dispose(); } on Object { }
if (!mounted) return;
setState(() {
 _createEngine();
 _previewKey = UniqueKey();
 …
});
```

El comentario original: «The old controller is released before the new one exists: both share a single platform camera session, and disposing it afterwards would tear down the session the new controller has just claimed.»

`_previewKey = UniqueKey()` es la otra mitad. Sin cambiar la clave, Flutter reutilizaría el elemento `MobileScanner` existente y lo dejaría atado al controlador viejo. Un controlador que no arrancó **no se puede reutilizar**: la recuperación consiste literalmente en construir uno nuevo.

2.4 Cómo se decide la fase visible

```
ScanPhase _phaseFor(MobileScannerState state) {
 if (_startFailure != null || state.error != null) return ScanPhase.unavailable;
 if (_paused || _handlingResult) return ScanPhase.paused;
 if (state.isRunning) return ScanPhase.scanning;
 return ScanPhase.starting;
}
```

El orden es una precedencia: el fallo gana sobre la pausa, y la pausa sobre el estado real del motor. Que `_handlingResult` produzca `paused` es lo que hace que la barra deje de moverse mientras se muestra la hoja de resultados, aunque técnicamente la cámara ya se detuvo.

`ScanStatusBar` traduce la fase a título, icono, color y —lo importante— al valor del `LinearProgressIndicator`:

```
value: busy && animate ? null : (busy ? 1 : 0)
```

`null` es la barra indeterminada que se mueve. Un valor fijo la dibuja quieta. Así, con movimiento reducido activado, una cámara ocupada se dibuja llena (valor `1`) y una en pausa vacía (valor `0`): el estado sigue siendo legible **sin animación**. Hay cinco pruebas de widget sobre exactamente este comportamiento.

2.5 El marco de lectura, y por qué no tiene umbral

```
MobileScanner(
 key: _previewKey,
 controller: _engine.controller,
 scanWindow: _settings.useScanWindow ? window : null,
 // No threshold: …
 tapToFocus: true,
 onDetect: (capture) => unawaited(_handleCapture(capture, source: 'Cámara')),
 errorBuilder: (context, error) => _ScannerError(error: error, onRetry: _restartCamera),
)
```

El comentario que ocupa el lugar del parámetro ausente es el registro del defecto que originó la versión 1.1.0: la ventana de lectura se expresa en porcentajes de 0 a 1 y el umbral estaba en `4`, es decir, «no recalcular nunca». Si el primer cálculo salía mal —porque el tamaño de la cámara y la orientación pueden llegar después del primer cuadro—, **todos los códigos quedaban fuera de la región analizada**. La corrección fue quitar el umbral.

El rectángulo es cuadrado y ocupa el 68 % del lado menor, centrado (`LayoutBuilder` → `Rect.fromCenter`). Cuando `useScanWindow` está desactivado se pasa `null` y se analiza toda la imagen: es la salida que ofrece el README cuando un código no entra.

2.6 `_handleCapture`: deduplicación en dos niveles

```
if (_handlingResult || _paused || !mounted) return;
final Map<String, Barcode> unique = <String, Barcode>{};
for (final Barcode barcode in capture.barcodes) {
 final String raw = ScanRecord.payloadForBarcode(barcode);
 if (raw.isNotEmpty) unique.putIfAbsent(raw, () => barcode);
}
if (unique.isEmpty) return;
```

Primer nivel: dentro de una misma captura, dos detecciones del mismo texto producen un solo registro. `putIfAbsent` conserva la primera.

```
final List<String> signature = unique.keys.toList()..sort();
final String joined = signature.join('|');
if (_lastSignature == joined && _lastDetectedAt != null &&
   now.difference(_lastDetectedAt!) < const Duration(seconds: 2)) return;
```

Segundo nivel: entre capturas, la misma **combinación** de códigos dentro de 2 segundos se ignora. La firma se ordena antes de unir para que el orden en que el motor devuelve los códigos no genere firmas distintas del mismo conjunto.

Esto es necesario aunque el controlador use `DetectionSpeed.noDuplicates`, porque ese modo se reinicia con cada `start()` y la pantalla arranca y para la cámara constantemente.

Después:

```
try {
 await _stopCamera();
 await _feedback.success(sound: …, vibration: …);
 final records = unique.values.map((b) => ScanRecord.fromBarcode(b, source: source, scannedAt: now)).toLis
 await _persistAndShow(records);
```

```
} finally {
 if (mounted) setState(() => _handlingResult = false);
 if (mounted && !_paused) await _startCamera();
}
```

La cámara se detiene **antes** de la confirmación sonora y del análisis: sin eso, seguiría entregando cuadros mientras se construye la hoja de resultados, y cada uno volvería a entrar en `_handleCapture`. El `finally` garantiza que la cámara vuelve aunque `_persistAndShow` lance.

Todos los registros de una misma captura comparten `scannedAt: now`. Sus `id` siguen siendo distintos porque el `rawValue` difiere.

2.7 `_persistAndShow`: dónde se decide qué se guarda

```
if (_settings.saveHistory && !_settings.privateMode) {
 final persistent = records.where((r) => !r.isSensitive).toList(growable: false);
 await widget.store.addAll(persistent);
}
…
await showModalBottomSheet<void>(… ScanResultsSheet(records: records, …));
```

Tres condiciones se combinan con **Y**: el ajuste global `saveHistory`, la ausencia de `privateMode`, y que el registro concreto no sea sensible. Nótese que la hoja de resultados recibe **`records`, no `persistent`**: lo sensible se muestra, simplemente no se guarda.

`isSensitive` es `parsed.sensitive`, que el intérprete pone a `true` para OTP, criptomonedas, los tres formatos de pago, AAMVA y **Wi-Fi solo si trae contraseña**. Un QR de red abierta sí se guarda.

---

3 · Lotes de imágenes y de PDF

3.1 El diálogo de progreso, y por qué se cierra así

`_runBatchDialog` resuelve un problema concreto: mostrar un diálogo modal mientras corre una operación asíncrona, y cerrarlo con seguridad tanto si termina como si falla.

```
bool dialogOpen = true;
final NavigatorState navigator = Navigator.of(context, rootNavigator: true);
final Future<void> dialog = showDialog<void>(…).whenComplete(() => dialogOpen = false);
await Future<void>.delayed(Duration.zero);
try {
 return await operation();
} finally {
 if (dialogOpen && navigator.mounted) navigator.pop();
 await dialog;
 progress.dispose();
 token.dispose();
}
```

Cuatro decisiones:

1. **`Navigator.of` se captura antes** de la operación. Después, el `context` podría estar desmontado y `Navigator.of(context)` lanzaría.
2. **`Future.delayed(Duration.zero)`** cede el turno para que el diálogo llegue a construirse antes de que empiece el trabajo. Sin eso, una operación rápida podría intentar cerrar un diálogo que aún no existe.
3. **`dialogOpen`** evita un `pop` de más si el diálogo ya se cerró por otra vía.
4. **`await dialog`** espera a que la animación de cierre termine antes de liberar `progress` y `token`, que el propio diálogo sigue escuchando.

3.2 Galería

```
final List<XFile> images = await _imagePicker.pickMultiImage(limit: 20);
```

El límite de 20 es del selector, no de la aplicación: el sistema operativo no deja seleccionar más.

Dentro del lote, por cada imagen: `token.throwIfCancelled()`, actualizar progreso, `_engine.analyzeImage(path)`, deduplicar por texto dentro de la imagen, y `await Future<void>.delayed(Duration.zero)` para ceder el turno al bucle de eventos —sin eso, el diálogo no se repintaría y el botón «Cancelar» no respondería.

El `finally` interno reanuda la cámara si procede; el externo distingue tres finales: cancelación (mensaje explícito de que **no se modificó el historial**), cualquier otro error, y éxito con cero códigos.

3.3 PDF

El caso del PDF tiene una complicación adicional: el trabajo son **dos fases** —renderizar páginas a PNG y después analizar esos PNG—, y el progreso debe reflejarlo. La solución es contar sobre `total * 2`:

```
onProgress: (current, total) => progress.update(
 label: 'Renderizando página $current de $total',
 current: current,
 total: total * 2,
),
```

y en la segunda fase `current: pages.length + index`. Así la barra recorre la mitad izquierda renderizando y la derecha analizando.

`PdfPageRenderer.pickAndRender` merece leerse con cuidado:

```
final double scale = _renderScale(page.width, page.height);
final PdfPageRenderCancellationToken renderToken = page.createCancellationToken();
void cancelRender() {
 if (cancellationToken?.isCancelled ?? false) renderToken.cancel();
}
cancellationToken?.addListener(cancelRender);
cancelRender();
try {
 pdfImage = await page.render(…, cancellationToken: renderToken);
} finally {
 cancellationToken?.removeListener(cancelRender);
}
```

La cancelación se propaga a **dos niveles**: el bucle del lote y el renderizador de la página. `cancelRender()` se llama una vez inmediatamente después de suscribir, para cubrir el caso de que el usuario ya hubiera cancelado antes de que se registrara el *listener*. Y el `removeListener` en `finally` evita acumular suscripciones página a página.

`_renderScale` acota la escala a `[1.5, 4.0]` apuntando a 2400 px en el lado mayor. Es el compromiso entre legibilidad de un código pequeño dentro de una página y memoria: un A4 a 4× son unos 3400×4800 píxeles.

La limpieza es exhaustiva. Hay tres caminos:

- **éxito con cero páginas útiles**: se borra el directorio temporal;
- **excepción**: `cleanup(rendered)` borra los PNG ya escritos y el directorio, y se relanza;
- **siempre**: `document.dispose()` en el `finally`.

Además el llamante vuelve a llamar a `cleanup(pages)` en su propio `finally`. Es idempotente: `cleanup` comprueba `exists()` antes de borrar.

4 · Interpretación del contenido

4.1 La cadena completa

`ScanRecord.fromBarcode` es donde convergen las tres piezas:

```
final String raw = payloadForBarcode(barcode);
final SecurityAssessment assessment = ScanSecurityAnalyzer.analyze(raw);
final ParsedContent parsed = ContentParserRegistry.instance.parse(raw);
final String digest = sha256.convert(utf8.encode('$raw|${timestamp.microsecondsSinceEpoch}')).toString().s
```

`payloadForBarcode` resuelve el caso incómodo de un código **binario**: si no hay texto legible pero sí bytes, devuelve `binary-base64:<base64>`. Ese prefijo es el que reconocen después tanto el analizador (`_isKnownStructuredContent`) como el intérprete (rama 1). Es un contrato interno de tres puntos que hay que cambiar a la vez si se toca.

`displayValue` se calcula aparte:

```
final String display = readable.isNotEmpty
   ? readable
   : binaryBytes == null ? raw : 'Datos binarios (${binaryBytes.length} bytes)\n$raw';
```

Es decir, para un código binario el valor mostrado antepone una línea legible al Base64. `rawValue` sigue siendo el Base64 puro, que es lo que se copia y se comparte.

`_decodedBytes` distingue dos tipos concretos con `is`:

```
if (value is DecodedBarcodeBytes) return value.bytes;
if (value is DecodedVisionBarcodeBytes) return value.bytes ?? value.rawBytes;
return null;
```

Está escrito así porque un `switch` sobre esos tipos tenía una rama inalcanzable que el analizador marcaba; el defecto está registrado en `VALIDATION.md`, apartado de correcciones de 1.0.0. `DecodedVisionBarcodeBytes` prioriza `bytes` sobre `rawBytes` porque Apple Vision entrega los dos y el primero ya viene decodificado.

4.2 `_segments`: el analizador de escapes

Es la función que hace posible interpretar un QR de Wi-Fi con puntos y comas o dos puntos dentro del nombre o de la contraseña:

```
for (final int rune in value.runes) {
 final String char = String.fromCharCode(rune);
 if (escaped) { current.write(char); escaped = false; }
 else if (char == r'\') { escaped = true; }
 else if (char == ';') { parts.add(current.toString()); current.clear(); }
 else { current.write(char); }
}
```

Máquina de estados de dos estados sobre puntos de código, no sobre unidades UTF-16: `value.runes` evita partir un carácter fuera del plano básico. Después, cada parte se divide por el **primer** `:`, de modo que un valor que contenga dos puntos sin escapar no rompe la clave.

Hay prueba dedicada con el caso exacto: `WIFI:T:WPA;S:Mi\;Red;P:clave\:segura;H:false;;` debe producir `Red = Mi;Red` y `Contraseña = clave:segura`.

4.3 `_tlv`: EMVCo sin lanzar

```
while (index + 4 <= raw.length) {
 final String id = raw.substring(index, index + 2);
```

```
 final int? length = int.tryParse(raw.substring(index + 2, index + 4));
 if (length == null || index + 4 + length > raw.length) break;
 result[id] = raw.substring(index + 4, index + 4 + length);
 index += 4 + length;
}
```

`break` en vez de excepción: un QR de pago truncado o mal formado devuelve los campos que sí se pudieron leer, en lugar de perderse por completo. El resultado siempre incluye el campo `Carga` con el texto íntegro, para que el usuario pueda verificarlo aunque la interpretación sea parcial.

4.4 Los formatos de pago por posición de línea

Swiss QR Bill y EPC/SEPA se interpretan **por número de línea**:

```
if (lines.length > 3) 'IBAN': lines[3],
if (lines.length > 5) 'Nombre': lines[5],
if (lines.length > 18) 'Monto': lines[18],
```

Es lo que especifican ambos estándares —son formatos posicionales— pero significa que una variante de versión con líneas insertadas produciría campos desplazados sin ningún aviso. Los `if (lines.length > N)` evitan el desbordamiento, no el desalineamiento. R-18.

Los cuatro tipos de pago y el documento AAMVA marcan `sensitive: true`, lo que los deja fuera del historial automático y hace que sus campos aparezcan enmascarados si `hideSensitiveValues` está activo.

4.5 Cómo decide `ScanRecordCard` qué enmascarar

```
final bool conceal = record.isSensitive && widget.settings.value.hideSensitiveValues && !_revealed;
…
final bool sensitiveField = record.parsed.sensitive &&
   <String>{'Contraseña', 'Secreto', 'Dirección', 'IBAN', 'Carga'}.contains(field.key);
final String value = conceal && sensitiveField ? '••••••••' : field.value;
```

El enmascaramiento es **por campo**, con una lista de cinco nombres. Un registro sensible muestra el resto de sus campos en claro: en un QR de Wi-Fi el nombre de la red se ve y la contraseña no. El botón del ojo alterna `_revealed`, que es estado local del widget y **no se persiste**.

Consecuencia que conviene saber: un campo nuevo con datos sensibles y otro nombre —por ejemplo `Clave` o `Semilla`— **no se enmascararía**. La lista es literal. R-19.

---

5 · Cifrado y persistencia

5.1 El sobre

```
return jsonEncode(<String, Object?>{
 'version': currentVersion,
 'algorithm': currentAlgorithm,
 'keyId': selectedKeyId,
 'createdAt': DateTime.now().toUtc().toIso8601String(),
 'payload': <String, String>{
   'nonce': base64Encode(box.nonce),
   'cipherText': base64Encode(box.cipherText),
   'mac': base64Encode(box.mac.bytes),
 },
});
```

Los cuatro campos de cabecera son **metadatos en claro** y están ahí a propósito: sin ellos, un descifrador futuro no podría decidir con qué llave ni con qué algoritmo intentar. `createdAt` no se usa en ninguna decisión del código

—**comprobado**: no aparece leído en ningún sitio— y existe como información forense.

El sobre heredado es simplemente el objeto `payload` suelto. La detección es la ausencia de `version`, no una versión `1` explícita, porque el formato inicial no escribía ningún número.

5.2 La doble barrera de la llave ausente

```
final SecretKey? key = await _keyProvider.read(keyId);
if (key == null) {
 _missingKeyIds.add(keyId);
 throw StateError('encryption_key_missing:$keyId');
}
_missingKeyIds.remove(keyId);
```

y, en `encryptJson`:

```
if (_missingKeyIds.contains(selectedKeyId)) {
 throw StateError('encryption_key_missing:$selectedKeyId');
}
```

Es la propiedad más importante del cifrador. Sin la segunda comprobación: `decryptJson` fallaría, el repositorio registraría la incidencia, y la siguiente escritura llamaría a `readOrCreate`, que **crearía una llave nueva bajo el mismo identificador**. A partir de ahí los registros antiguos serían irrecuperables incluso si la llave original reapareciera, porque el almacén ya tendría otra.

La cuarta línea —`_missingKeyIds.remove(keyId)` tras un descifrado correcto— es la salida: si la llave vuelve a aparecer (por ejemplo tras una restauración del sistema), el bloqueo se levanta solo.

Hay una prueba que recorre exactamente este camino: `test/core/payload_cipher_test.dart`, «a missing decryption key is never replaced silently».

5.3 Actualización perezosa de sobres

Ambos repositorios hacen lo mismo tras un descifrado correcto:

```
final CipherEnvelopeInfo info = _cipher.inspect(encrypted);
if (info.legacy || info.version < PayloadCipher.currentVersion) {
 await _store.record(snapshot.key).update(_database.database, <String, Object?>{
   'payload': await _cipher.upgradeEnvelope(encrypted),
 });
}
```

`inspect` no descifra, así que la comprobación es barata y ocurre en cada lectura. La reescritura solo cuando hace falta.

Cuidado: la condición **no compara el `keyId`**. `upgradeEnvelope` sí lo hace (`info.keyId == _activeKeyId`), pero como aquí no se llama salvo que la versión sea antigua, un registro cifrado con una llave anterior pero versión actual **no se actualiza al leerlo**. Lo actualiza la rotación completa, no la lectura. Es coherente —rotar es una operación explícita— pero no está documentado.

5.4 Migración del historial desde preferencias

Es el flujo con más precauciones de todo el repositorio, y por buenas razones: convierte un historial en claro guardado en preferencias a la base cifrada, y un fallo a mitad podría perder datos del usuario.

```
final String encryptedBackup = await _cipher.encryptJson(<String, Object?>{
 'schemaVersion': 1,
 'encoding': 'utf8-base64',
 'raw': base64Encode(utf8.encode(stored)),
});
```

```
await _preferences.setString(_legacyBackupKey, encryptedBackup);
```

Paso 1: respaldo cifrado del origen exacto, antes de tocar nada. El comentario lo dice: «Preserve the exact legacy source, but never duplicate it in clear text». Se guarda el texto original en Base64 dentro de un sobre cifrado, no los registros reinterpretados: si la reinterpretación fuera la que falla, el respaldo sigue siendo fiel.

```
await _migrateRecordsAtomically(records);
```

Paso 2: inserción con verificación dentro de la transacción.

```
final Set<String> before = await ids();
final Set<String> expected = <String>{...before, ...records.map((r) => r.id)};
if (expected.length > maxItems) throw StateError('migration_limit_exceeded');
// … precifrado fuera de la transacción …
await _database.database.transaction((transaction) async {
 for (…) { await _store.record(…).put(transaction, …); }
 final stored = await _store.find(transaction);
 final actual = stored.map((i) => i.key).toSet();
 if (!actual.containsAll(expected)) throw StateError('migration_verification_failed');
});
```

La verificación ocurre **dentro** de la transacción, así que lanzar la revierte por completo. Y el cifrado se precalcula fuera para no mantener la base bloqueada durante miles de operaciones AES.

```
await _preferences.remove(_legacyStorageKey);
await _preferences.remove(_migrationErrorKey);
await _preferences.setBool(_migrationKey, true);
```

Paso 3: solo entonces se borra el origen.

Y si algo falla:

```
} on Object catch (error) {
 final String code = 'legacy_${error.runtimeType}';
 await _preferences.setString(_migrationErrorKey, code);
 await _recovery.record(entityType: RecoveryEntityType.migration, entityId: 'history_v1_to_v2', code: code
 …
 return HistoryMigrationStatus(completed: false, errorCode: code, backupPresent: backupPresent);
}
```

El origen queda intacto, la conversión **no se marca completada** (así que se reintentará en el siguiente arranque), se crea una incidencia en el Centro de recuperación y Ajustes muestra la opción «Reintentar migración del historial».

En una instalación limpia todo esto se resuelve en dos líneas: no hay `scan_history_v1`, se marca completada y se devuelve.

5.5 Recorte del historial

```
Future<void> _trim() async {
 final int count = await _store.count(_database.database);
 if (count <= maxItems) return;
 final old = await _store.find(_database.database,
   finder: Finder(sortOrders: [SortOrder('scannedAt')], limit: count - maxItems));
 await _database.database.transaction((t) async {
   for (final snapshot in old) await _store.record(snapshot.key).delete(t);
 });
}
```

Ordena **ascendente** por `scannedAt` (`SortOrder('scannedAt')` sin `false`) y borra exactamente el excedente: los más antiguos. El campo en claro `scannedAt` existe precisamente para poder hacer esto sin descifrar 5000 registros.

6 · Rotación de llave

`DataMaintenanceService.rotateEncryptionKey` merece leerse entera porque su corrección depende del orden.

```
if (_rotationInProgress) throw StateError('encryption_rotation_in_progress');
_rotationInProgress = true;
final String keyId = 'v3_${DateTime.now().toUtc().millisecondsSinceEpoch}';
```

La bandera es la protección contra reentrada. La interfaz también la tiene (`_rotatingKey` deshabilita la fila), pero la clase no confía en la interfaz.

El identificador incluye la marca de tiempo, así que dos rotaciones producen llaves distintas y ninguna sobrescribe a la anterior en el almacén seguro. *Inferencia*: las llaves antiguas **no se borran** tras una rotación exitosa; quedan en el Keychain o Keystore bajo su propio `ucs_database_key_v3_…`. Es prudente (permite recuperar un registro que se hubiera quedado atrás) pero significa que el almacén crece una entrada por rotación. R-20.

```
for (final item in history) {
 final String? payload = item.value['payload'] as String?;
 if (payload == null) throw StateError('history_payload_missing:${item.key}');
 historyPayloads[item.key] = await _cipher.encryptJson(await _cipher.decryptJson(payload), keyId: keyId);
}
```

Un registro sin `payload` **aborta la rotación completa** en vez de saltárselo. Es la decisión correcta: rotar dejando registros atrás produciría una base con dos llaves activas de facto. Hay prueba dedicada («rotation aborts before changing metadata when a payload is incomplete»).

```
await _database.database.transaction((transaction) async {
 for (…) await _history.record(…).update(transaction, …);
 for (…) await _inventory.record(…).update(transaction, …);
 await _metadata.saveActiveKeyId(keyId, transaction: transaction);
});
_cipher.activateKey(keyId);
```

Los datos y el metadato de llave activa se escriben en **la misma transacción**. Si se separaran, un fallo entre ambas dejaría la base con datos nuevos y llave declarada vieja, o al revés. Y `activateKey` —que solo cambia una variable en memoria— ocurre **después** de que la transacción confirme.

```
} on Object {
 try { await _cipher.forgetKey(keyId); } on Object { }
 rethrow;
} finally {
 _rotationInProgress = false;
}
```

El `catch` limpia la llave huérfana. El `try` anidado y vacío está ahí para que un fallo al borrar **no sustituya al error original de la rotación**, que es el que el usuario necesita ver. El comentario del código lo explica. Hay prueba dedicada («failed rotation removes a newly generated orphan key»).

---

7 · Inventario continuo

7.1 La cola serial

Nueve métodos de `InventoryStore` pasan por `_writes.run`. El patrón es siempre el mismo:

```
Future<void> addScan(ScanRecord record) => _writes.run<void>(() async {
     final InventorySession? session = activeSession;
     if (session == null) return;
     …
```

```
    await _replaceUnlocked(session.copyWith(items: items));
  });
```

`_replaceUnlocked` se llama así porque **asume que ya está dentro de la cola**. Llamarlo desde fuera rompería la garantía. No hay nada en el código que lo impida; el nombre es toda la protección. *Inferencia*: es una convención, no un mecanismo.

La razón de la cola es concreta: el inventario recibe lecturas continuas de la cámara mientras el usuario edita cantidades y notas. `InventorySession` es inmutable, así que dos escrituras concurrentes leerían el mismo estado base y la segunda perdería el cambio de la primera.

`AsyncWriteQueue` tiene una propiedad que merece nombrarse:

```
_tail = _tail.then<void>((_) async {
 try { completer.complete(await operation()); }
 on Object catch (error, stack) { completer.completeError(error, stack); }
});
```

El error se entrega al `Completer` del llamante, pero **la cadena `_tail` siempre resuelve con éxito**. Un fallo no envenena la cola. Hay prueba dedicada.

7.2 El conteo

```
final InventoryItem? existing = items[record.rawValue];
items[record.rawValue] = existing == null
   ? InventoryItem(code: record.rawValue, format: record.format,
       label: record.parsed.summary ?? record.contentType,
       quantity: 1, firstScannedAt: now, lastScannedAt: now)
   : existing.copyWith(quantity: existing.quantity + 1, lastScannedAt: now);
```

La clave del mapa es el **texto bruto del código**. La etiqueta se toma de `parsed.summary` si existe (el dominio de una URL, el nombre de un contacto, el GTIN de un GS1) y si no del tipo de contenido.

`setQuantity(code, 0)` o negativo **elimina el ítem** del mapa. Es lo que hace «Quitar del inventario», y también lo que hace pulsar «restar» sobre un ítem con una unidad.

7.3 Lo que el inventario no filtra

`_onDetect` de `InventoryScreen` llama a `widget.store.addScan(record)` **sin comprobar `record.isSensitive`**. Un QR de Wi-Fi con contraseña, un OTP o un QR de pago escaneado en una sesión de inventario **se persiste** —cifrado, como todo lo demás— con su texto íntegro como clave del ítem.

Es una asimetría real con `_persistAndShow`, que sí filtra. Registrada como R-03. No es una fuga: el dato está cifrado con la misma llave que el historial. Pero contradice la garantía enunciada en `docs/SECURITY.md` punto 5 («No persistir automáticamente OTP ni Wi-Fi con contraseña») y en la política de privacidad.

7.4 La cámara del inventario

`InventoryScreen` **no usa `ScannerEngine`**. Construye su propio `MobileScannerController` y lo maneja directamente. Sus opciones difieren de las de `MobileScannerEngine`:

Opción	`MobileScannerEngine`	`InventoryScreen`
`detectionSpeed`	`noDuplicates`	`noDuplicates`
`formats`	`[]` (todos)	`[]` (todos)
`invertImage`	`true`	`true`

`autoZoom`	`true`	`true`
`torchEnabled`	`_settings.autoTorch`	**no se pasa**
`returnImage`	`false`	**no se pasa**

`autoTorch` no se aplica en el inventario. Es una diferencia de comportamiento observable y no documentada. R-04.

Su marco de lectura es un rectángulo horizontal (`width: maxWidth.clamp(180,260)`, `height: 120`) en lugar del cuadrado del escáner, porque los códigos de producto son lineales y anchos.

Y su ventana antirrebote es de **1200 ms sobre un solo código**, no de 2 s sobre la combinación: en un inventario, escanear dos veces el mismo producto debe poder sumar dos unidades sin esperar dos segundos.

---

8 · Recuperación

8.1 Cuándo se crea una incidencia

Ambos repositorios registran en dos casos: `payload` ausente (`code: 'missing_payload'`) y descifrado fallido (`code: 'decrypt_<Tipo>'`, con la carga cifrada guardada). La migración registra un tercero (`legacy_<Tipo>`).

El `id` de la incidencia es determinista:

```
final String source = '${entityType.name}|$entityId|$code';
final String id = sha256.convert(source.codeUnits).toString().substring(0, 24);
```

Lo cual significa que **la misma incidencia detectada dos veces no se duplica**: la segunda sobrescribe a la primera con estado `unresolved`. Es lo que evita que cada arranque llene la lista con la misma entrada. El efecto lateral es que marcar «revisado» un problema recurrente lo devuelve a la lista en la siguiente lectura.

8.2 `retry`

```
final Map<String, dynamic> clear = await _cipher.decryptJson(encrypted);
final String upgraded = await _cipher.upgradeEnvelope(encrypted);
switch (issue.entityType) {
 case RecoveryEntityType.history:
   final ScanRecord record = ScanRecord.fromJson(clear);
   await _history.record(issue.entityId).put(…, {'id': record.id, 'scannedAt': …, 'payload': upgraded});
```

Descifra **y reconstruye el modelo** antes de reinsertar. Si el JSON descifrado no fuera un `ScanRecord` válido, `fromJson` lanza y el `catch` global devuelve `false`: no se reinserta un registro corrupto.

Nótese que se reinserta con `record.id` recalculado del JSON, no con `issue.entityId`. En un caso normal coinciden.

El `catch` de `retry` es `on Object { return false; }`: cualquier fallo se convierte en un `false` que la interfaz traduce a «No fue posible recuperar el elemento con la llave disponible». Es correcto para el usuario, pero **se pierde la causa**: no se registra en `AppDiagnostics`. R-21.

8.3 `exportBundle`

```
'notice': 'Encrypted payloads remain encrypted. No decryption key is included.',
'issues': issues.map((i) => i.toJson()).toList(growable: false),
```

El paquete contiene los sobres **tal cual**, todavía cifrados. Hay una prueba que lo verifica: cifra un objeto con la palabra `never-export-clear`, genera el paquete y comprueba que ni esa palabra ni la cadena `ucs_database_key`

aparecen.

---

9 · Generador

Flujo simple: nueve tipos de carga y diez formatos visuales, ambos como enumeraciones privadas. `_payload` construye la cadena según el tipo, y `BarcodeWidget` la dibuja.

Dos detalles merecen mención.

El escape del Wi-Fi:

```
static String _escape(String value) => value
   .replaceAll(r'\', r'\\')
   .replaceAll(';', r'\;')
   .replaceAll(':', r'\:')
   .replaceAll(',', r'\,');
```

El orden importa: la barra invertida se escapa **primero**, o se escaparían las barras que la propia función acaba de introducir. Es la operación inversa de `ContentInterpreter._segments`, así que un QR generado por esta pantalla se reinterpreta correctamente al escanearlo.

La normalización de URL:

```
static String _normalizeUrl(String value) =>
   RegExp(r'^https?://', caseSensitive: false).hasMatch(value.trim())
       ? value.trim() : 'https://${value.trim()}';
```

Antepone `https://`, nunca `http://`.

El PNG se produce capturando el `RepaintBoundary` a `pixelRatio: 3`; el SVG lo genera `barcode_widget` a resoluciones fijas (900×900 para 2D, 1200×420 para lineales). Ambos se comparten con `share_plus` sin escribir a disco: van como `XFile.fromData`.

`errorBuilder` de `BarcodeWidget` cubre el caso de un contenido inválido para el formato elegido —por ejemplo, letras en un EAN-13— con un mensaje explícito.

---

10 · Exportación

```
static Future<void> _shareCsv(List<List<String>> rows, String name, String title) async {
 final Uint8List bytes = Uint8List.fromList(await compute<…>(_encodeCsv, rows));
 await SharePlus.instance.share(ShareParams(title: title, files: [XFile.fromData(bytes, mimeType: 'text/cs
}
```

`compute` mueve la codificación a otro isolate. Con 5000 registros, generar el CSV en el isolate principal produciría un salto visible en la interfaz.

`_encodeCsv` y `_encodeXlsx` son **funciones de nivel superior**, no métodos estáticos de la clase: `compute` exige un punto de entrada de nivel superior o estático, y en Dart 3 una función de nivel superior es la forma que siempre funciona.

```
List<int> _encodeCsv(List<List<String>> rows) {
 String csvCell(String value) => '"${value.replaceAll('"', '""')}"';
 final String csv = rows.map((row) => row.map(csvCell).join(',')).join('\r\n');
 return <int>[0xEF, 0xBB, 0xBF, ...utf8.encode(csv)];
}
```

Tres decisiones que solo tienen sentido si el destino es Excel en Windows: **todas** las celdas entrecomilladas (no solo las que lo necesitan), `\r\n` como separador de línea, y el BOM UTF-8 al principio. Sin el BOM, Excel abre el archivo en la página de códigos del sistema y los acentos salen rotos.

`_encodeXlsx` crea la hoja con el nombre pedido y **borra la hoja por defecto** si el nombre difiere:

```
final String firstSheet = workbook.getDefaultSheet() ?? 'Sheet1';
final Sheet sheet = workbook[sheetName];
…
if (firstSheet != sheetName) workbook.delete(firstSheet);
```

Sin esa última línea, cada archivo llevaría una hoja «Sheet1» vacía junto a la buena.

---

11 · Pantallas resumidas

`HistoryScreen` (284 líneas). Filtra en memoria sobre `store.history` por texto —que busca en `rawValue`, `contentType`, `format`, etiquetas y notas—, por nivel de riesgo y por favoritos. Se suscribe al store con `addListener` y llama a `setState` vacío. Exporta e importa desde un menú. El diálogo de importación muestra la vista previa (válidos, duplicados, rechazados) **antes** de ofrecer las tres estrategias. Nada se escribe hasta elegir una.

`GeneratorScreen` (310 líneas). Ver el apartado 9.

`FormatsScreen` (170 líneas). Puramente informativa: tres grupos con 20 nombres de formato escritos a mano y dos tarjetas de texto. No consulta al motor ni a `BarcodeLabels`.

`SettingsScreen` (378 líneas). Cinco secciones de `SwitchListTile`, `DropdownButton` y `ListTile`. Las tres operaciones con consecuencias —rotar llave, borrar historial, reintentar migración— tienen diálogo de confirmación. `_rotateEncryptionKey` usa el mismo patrón de diálogo bloqueante que `_runBatchDialog`, con `dialogOpen` y `navigator` capturado por adelantado.

`RecoveryScreen` (139 líneas). `FutureBuilder` sobre `service.load(unresolvedOnly: false)` —muestra también las resueltas— con un menú de cuatro acciones por incidencia. Solo ofrece «Reintentar recuperación» si `issue.encryptedPayload != null`, y «Reintentar migración» solo para incidencias de tipo `migration`.

---

12 · Lo que un lector nuevo descubriría a golpes

Recopilado de todo lo anterior, para no tener que releerlo:

1. **Un clon no compila.** Falta `python tool/bootstrap.py`.
2. **Leer el historial escribe en la base**, si algún sobre está desactualizado.
3. **El momento de cada lectura es visible sin la llave**: `scannedAt` va en claro para poder ordenar y podar.
4. **Un registro que no descifra no rompe la carga**; se omite y se registra.
5. **El inventario sí guarda contenido sensible**; el historial no.
6. **`autoTorch` no se aplica en el inventario.**
7. **El temporizador del portapapeles sobrevive al widget** que lo creó, y comprueba que el contenido no haya cambiado antes de borrar.
8. **Los diagnósticos se pierden al cerrar**: viven solo en memoria.
9. **Las llaves antiguas no se borran** tras una rotación exitosa.
10. **`restablecer preferencias visuales` no toca** `save_history`, `private_mode` ni `biometric_lock`.

11. **Desinstalar borra el historial para siempre**: la llave se va con la aplicación y no hay copia remota porque no hay servidor.
12. **Lo exportado va en claro.** Un CSV del historial es un archivo normal.
13. **Un EAN-13 que empiece por 978 o 979 se clasifica como ISBN**, no como producto.
14. **La lista de campos que se enmascaran es literal**: `Contraseña`, `Secreto`, `Dirección`, `IBAN`, `Carga`.

Continúa en 07 · Base de datos.

07 · Base de datos y persistencia

Conclusión primero

Universal Code Scanner **no usa SQL**. Su persistencia son tres mecanismos distintos, cada uno con una responsabilidad separada:

Mecanismo	Qué guarda	Cifrado
Sembast (NoSQL local, documental)	Historial, sesiones de inventario, incidencias de recuperación, metadatos de esquema y de cifrado	**Sí**, el contenido; no las claves ni las marcas de tiempo de ordenación
`flutter_secure_storage` (Keychain / Keystore)	Las llaves AES-256	Lo protege el sistema operativo
`SharedPreferencesAsync`	Preferencias de usuario y estado de migración	**No**

No hay ORM, no hay migraciones SQL, no hay servidor de base de datos. El motivo es coherente con el producto: una aplicación que no envía nada a ningún sitio no necesita un esquema relacional que compartir, y Sembast es un archivo JSON apend-only sin dependencias nativas.

El motor: Sembast

Aspecto	Valor	Archivo
Paquete nativo	`sembast 3.8.9+1`	`pubspec.yaml`
Paquete web	`sembast_web 2.4.5+1` (sobre IndexedDB)	idem
Nombre del archivo	`universal_code_scanner_v2.db`	`database_opener_io.dart`, `database_opener_web.dart`
Ubicación nativa	`getApplicationSupportDirectory()`	`database_opener_io.dart`
Ubicación web	IndexedDB del origen	`database_opener_web.dart`
`version` de Sembast	`2`	Ambos abridores
Plataforma no soportada	Lanza `UnsupportedError`	`database_opener_stub.dart`

La selección de implementación es una exportación condicional:

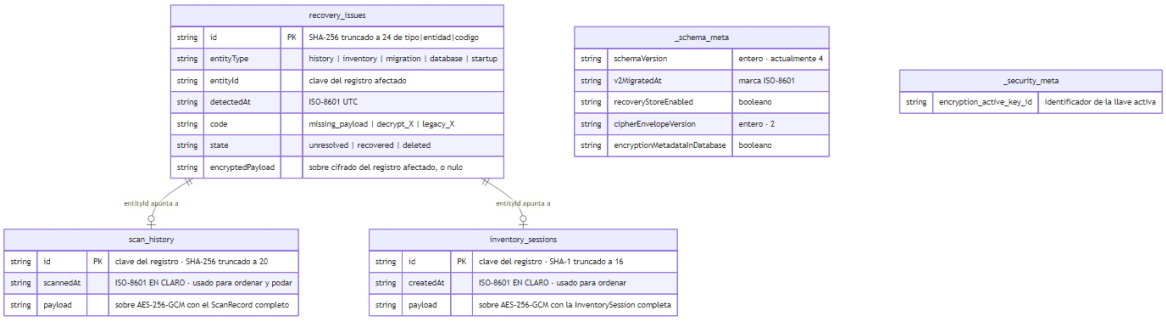
```
export 'database_opener_stub.dart'
   if (dart.library.io) 'database_opener_io.dart'
   if (dart.library.js_interop) 'database_opener_web.dart';
```

Las dos versiones que no hay que confundir. Sembast recibe `version: 2` al abrir el archivo, y ese número no se ha tocado desde que el archivo pasó a llamarse `…_v2.db`. `SchemaMigrator` mantiene **su propio** contador en el almacén `_schema_meta`, actualmente en `4`. Son independientes: el primero es la versión de formato de Sembast; el segundo, la del esquema lógico de la aplicación.

La base temporal

`AppDatabase.openTemporary()` abre `databaseFactoryMemory` con nombre `ucs_temporary_<microsegundos>`. Nunca toca disco, se pierde al cerrar y el nombre único evita que dos aperturas en la misma sesión compartan estado. Es lo que respalda el «modo temporal» de la pantalla de inicio seguro, y también lo que usan todas las pruebas que necesitan una base.

Diagrama de almacenes



Lo que muestra: los cinco almacenes y qué campos van en claro.

Lo que no muestra: que Sembast no impone ninguna integridad referencial. La relación entre `recovery_issues.entityId` y las claves de los otros dos almacenes es **una convención del código**, no una restricción del motor. Si el usuario borra un registro del historial, la incidencia asociada sigue existiendo y apuntando a una clave inexistente; su acción «Descartar» simplemente no encontrará nada que borrar y marcará la incidencia como `deleted`.

Tampoco muestra que el contenido de `payload` es opaco: los campos reales del `ScanRecord` viven dentro y no son consultables por Sembast.

Diccionario de datos

Almacén `scan_history`

`stringMapStoreFactory.store('scan_history')`. Escrito por `HistoryRepository`, `RecoveryService.retry` y `DataMaintenanceService.rotateEncryptionKey`.

Campo	Tipo	En claro	Origen	Uso
`id`	`String`	Sí	`ScanRecord.id`	Clave del registro; tam bién duplicado como c ampo
`scannedAt`	`String` ISO-8601	**Sí**	`ScanRecord.scannedA t`	`SortOrder` en `load`, `Fi lter.lessThan` en `pru neOlderThan`, `SortOrd er` ascendente en `_tr im`
`payload`	`String` (JSON del sob re)	No	`PayloadCipher.encry ptJson(record.toJso n())`	Todo el contenido

Dentro del `payload`, una vez descifrado, está el `ScanRecord` completo:

Campo	Tipo	Nulo	Significado

`id`	`String`	no	SHA-256 de `rawValue|microsegundos`, 20 caracteres
`rawValue`	`String`	no	Texto exacto del código, o `binary-base64:<base64>`
`displayValue`	`String`	no	Lo que se muestra; para binario antepone `Datos binarios (N bytes)`
`format`	`String`	no	Etiqueta legible de la simbología (`BarcodeLabels.format`)
`contentType`	`String`	no	`ParsedContent.title`
`scannedAt`	`String` ISO-8601	no	Momento de la lectura, hora local
`source`	`String`	no	`Cámara`, `Imagen`, `PDF · página N`, `Inventario`, `Importado`
`riskLevel`	`String`	no	`low`, `caution` o `high`
`riskReasons`	`List<String>`	no	Motivos en español; puede estar vacía
`canOpen`	`bool`	no	Si existe una acción externa posible
`parsed`	`Map`	no	`ParsedContent` serializado
`favorite`	`bool`	no	Marca del usuario
`tags`	`List<String>`	no	Etiquetas del usuario, deduplicadas
`notes`	`String`	no	Nota libre del usuario

Y dentro de `parsed`:

Campo	Tipo	Significado
`kind`	`String`	Uno de los 17 valores de `ContentKind`
`title`	`String`	Nombre legible del tipo, en español
`summary`	`String?`	Resumen de una línea: dominio, nombre del contacto, GTIN…
`fields`	`Map<String,String>`	Campos interpretados, con nombre en español
`sensitive`	`bool`	Decide si se persiste automáticamente y si se enmascara

Nota sobre las fechas. `scannedAt` se guarda con `toIso8601String()` sobre un `DateTime` **local** (`DateTime.now()`), sin sufijo de zona. Comparar registros creados en zonas horarias distintas produciría un orden incorrecto. Los `detectedAt` de recuperación y los `createdAt`/`exportedAt` de los sobres sí usan `toUtc()`. Es una

inconsistencia real. R-22.

Almacén `inventory_sessions`

`stringMapStoreFactory.store('inventory_sessions')`. Escrito por `InventoryRepository`, `RecoveryService.retry` y la rotación.

Campo	Tipo	En claro	Uso
`id`	`String`	Sí	SHA-1 de `microsegundos|nombre`, 16 caracteres
`createdAt`	`String` ISO-8601	**Sí**	`SortOrder('createdAt', false)` en `load`
`payload`	`String`	No	Sobre cifrado con la sesión completa

Dentro del `payload`, `InventorySession`:

Campo	Tipo	Nulo	Significado
`id`	`String`	no	El mismo de la clave
`name`	`String`	no	Nombre dado por el usuario, o `Inventario DD-MM-AAAA`
`createdAt`	`String`	no	Creación
`closedAt`	`String?`	**sí**	`null` mientras la sesión está abierta
`items`	`Map<String, InventoryItem>`	no	**Indexado por el texto bruto del código**

Y cada `InventoryItem`:

Campo	Tipo	Significado	Inmutable tras crear
`code`	`String`	Texto bruto del código; igual a la clave	Sí
`format`	`String`	Simbología	Sí
`label`	`String`	`parsed.summary` o `contentType`	Sí
`quantity`	`int`	Unidades contadas	No
`firstScannedAt`	`String`	Primera lectura	Sí
`lastScannedAt`	`String`	Última lectura o edición	No
`notes`	`String`	Nota del usuario	No

`InventoryItem.fromJson` es tolerante: `format`, `label`, `quantity` y `notes` tienen valores por defecto (`'Desconocido'`, `'Producto'`, `1`, `''`). `code`, `firstScannedAt` y `lastScannedAt` **no**: su ausencia lanza. Es lo que hace que un JSON importado sin esos campos se rechace.

Almacén `recovery_issues`

`stringMapStoreFactory.store('recovery_issues')`. Escrito solo por `RecoveryRepository`.

Campo	Tipo	Significado
`id`	`String`	SHA-256 de `entityType|entityId|code`, 24 caracteres. **Determinista**: la misma incidencia no se duplica
`entityType`	`String`	Uno de los 5 valores; solo 3 se emiten
`entityId`	`String`	Clave del registro afectado, o `history_v1_to_v2` para la migración
`detectedAt`	`String` ISO-8601 **UTC**	Momento de la detección
`code`	`String`	`missing_payload`, `decrypt_<Tipo>`, `legacy_<Tipo>`
`state`	`String`	`unresolved`, `recovered`, `deleted`
`encryptedPayload`	`String?`	El sobre del registro afectado, **todavía cifrado**

Todo este almacén se guarda **sin cifrar**, pero no contiene datos del usuario: solo identificadores técnicos y, cuando existe, un sobre que ya está cifrado.

Almacén `_schema_meta`

`StoreRef<String, Object?>('_schema_meta')`. Escrito solo por `SchemaMigrator`.

Clave	Tipo	Escrita en	Valor
`schemaVersion`	`int`	Siempre	Versión final tras migrar; actualmente `4`
`v2MigratedAt`	`String`	Migración a 2	Marca ISO-8601 UTC
`recoveryStoreEnabled`	`bool`	Migración a 3	`true`
`cipherEnvelopeVersion`	`int`	Migración a 3	`2`
`encryptionMetadataInDatabase`	`bool`	Migración a 4	`true`

Las cuatro marcas descriptivas no las lee nadie. Comprobado con `grep`: `v2MigratedAt`, `recoveryStoreEnabled`, `cipherEnvelopeVersion` y `encryptionMetadataInDatabase` solo aparecen en `schema_migrator.dart`. Son registro histórico, no configuración. R-23.

Almacén `_security_meta`

`StoreRef<String, Object?>('_security_meta')`. Escrito por `EncryptionMetadataRepository` y por la rotación.

Clave	Tipo	Significado
`encryption_active_key_id`	`String`	Identificador de la llave con la que se cifra ahora: `v2` o `v3_<epoch_ms>`

Migraciones de esquema

`SchemaMigrator.migrate()` es idempotente y transaccional:

```
final int initial = await _meta.record('schemaVersion').get(database) as int? ?? 1;
int version = initial;
final List<int> applied = <int>[];
await database.transaction((transaction) async {
 if (version < 2) { … version = 2; applied.add(2); }
 if (version < 3) { … version = 3; applied.add(3); }
 if (version < 4) { … version = 4; applied.add(4); }
 await _meta.record('schemaVersion').put(transaction, version);
});
```

Paso	Desde	Hasta	Qué hace
1	1 (o base nueva)	2	Escribe `v2MigratedAt`
2	2	3	Reserva la recuperación (`recoveryStoreEnabled`) y declara el sobre v2 (`cipherEnvelopeVersion`)
3	3	4	Declara que los metadatos de cifrado viven en la base (`encryptionMetadataInDatabase`)

Ninguno de los tres pasos crea almacenes ni transforma datos: **Sembast crea los almacenes de forma perezosa** al primer uso, y el comentario del código lo dice explícitamente. Las migraciones son, por tanto, marcas de estado.

Una base nueva parte de `1` (el `?? 1` del principio) y aplica los tres pasos en una sola transacción. Una segunda llamada no aplica ninguno: la prueba `test/core/schema_migrator_test.dart` verifica exactamente eso —`first.applied` no vacía y `second.applied` vacía— y que `toVersion` es `currentVersion` en ambos casos.

`SchemaMigrationResult` devuelve `fromVersion`, `toVersion` y `applied`, pero **el resultado se descarta**: `AppDatabase.open()` hace `await SchemaMigrator(...).migrate();` sin usarlo. Solo la prueba lo consume. R-24.

El sobre de cifrado

Cada `payload` es un JSON con esta forma:

```
{
 "version": 2,
 "algorithm": "AES-256-GCM",
 "keyId": "v2",
 "createdAt": "2026-08-27T12:00:00.000Z",
 "payload": {
   "nonce": "<base64>",
   "cipherText": "<base64>",
   "mac": "<base64>"
 }
}
```

Los cuatro campos de cabecera van **en claro** a propósito: sin ellos no habría forma de decidir con qué llave y con qué algoritmo intentar el descifrado. No revelan nada del contenido.

`createdAt` **no se lee en ninguna decisión del código** —comprobado con `grep`— y existe como dato forense.

El sobre heredado

El formato inicial no escribía cabecera: el `payload` era directamente el objeto `{nonce, cipherText, mac}`. La detección es la **ausencia** del campo `version`:

```
final bool legacy = !envelope.containsKey('version');
```

Un sobre heredado se descifra con `legacyKeyId`, que vale `'v2'` —el mismo valor que `currentKeyId`. Ver la nota R-13 en 05-technical-reference.md.

Reglas de compatibilidad

Caso	Comportamiento
`version` entre 1 y `currentVersion`	Se acepta
`version` mayor que `currentVersion`	`FormatException('Versión de cifrado no compatible: N')`
`algorithm` distinto de `AES-256-GCM`	`FormatException('Algoritmo no compatible: X')`
Llave del `keyId` ausente	`StateError('encryption_key_missing:<keyId>')` y bloqueo de escritura con ese identificador
MAC que no valida	Excepción de `cryptography`; el registro se aísla como incidencia

Las tres primeras tienen prueba dedicada en `test/core/payload_cipher_test.dart`.

Actualización perezosa

Tras cada descifrado correcto, ambos repositorios comprueban con `inspect` —que no descifra— si el sobre es heredado o de versión anterior, y en ese caso lo reescriben. **Leer el historial escribe en la base.** La condición **no compara el `keyId`**, así que un registro cifrado con una llave anterior pero versión actual no se actualiza al leerlo: eso lo hace la rotación.

Rotación de llave

Detallada en 06-deep-code-explanation.md, apartado 6. Resumen del contrato de datos:

1. La nueva llave se llama `v3_<epoch_ms>`.
2. Todos los sobres de **ambos** almacenes se recifran.
3. Registros y metadato `encryption_active_key_id` se escriben en **una sola transacción**.
4. Un registro sin `payload` aborta la operación completa.
5. Si falla, la llave nueva se borra del almacén seguro.
6. Las llaves **anteriores no se borran**: quedan en el almacén seguro bajo `ucs_database_key_<keyId>`.

Llaves en el almacén seguro

Aspecto	Valor
Prefijo de clave	`ucs_database_key_`

Clave completa	`ucs_database_key_v2`, `ucs_database_key_v3_<epoch_ms>`
Formato del valor	La llave AES de 256 bits en Base64
Caché en memoria	`SecureStorageKeyProvider._cache`, por `keyId`
Backend	Keychain en Apple, Keystore/EncryptedSharedPreferences en Android, según `flutter_secure_storage 10.3.1`

En Android, `allowBackup="false"` (aplicado por `tool/bootstrap.py`) impide que el sistema restaure una copia de seguridad sin el material criptográfico compatible. En la web, `flutter_secure_storage` **no ofrece un almacén protegido por el sistema**: su implementación web depende del origen del navegador, tal como advierte `docs/SECURITY.md`. **Requiere validación** el comportamiento concreto en la PWA publicada.

Preferencias

`SharedPreferencesAsync` guarda 22 claves en total: 17 de configuración, 4 de estado de migración del historial y 1 heredada de metadatos de cifrado. El catálogo completo está en 05-technical-reference.md y la semántica de cada una en 10-configuration.md.

Nada de lo guardado en preferencias está cifrado. Contiene decisiones del usuario, no datos escaneados —con una excepción histórica: `scan_history_v1`, el historial en claro de una versión anterior, que la migración convierte y elimina. Su respaldo (`scan_history_v1_backup`) **sí va cifrado**.

Qué se pierde y cuándo

Acción	Historial	Inventarios	Preferencias	Llaves
Cerrar la aplicación	Se conserva	Se conserva	Se conservan	Se conservan
Cerrar en **modo temporal**	**Se pierde**	**Se pierde**	Se conservan	La de memoria se pierde
«Borrar todo el historial»	**Se pierde**	Se conserva	Se conservan	Se conservan
Rotar la llave	Se conserva, recifrado	Se conserva, recifrado	Se conservan	Se añade una nueva
Retención de 30/90/365 días	Se poda lo antiguo	Se conserva	Se conservan	Se conservan
Superar 5000 registros	Se recorta lo más antiguo	Se conserva	Se conservan	Se conservan
Borrar los datos de la aplicación	**Se pierde**	**Se pierde**	**Se pierden**	Dependen del sistema
Desinstalar	**Se pierde para siempre**	**Se pierde para siempre**	**Se pierden**	**Se pierden**

La última fila es la que conviene decir en voz alta: **no hay copia remota porque no hay servidor**. Exportar antes de desinstalar es la única forma de conservar el historial.

Relación entre almacenes y módulos

Almacén	Lo lee	Lo escribe
`scan_history`	`HistoryRepository.load`, `.ids`, `.pruneOlderThan`, `._trim`, `DataMaintenanceService`	`HistoryRepository` (5 métodos), `RecoveryService.retry`, `RecoveryService.discard`, `DataMaintenanceService`
`inventory_sessions`	`InventoryRepository.load`, `DataMaintenanceService`	`InventoryRepository.save`, `.remove`, `RecoveryService.retry`, `.discard`, `DataMaintenanceService`
`recovery_issues`	`RecoveryRepository.load`, `.mark`, `.clearResolved`	`RecoveryRepository.record`, `.mark`, `.clearResolved`
`_schema_meta`	`SchemaMigrator.migrate`	`SchemaMigrator.migrate`
`_security_meta`	`EncryptionMetadataRepository.loadActiveKeyId`	`.saveActiveKeyId`, `DataMaintenanceService`

Documentación previa relacionada

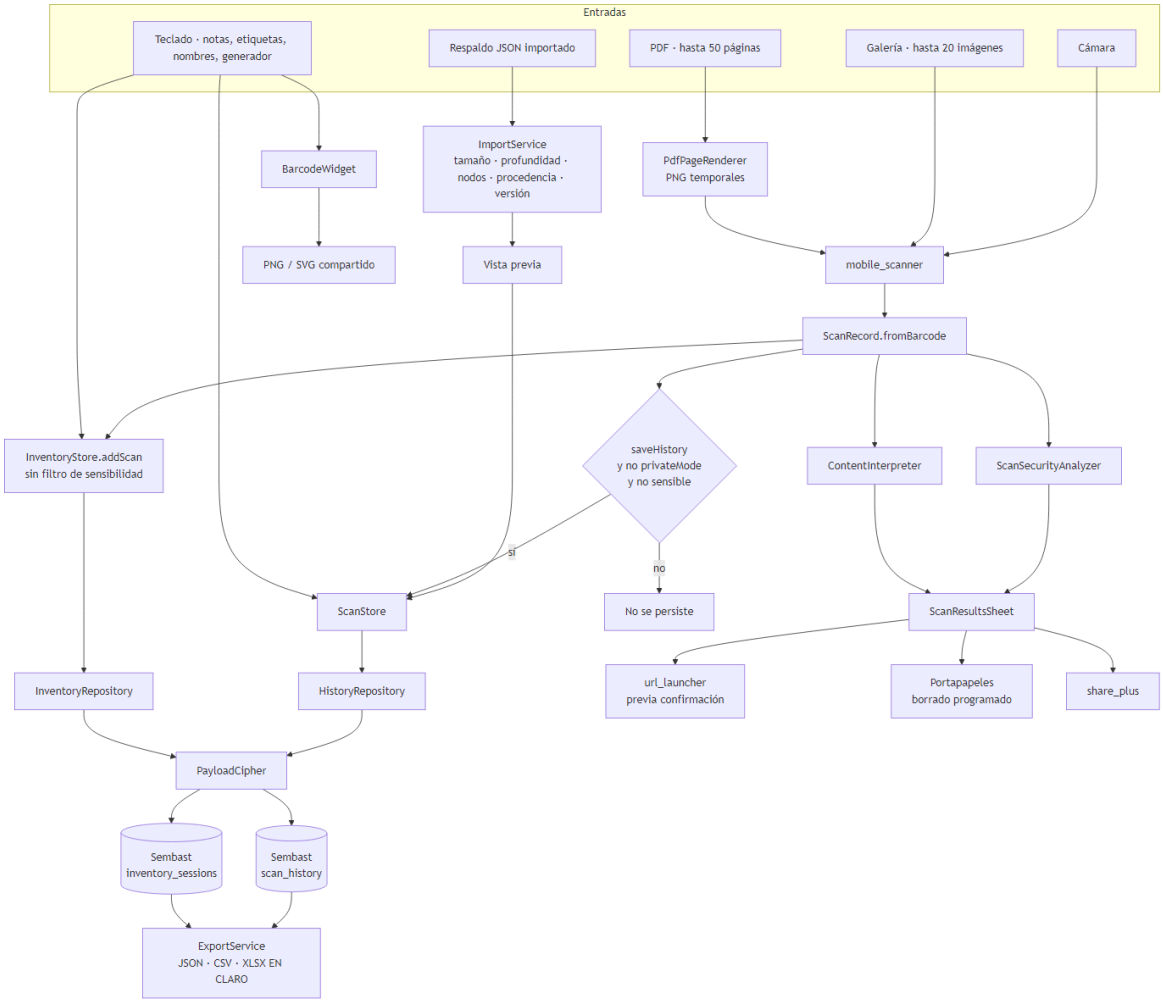
- `../quality/MIGRATIONS.md`: esquema, sobres y rotación desde la perspectiva de la política.
- `../quality/RECOVERY.md`: el Centro de recuperación.
- `../quality/COMPATIBILITY_CONTRACT.md`: los ocho compromisos sobre los datos del usuario que no pueden romperse.

Continúa en 08 · Flujo de datos.

08 · Flujo de datos

Este documento sigue **cada dato** desde su origen hasta su destino: quién lo crea, qué validación atraviesa, cómo se transforma, dónde se guarda y por dónde puede salir del dispositivo.

Panorama



Lo que muestra: las cinco entradas, el punto único de convergencia (`ScanRecord.fromBarcode`), la bifurcación entre persistencia y presentación, y las cinco salidas del dispositivo.

Lo que no muestra: que ninguna de las cinco salidas ocurre sin una acción explícita del usuario, y que la única que atraviesa un diálogo obligatorio es `url_launcher`. Tampoco muestra el flujo de recuperación ni el de rotación de llave, que operan sobre datos ya persistidos.

Entrada 1 · Cámara

Etapa	Qué ocurre	Validación
Captura	`MobileScanner.onDetect` entrega un `BarcodeCapture`	El motor ya validó la simbología
Filtro	`_handleCapture` descarta si está en pausa, ya procesando o desmontado	—

Extracción	`ScanRecord.payloadForBarcode` obtiene texto o `binary-base64:`	Se descartan los códigos con carga vacía
Deduplicación intra-captura	`unique.putIfAbsent(raw, …)`	Por texto exacto
Deduplicación entre capturas	Firma ordenada + ventana de 2 s	—
Construcción	`ScanRecord.fromBarcode`	Analizador e intérprete
Persistencia	Condicionada por tres banderas	Ver más abajo
Presentación	`ScanResultsSheet` con **todos** los registros	—

El `source` es siempre `'Cámara'`.

Entrada 2 · Galería

Etapa	Qué ocurre
Selección	`ImagePicker.pickMultiImage(limit: 20)`
Preparación	Se detiene la cámara y se abre el diálogo de progreso
Por imagen	`token.throwIfCancelled()` → `_engine.analyzeImage(path)` → deduplicar por texto → `ScanRecord.fromBarcode(source: 'Imagen')`
Cesión	`await Future<void>.delayed(Duration.zero)` entre imágenes
Final	Reanudar la cámara; persistir y mostrar, o avisar de que no se encontró nada

Cada imagen tiene su propio `DateTime.now()`, así que registros de la misma tanda pueden diferir en milisegundos.

Qué no pasa: las imágenes originales **no se copian ni se guardan**. Se leen por ruta desde donde el selector las entregó.

Entrada 3 · PDF

Etapa	Qué ocurre	Límite
Selección	`FilePicker.pickFiles(type: custom, allowedExtensions: ['pdf'], withData: false)`	Un solo archivo
Apertura	`PdfDocument.openFile(path)`	—
Directorio	`getTemporaryDirectory()` + `ucs_pdf_<microsegundos>`	—
Por página	`page.render(...)` a PNG con escala `_renderScale`	50 páginas, 2400 px en el lado mayor

Escritura	`File(outputPath).writeAsBytes(..., flush: true)`	`page_N.png`
Análisis	`_engine.analyzeImage(page.imagePath)`	—
Limpieza	`PdfPageRenderer.cleanup(pages)` borra PNG y directorio	En éxito, error y cancelación

El `source` es `'PDF · página N'`, con el número real de página.

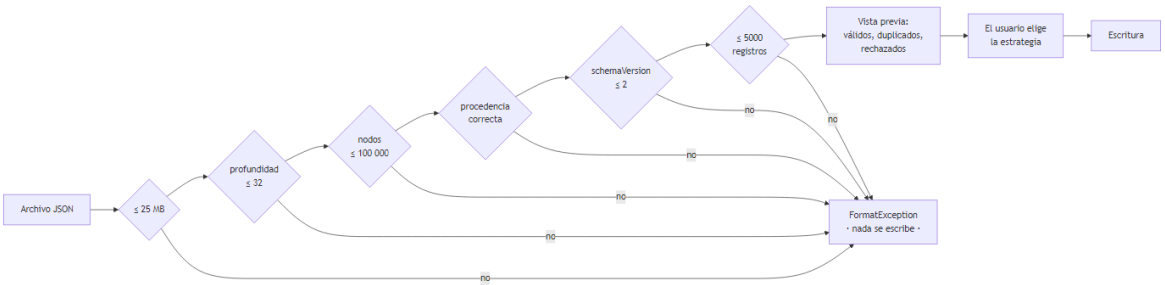
`withData: false` es importante: el selector devuelve la ruta, no los bytes. Un PDF de 200 MB no se carga en memoria; se abre en disco.

Los PNG temporales son datos en claro en disco, durante el tiempo que dura el análisis. Es inevitable —el motor de lectura recibe una ruta— y la limpieza cubre los tres caminos de salida. Está registrado como riesgo aceptado en 11-security.md.

No disponible en web: `_scanFromPdf` sale de inmediato si `kIsWeb`, y el menú de importación está deshabilitado (`enabled: !kIsWeb`).

Entrada 4 · Respaldo JSON

Es la entrada con más validación, porque es la única que viene de un archivo que el usuario no creó necesariamente.



Lo que muestra: que hay seis puertas antes de que un byte llegue a la base, y que la escritura solo ocurre después de una elección explícita.

Lo que no muestra: dos cosas. La primera, que un registro individual mal formado **no aborta la importación**: se cuenta en `rejected` y se sigue; solo los fallos de forma del archivo completo lanzan. La segunda, que si la raíz del JSON es una **lista** en vez de un objeto se acepta como formato heredado (`schemaVersion: 1`) **sin comprobar procedencia ni versión**, saltándose las puertas cuarta y quinta.

`_validateJsonShape` usa una pila explícita en lugar de recursión, precisamente para que un JSON profundo no provoque un desbordamiento de pila antes de llegar a la comprobación de profundidad. El límite de profundidad se evalúa por rama; el de nodos, sobre el total.

El inventario sigue el mismo camino con dos diferencias: acepta un objeto heredado sin campo `application`, y comprueba `maxInventoryItems = 10000`. Se importa **siempre como copia**: `InventoryStore.importSession` genera un `id` nuevo y añade `(importado)` al nombre.

Entrada 5 · Teclado

Dato	Dónde se escribe	Validación	Destino

Notas de un registro	`HistoryScreen._edit`	`trim()`	`ScanRecord.notes`, cifrado
Etiquetas	idem	Separadas por comas, `trim()`, vacías descartadas, `toSet()`	`ScanRecord.tags`, cifrado
Nombre de sesión	`InventoryScreen`	Si queda vacío tras `trim()`, se usa `Inventario DD-MM-AAAA`	`InventorySession.name`, cifrado
Nota de producto	`InventoryScreen._editItemNotes`	`trim()`	`InventoryItem.notes`, cifrado
Cantidad	Botones ± y `setQuantity`	`≤ 0` elimina el ítem	`InventoryItem.quantity`, cifrado
Campos del generador	`GeneratorScreen`	Ninguna previa; `BarcodeWidget.errorBuilder` avisa si el formato no admite el contenido	**No se persiste**

Lo que el usuario escribe en el generador no se guarda en ningún sitio. Vive en cuatro `TextEditingController` que se liberan al salir de la pestaña.

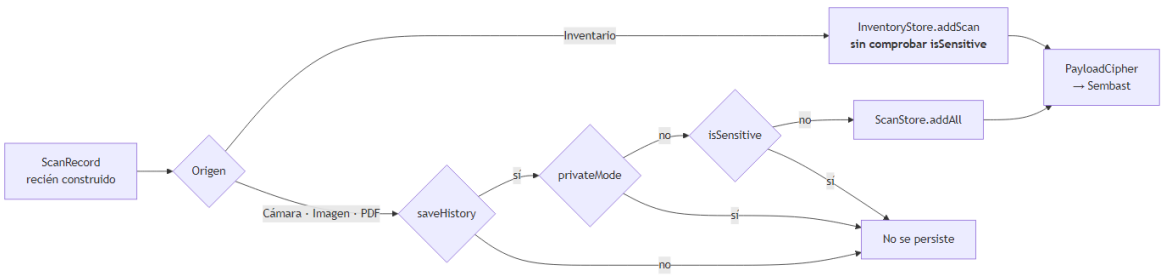
Transformación central: `ScanRecord.fromBarcode`

Es el único punto donde convergen todas las lecturas.

Paso	Entrada	Salida
1	`Barcode`	`raw` = texto, o `binary-base64:<b64>`, o cadena vacía
2	`raw`	`SecurityAssessment(level, reasons, canOpen, normalizedHost)`
3	`raw`	`ParsedContent(kind, title, fields, summary, sensitive)`
4	`raw` + marca de tiempo	`id` = SHA-256 truncado a 20
5	Todo lo anterior	`ScanRecord` inmutable

`normalizedHost` del `SecurityAssessment` **se descarta**: `ScanRecord` no lo guarda. Solo se usa el `level`, los `reasons` y el `canOpen`. R-25.

Decisión de persistencia



Lo que muestra: la asimetría real entre el escáner y el inventario. El escáner aplica tres condiciones; el inventario, ninguna.

Lo que no muestra: que en ambos casos el dato acaba **cifrado con la misma llave**. La diferencia no es de protección criptográfica, sino de que un secreto que el usuario esperaba que no se guardara, en el inventario sí se guarda. Registrado como R-03.

`isSensitive` es `true` para: OTP, criptomonedas, EMVCo, EPC/SEPA, Swiss QR, AAMVA y Wi-Fi **solo si trae contraseña**.

Salidas del dispositivo

Salida	Mecanismo	Confirmación previa	Qué sale
Abrir un enlace o acción	`url_launcher`	**Obligatoria** si `confirmBeforeOpen` o riesgo ≠ `low`	La URI
Compartir el contenido leído	`share_plus`	Obligatoria **solo si el registro es sensible**	El `rawValue`, o un `.vcf`/`.ics`
Copiar al portapapeles	`Clipboard.setData`	Ninguna	El `rawValue`, con borrado programado
Exportar historial o inventario	`share_plus`	Ninguna	JSON, CSV o XLSX **en claro**
Compartir un código generado	`share_plus`	Ninguna	PNG o SVG
Copiar el diagnóstico	`Clipboard.setData`	Ninguna	Solo tipos de error y huellas
Copiar el paquete de recuperación	`Clipboard.setData`	Ninguna	Sobres **todavía cifrados**, sin llaves

Apertura de un enlace

```
final bool shouldConfirm = widget.settings.value.confirmBeforeOpen || record.riskLevel != RiskLevel.low;
```

Es una **disyunción**: aunque el usuario desactive «Confirmar antes de abrir», un riesgo `caution` o `high` sigue exigiendo el diálogo. El texto del diálogo cambia según el nivel.

`_actionUri` decide qué se puede abrir:

```
if (record.canOpen) return ScanSecurityAnalyzer.normalizedActionUri(record.rawValue);
return switch (record.parsed.kind) {
 ContentKind.phone || ContentKind.email || ContentKind.sms || ContentKind.geo => Uri.tryParse(record.rawVa
 _ => null,
};
```

La rama de reserva cubre el caso en que el analizador dijera `canOpen: false` pero el intérprete identificó un tipo con acción natural. En la práctica esos cuatro tipos ya obtienen `canOpen: true` del analizador —son esquemas de sistema seguros— así que la rama es redundante en 1.1.0.

Portapapeles

```
await Clipboard.setData(ClipboardData(text: value));
if (clearAfterSeconds <= 0) return;
Timer(Duration(seconds: clearAfterSeconds), () async {
 final ClipboardData? current = await Clipboard.getData(Clipboard.kTextPlain);
```

```
  if (current?.text == value) await Clipboard.setData(const ClipboardData(text: ''));
});
```

Tres propiedades:

1. **`0` significa nunca borrar**, y es una opción del desplegable.
2. **El borrado comprueba antes** que el portapapeles siga conteniendo el mismo valor: si el usuario copió otra cosa entre medias, no se le borra.
3. **El temporizador no se guarda ni se cancela.** Sobrevive al `dispose` del widget que lo creó y se dispara aunque el usuario haya salido de la pantalla. Es intencional —el objetivo es limpiar el portapapeles pase lo que pase— pero significa que no hay forma de cancelarlo. R-26.

Compartir

`ScanRecordCard._shareRecord` tiene tres caminos:

Condición	Qué se comparte
`record.isSensitive`	Diálogo de advertencia primero; si se cancela, no se comparte
`kind == contact`	Archivo `contacto.vcf` con MIME `text/vcard`
`kind == event`	Archivo `evento.ics` con MIME `text/calendar`
Resto	Texto plano (`ShareParams(text: record.rawValue)`)

El contacto y el evento se comparten como **archivo con el `rawValue` íntegro**, de modo que la aplicación de destino lo reconozca y ofrezca guardarlo.

Exportación

Todo lo exportado sale **en claro**. La entrada del menú no advierte de ello; la advertencia vive en `docs/SECURITY.md` («La exportación JSON, CSV y XLSX contiene datos descifrados por decisión explícita del usuario»). R-27.

El JSON lleva sobre de procedencia (`application`, `schemaVersion: 2`, `exportedAt`, `type`), que es lo que después valida `ImportService`. CSV y XLSX no llevan sobre: **no se pueden reimportar**.

Datos que nunca salen del dispositivo

Dato	Por qué
Las llaves AES	Solo `flutter_secure_storage`; el paquete de recuperación lo v erifica con prueba
El texto de los mensajes de error	`AppDiagnostics` guarda `runtimeType`, no el mensaje
Las contraseñas Wi-Fi y los secretos OTP leídos desde el escán er	No se persisten; se muestran enmascarados si el ajuste está act ivo
Lo que se escribe en el generador	No se persiste en ningún sitio
El historial y los inventarios	No existe canal de red saliente propio

Comprobación de la última fila:

```
grep -rn "HttpClient\|package:http\|WebSocket\|dart:html" lib --include=*.dart
```

no devuelve nada, y `pubspec.yaml` no declara ningún cliente HTTP. El único paquete que entrega datos fuera es `url_launcher`, que delega en el sistema operativo una URI que el usuario ya confirmó, y `share_plus`, que abre el selector del sistema.

Ciclo de vida de un dato concreto: una URL escaneada

Para hacerlo tangible, el recorrido completo de `https://example.com/pago?id=42`:

#	Dónde	Qué le pasa
1	`MobileScanner`	El motor devuelve un `Barcode` con `rawValue`
2	`payloadForBarcode`	Devuelve el texto tal cual
3	`ScanSecurityAnalyzer.analyze`	HTTPS, host normal, sin señales → `low`, `canOpen: true`
4	`ContentInterpreter.parse`	`ContentKind.url`, `summary: example.com`, campos `Dominio`, `Protocolo`, `Ruta`, `Consulta`, `Dirección`
5	`ScanRecord.fromBarcode`	`id` = SHA-256 de `URL|microsegundos`, 20 caracteres
6	`_persistAndShow`	No es sensible → entra en `ScanStore.addAll`
7	`HistoryRepository.upsert`	`encryptJson(record.toJson())`
8	Sembast	`{id, scannedAt: "2026-08-27T…", payload: "{version:2,…}"}`
9	`ScanResultsSheet`	Cinco campos con nombre, banda verde «Sin señales evidentes»
10	El usuario pulsa «Abrir»	Diálogo si `confirmBeforeOpen`; después `launchUrl`
11	Más tarde, en Historial	`load` descifra, `inspect` confirma que el sobre está al día, se muestra
12	A los 90 días con retención activa	`pruneOlderThan` lo borra por `scannedAt`

En ningún punto de esas doce etapas la URL sale del dispositivo, salvo en la etapa 10 y solo con confirmación.

Continúa en 09 · APIs e integraciones.

09 · APIs e integraciones

No hay red. Y aquí está la demostración

Este repositorio **no define ninguna API HTTP, no consume ninguna, y no abre ningún socket**. No es una afirmación de intención: es verificable con tres comandos.

```
# 1 · ¿Hay algún cliente de red en el código?
grep -rn "HttpClient\|package:http\|WebSocket\|Socket(\|dart:html\|XMLHttpRequest" lib --include="*.dart"
# → sin resultados
```

```
# 2 · ¿Hay algún paquete de red entre las dependencias?
grep -nE "^  (http|dio|grpc|graphql|firebase|supabase|sentry|amplitude|posthog)" pubspec.yaml
# → sin resultados
```

(La búsqueda va anclada al inicio de la línea de dependencia a propósito: un `grep` suelto de `dio` encuentra `au**dio**players`, que no es un cliente de red sino el reproductor del tono de lectura.)

```
# 3 · ¿Qué es lo único que envía algo fuera de la aplicación?
grep -rn "launchUrl\|SharePlus" lib --include="*.dart"
# → 1 uso de launchUrl, en scan_result_sheet.dart:239
# → 9 usos de SharePlus.instance.share: export_service.dart (4),
#   generator_screen.dart (3) y scan_result_sheet.dart (2)
```

Los tres se ejecutaron sobre el commit `c1f9878` y dieron ese resultado.

Las dos únicas rutas por las que un dato abandona el proceso son delegaciones al sistema operativo:

Paquete	Qué hace	Confirmación
`url_launcher 6.3.2`	Entrega una URI al sistema, que decide q ué aplicación la abre	Diálogo obligatorio si `confirmBeforeOpe n` o riesgo ≠ `low`
`share_plus 12.0.2`	Abre el selector de compartir del sistema con texto o archivos	Diálogo solo si el registro es sensible

Ninguno de los dos es un cliente de red: no hablan con ningún servidor, hablan con el sistema operativo.

Qué se documenta entonces en este capítulo

Como no hay red, lo que sí existe y merece contrato son:

1. **Los contratos de datos JSON** que la aplicación produce y consume.
2. **Las interfaces de plataforma** que usa a través de plugins.
3. **Las fronteras internas** que otro código podría implementar.

---

1 · Contratos de datos JSON

Hay cinco formatos JSON con forma estable. Tres son de intercambio con el usuario, uno es interno persistido y uno es de diagnóstico.

1.1 Respaldo de historial · `type: history`

Produce: `ExportService.shareHistoryJson`. **Consume**: `ImportService.parseHistoryBytes`. **Nombre de archivo**: `historial-escaneos.json`.

```
{
 "application": "Universal Code Scanner",
 "schemaVersion": 2,
 "exportedAt": "2026-08-27T12:00:00.000Z",
 "type": "history",
 "records": [
   {
     "id": "a1b2c3d4e5f60718293a",
     "rawValue": "https://example.com/producto?id=42",
     "displayValue": "https://example.com/producto?id=42",
     "format": "QR Code",
     "contentType": "Enlace web",
     "scannedAt": "2026-08-27T09:15:32.100",
     "source": "Cámara",
     "riskLevel": "low",
     "riskReasons": [],
     "canOpen": true,
     "parsed": {
       "kind": "url",
       "title": "Enlace web",
       "summary": "example.com",
       "fields": {
         "Dominio": "example.com",
         "Protocolo": "HTTPS",
         "Ruta": "/producto",
         "Consulta": "id=42",
         "Dirección": "https://example.com/producto?id=42"
       },
       "sensitive": false
     },
     "favorite": false,
     "tags": [],
     "notes": ""
   }
 ]
}
```

Reglas de aceptación en la importación:

Regla	Comprobación	Fallo
Tamaño	`bytes.length ≤ 25 MB`	`FormatException('El archivo supera el límite de 25 MB.')`
Profundidad	`≤ 32` por rama	`FormatException('El archivo contiene una estructura demasiado profunda.')`
Nodos	`≤ 100 000` en total	`FormatException('El archivo contiene demasiados elementos.')`
Procedencia	`application == 'Universal Code Scanner'` **y** `type == 'history'`	`FormatException('El respaldo no pertenece al historial de Universal Code Scanner.')`
Versión	`schemaVersion ≤ 2`	`FormatException('El respaldo fue creado por una versión más reciente.')`
Estructura	`records` debe ser una lista	`FormatException('El respaldo no contiene registros.')`

Cantidad	`≤ 5000`	`FormatException('El respaldo supera el límite de registros.')`

Formato heredado aceptado: si la raíz es una **lista** en vez de un objeto, se acepta como `schemaVersion: 1` sin comprobar procedencia. Es la compatibilidad con la versión que exportaba el array desnudo.

Tolerancia por registro: un elemento que no sea un mapa, o que no pueda reconstruirse como `ScanRecord`, se cuenta en `rejected` y **no aborta la importación**.

Compromiso de compatibilidad: el punto 6 de `../quality/COMPATIBILITY_CONTRACT.md` dice que un respaldo con `schemaVersion: 2` seguirá siendo importable en versiones futuras.

1.2 Respaldo de inventario · `type: inventory`

Produce: `ExportService.shareInventoryJson`. **Consume**: `ImportService.parseInventoryBytes`. **Nombre de archivo**: `inventario-<id>.json`.

```
{
 "application": "Universal Code Scanner",
 "schemaVersion": 2,
 "exportedAt": "2026-08-27T12:00:00.000Z",
 "type": "inventory",
 "session": {
   "id": "0f1e2d3c4b5a6978",
   "name": "Bodega central agosto",
   "createdAt": "2026-08-20T08:00:00.000",
   "closedAt": null,
   "items": {
     "7801234567890": {
       "code": "7801234567890",
       "format": "EAN-13",
       "label": "Producto",
       "quantity": 12,
       "firstScannedAt": "2026-08-20T08:05:11.000",
       "lastScannedAt": "2026-08-20T09:41:02.000",
       "notes": "Estante 3"
     }
   }
 }
}
```

Reglas propias: `items.length ≤ 10 000`; se acepta también un objeto **heredado** sin campo `application` (se toma como `schemaVersion: 1` y se valida como sesión directamente).

La importación siempre crea una copia: `InventoryStore.importSession` genera un `id` nuevo, añade `(importado)` al nombre y usa `DateTime.now()` como `createdAt`. Nunca sobrescribe una sesión existente.

1.3 Paquete de recuperación

Produce: `RecoveryService.exportBundle`. **Consume**: nadie. **No existe importador.** Es un artefacto de diagnóstico para entregar a soporte. R-28.

```
{
 "application": "Universal Code Scanner",
 "type": "recovery",
 "schemaVersion": 1,
 "exportedAt": "2026-08-27T12:00:00.000Z",
 "notice": "Encrypted payloads remain encrypted. No decryption key is included.",
 "issues": [
   {
     "id": "3f9a1c8b2d7e4f60a5b3c1d2",
     "entityType": "history",
     "entityId": "a1b2c3d4e5f60718293a",
     "detectedAt": "2026-08-27T11:58:03.412Z",
```

```
    "code": "decrypt_StateError",
    "state": "unresolved",
    "encryptedPayload": "{\"version\":2,\"algorithm\":\"AES-256-GCM\",…}"
  }
 ]
}
```

Los `encryptedPayload` salen **todavía cifrados** y ninguna llave se incluye. Hay una prueba que lo comprueba cifrando un objeto con un marcador y verificando que ni ese marcador ni la cadena `ucs_database_key` aparecen en el paquete (`test/core/recovery_service_test.dart`).

1.4 Diagnóstico privado

Produce: `AppDiagnostics.exportJson`. **Consume**: nadie. Es texto para pegar en un reporte.

```
{
 "application": "Universal Code Scanner",
 "schemaVersion": 1,
 "generatedAt": "2026-08-27T12:00:00.000Z",
 "privacy": "No scan payloads, URLs, passwords, one-time codes, notes or inventory values are included.",
 "entries": [
  {
   "at": "2026-08-27T11:59:12.004Z",
   "area": "bootstrap",
   "errorType": "StateError",
   "stackFingerprint": "1a2b3c4d"
  }
 ]
}
```

Cuatro campos por entrada, y ninguno puede contener datos del usuario por construcción: `area` está saneada a `[a-zA-Z0-9_.-]` y recortada a 80 caracteres, `errorType` es el `runtimeType`, y `stackFingerprint` es un hash de 8 dígitos hexadecimales.

1.5 Sobre de cifrado (interno)

Es el formato de la columna `payload` de Sembast. Documentado en 07-database.md. No sale de la aplicación salvo dentro de un paquete de recuperación, y ahí sigue cifrado.

1.6 Contratos de la herramienta

Tres JSON más, producidos por `tool/` y no por la aplicación:

Archivo	Lo produce	Lo valida	Contenido
`SOURCE_MANIFEST.json`	`tool/generate_source_manifest.py`	`validate_structure.py::check_json_files`	`{schemaVersion: 1, files: [{path, size, sha256}]}` · 150 archivos
`docs/security/sbom-source.cdx.json`	`tool/generate_sbom.py`	`validate_structure.py::check_source_sbom` (exige `bomFormat == 'CycloneDX'`)	CycloneDX 1.5
`test_assets/manifest.json`	A mano	`validate_structure.py::check_test_assets`	`{schemaVersion: 1, items: [{file, expectedPayload, format, scenario}]}`
`RELEASE_SOURCE.json`	A mano	`check_json_files`	Qué se validó al publicar

`build/reports/sbom.cdx.json` y `build/reports/dependency-licenses.json` se generan en cada ejecución de la CI y **no se versionan**.

2 · Formatos de salida no JSON

CSV

Aspecto	Valor	Motivo
Codificación	UTF-8 **con BOM** (`EF BB BF`)	Sin el BOM, Excel en Windows abre el archivo en la página de códigos del sistema y rompe los acentos
Separador de línea	`\r\n`	Convención de Excel
Entrecomillado	**Todas** las celdas, siempre	Evita razonar sobre qué celda lo necesita
Escape	`"` se duplica a `""`	RFC 4180

Columnas del historial (8): `fecha, tipo, formato, origen, riesgo, favorito, etiquetas, contenido`. Las etiquetas se unen con `|`.

Columnas del inventario (8): `sesion, codigo, formato, descripcion, cantidad, primera_lectura, ultima_lectura, notas`.

XLSX

Generado con `excel 4.0.6` en un isolate aparte. Una sola hoja, con nombre `Historial` o `Inventario`; la hoja por defecto que crea la librería se elimina si su nombre difiere. Todas las celdas son `TextCellValue`: **no hay tipos numéricos ni de fecha**, así que una hoja de cálculo mostrará las cantidades como texto.

Columnas del historial (9): añade `Notas` respecto al CSV. Columnas del inventario (8): las mismas que su CSV.

vCard e iCalendar

`ScanRecordCard._shareRecord` comparte el `rawValue` íntegro como archivo cuando el tipo es contacto o evento:

Tipo	Extensión	MIME
`ContentKind.contact`	`.vcf`	`text/vcard`
`ContentKind.event`	`.ics`	`text/calendar`

No se reformatea nada: se entrega exactamente lo que decía el código.

PNG y SVG del generador

Formato	Cómo se produce	Tamaño
PNG	Captura del `RepaintBoundary` a `pixelRatio: 3`	El del widget × 3
SVG	`Barcode.toSvg(payload, …)`	900×900 para 2D, 1200×420 para lineales

Ambos se comparten como `XFile.fromData`: **no se escriben a disco**.

3 · Interfaces de plataforma

Estas sí son integraciones reales, y son las únicas que existen.

Interfaz	Plugin	Qué pide	Permiso	Degradación
Cámara	`mobile_scanner 7.4.0`	Cuadros de vídeo y detección	`CAMERA` en Android; `NSCameraUsageDescription` en Apple	Fase `unavailable` con motivo y botón de reintento
Galería	`image_picker 1.2.3`	Hasta 20 imágenes	Lo gestiona el selector del sistema	Lista vacía → no ocurre nada
Archivos	`file_picker 11.0.3`	Un `.json` o un `.pdf`	Selector del sistema	`null` → no ocurre nada
Almacén seguro	`flutter_secure_storage 10.3.1`	Guardar y leer llaves AES	Ninguno explícito	`read` devuelve `null` → `StateError('encryption_key_missing:…')`
Biometría	`local_auth 3.0.2`	Autenticación local	`USE_BIOMETRIC`; `NSFaceIDUsageDescription`	Ambos métodos capturan y devuelven `false`
Audio	`audioplayers 6.8.1`	Reproducir el tono empaquetado	Ninguno	Degrada a `SystemSound.play` y deja de intentarlo
Vibración	`flutter/services` (`HapticFeedback`)	Vibración media	Ninguno	`catch` vacío: un dispositivo sin vibrador no rompe la lectura
Portapapeles	`flutter/services` (`Clipboard`)	Leer y escribir texto	Ninguno	Ninguna
Compartir	`share_plus 12.0.2`	Selector del sistema	Ninguno	Ninguna
Abrir URI	`url_launcher 6.3.2`	Delegar al sistema	Ninguno	`opened == false` → mensaje «No existe una aplicación compatible»
Rutas	`path_provider 2.1.6`	Directorio de soporte y temporal	Ninguno	Fallo → arranque fallido → inicio seguro
PDF	`pdfrx 2.4.5`	Renderizar páginas	Ninguno	No disponible en web (`pdf_page_renderer_stub.dart`)

`BiometricService` merece mención por lo explícito de su degradación:

```
Future<bool> isAvailable() async {
 try { return await _authentication.isDeviceSupported(); } on Object { return false; }
}
Future<bool> authenticate() async {
 try { return await _authentication.authenticate(localizedReason: '…', persistAcrossBackgrounding: true);
 on Object { return false; }
}
```

Cualquier fallo se convierte en «no autenticado», nunca en «autenticado por defecto». Es la dirección correcta para un control de seguridad.

`persistAcrossBackgrounding: true` evita que el propio diálogo biométrico —que pasa la aplicación a inactivo— dispare un nuevo bloqueo en bucle.

---

4 · Fronteras internas implementables

Tres interfaces que otro código podría implementar sin tocar el resto.

`ScannerEngine`

```
abstract interface class ScannerEngine {
 MobileScannerController get controller;
 ValueListenable<MobileScannerState> get state;
 Future<void> start();
 Future<void> stop();
 Future<void> dispose();
 Future<void> toggleTorch();
 Future<void> switchCamera();
 Future<void> setZoomScale(double value);
 Future<BarcodeCapture?> analyzeImage(String path);
}
```

Problema de la abstracción: dos de sus nueve miembros —`controller` y `state`— devuelven tipos de `mobile_scanner`. Un motor alternativo tendría que producir un `MobileScannerController` y un `MobileScannerState`, lo cual la hace inservible como frontera real. Es una abstracción de conveniencia sobre el paquete actual, no un puerto independiente del proveedor. R-29.

Y `InventoryScreen` no la usa (R-04).

`ContentParser` y `ContentParserRegistry`

```
abstract interface class ContentParser {
 String get id;
 int get priority;
 bool canParse(String rawValue);
 ParsedContent parse(String rawValue);
}
```

Esta sí es una frontera limpia: no depende de ningún paquete externo. Registrar un parser propio es una llamada:

```
ContentParserRegistry.instance.register(MiParser());
```

Se ordena por prioridad descendente; el integrado tiene `-1000` y **no puede desregistrarse**. Hay una prueba que registra un parser de prueba, verifica que gana, lo desregistra y comprueba que el integrado vuelve a atender (`test/services/content_parser_registry_test.dart`).

No hay ningún punto de la aplicación desde el que un usuario o una configuración registre un parser. La extensibilidad es en tiempo de compilación, para quien bifurque el repositorio.

`EncryptionKeyProvider`

```
abstract interface class EncryptionKeyProvider {
 Future<SecretKey?> read(String keyId);
 Future<SecretKey> readOrCreate(String keyId, AesGcm algorithm);
 Future<void> forget(String keyId);
}
```

Dos implementaciones reales (`SecureStorageKeyProvider`, `MemoryEncryptionKeyProvider`) y una tercera en las pruebas (`_TrackingKeyProvider`, que envuelve la de memoria para observar creaciones y olvidos). Es la frontera mejor diseñada del repositorio: no filtra ningún tipo de plataforma.

`ScanTonePlayer`

```
abstract interface class ScanTonePlayer {
 Future<void> play();
 Future<void> dispose();
}
```

Dos miembros, cero acoplamiento. Existe para poder probar `ScanFeedback` sin un motor de audio, y las cuatro pruebas de `scan_feedback_test.dart` lo usan.

---

5 · Integraciones de infraestructura

No son de la aplicación, sino del repositorio.

Servicio	Para qué	Dónde se configura
GitHub Actions	Analizar, probar y compilar en cada `push` y PR	`.github/workflows/flutter-ci.yml`
GitHub Pages	Publicar la landing y la PWA	`.github/workflows/deploy-landing.yml`
GitHub Releases	Distribuir los APK con sus SHA-256	Manual, `gh release create`
Dependabot	Actualizaciones semanales de pub y mensuales de acciones	`.github/dependabot.yml`
`pub.dev`	Resolución de dependencias	`pubspec.yaml` / `pubspec.lock`

`deploy-landing.yml` publica en `https://vladimiracunadev-create.github.io/universal-code-scanner/` con la PWA en `/app/` y `APP_BASE_HREF = /universal-code-scanner/app/`. Detalle en 13-deployment-and-operations.md.

Continúa en 10 · Configuración.

10 · Configuración

Cómo se configura este sistema

No hay archivos `.env`, ni variables de entorno de aplicación, ni servidor de configuración remota. **Todo lo configurable vive en tres sitios**:

Nivel	Dónde	Quién lo cambia	Cuándo tiene efecto
Preferencias del usuario	`SharedPreferencesAsync`	La persona, desde Ajustes	Al instante
Constantes de compilación	Literales en `lib/`	Quien modifica el código	Al recompilar
Configuración del repositorio	`pubspec.yaml`, `.fvmrc`, `analysis_options.yaml`, `.github/`	Quien mantiene el proyecto	Al ejecutar la herramienta correspondiente

Las únicas variables de entorno del proyecto son de CI: `FLUTTER_VERSION` en ambos workflows y `APP_BASE_HREF` en el de publicación. Ninguna llega a la aplicación en ejecución.

---

1 · Preferencias del usuario

Diecisiete claves, todas escritas por `SettingsRepository.save` y leídas por `SettingsRepository.load`. La tabla incluye **la consecuencia real de cada valor**, no solo su nombre.

Apariencia

Clave	Tipo	Por defecto	Valores	Qué cambia
`theme_mode`	`String`	`system`	`system`, `light`, `dark`	`MaterialApp.themeMode`
`language`	`String`	`system`	`system`, `esCl`, `es` (en existe pero no se ofrece)	`MaterialApp.locale`
`high_contrast`	`bool`	`false`	—	Sube `contrastLevel` de `ColorScheme.fromSeed` a `1.0` y dibuja un borde de 1,5 px en las tarjetas
`large_controls`	`bool`	`false`	—	`VisualDensity(horizontal: 1, vertical: 1)`. **No cambia el tamaño del texto**: eso lo controla el sistema
`reduce_motion`	`bool`	`false`	—	Tres efectos: sin transiciones de página, barra de estado quieta, marco sin línea de barrido

Sobre `language`: el desplegable ofrece solo tres opciones. Si una versión futura hubiera guardado `en`, el desplegable lo traduce a `system` para no quedarse sin coincidencia (`settings_screen.dart`, líneas 66-69). Y aunque se guardara `en`, `_AppLocalizationsDelegate.isSupported` lo rechaza y Flutter recae en el primer idioma admitido: **nunca se produce una interfaz a medio traducir**. Cuatro pruebas protegen esa decisión.

Escáner

Clave	Tipo	Por defecto	Qué cambia
`use_scan_window`	`bool`	`true`	Si `true`, `scanWindow` es el cuadrado central del 68 %; si `false`, se pasa `null` y se analiza **toda** la imagen
`auto_torch`	`bool`	`false`	Enciende la linterna al crear el motor. **Solo en `ScannerScreen`**: el inventario no lo aplica
`sound_enabled`	`bool`	`true`	Reproduce el tono empaquetado tras cada lectura
`vibration_enabled`	`bool`	`true`	`HapticFeedback.mediumImpact()` tras cada lectura

`use_scan_window` es el ajuste que el README ofrece como solución cuando un código no entra en el marco: desactivarlo analiza la imagen completa a costa de más trabajo por cuadro.

`sound_enabled` y `vibration_enabled` son **independientes**: hay una prueba que comprueba que activar uno no activa el otro (`test/services/scan_feedback_test.dart`, «each channel honours its own setting»).

Privacidad y seguridad

Clave	Tipo	Por defecto	Qué cambia
`save_history`	`bool`	`true`	Si `false`, **ninguna lectura del escáner se persiste**. El inventario sigue guardando
`private_mode`	`bool`	`false`	Si `true`, mismo efecto que apagar `save_history`. Existe como interruptor temporal frente al permanente
`hide_sensitive_values`	`bool`	`true`	Enmascara con `••••••••` los campos `Contraseña`, `Secreto`, `Dirección`, `IBAN` y `Carga` de un registro sensible
`confirm_before_open`	`bool`	`true`	Exige diálogo antes de abrir. **Desactivarlo no elimina el diálogo para riesgo `caution` o `high`**
`biometric_lock`	`bool`	`false`	Bloquea la aplicación al pasar a segundo plano
`clear_clipboard_seconds`	`int`	`30`	`0`, `15`, `30`, `60` o `300`. `0` = nunca borrar

`history_retention_days`	`int`	`0`	`0`, `30`, `90` o `365`. `0` = sin límite

Tres detalles que conviene conocer:

`save_history` y `private_mode` hacen lo mismo. La condición es `if (_settings.saveHistory && !_settings.privateMode)`. La diferencia es de intención: uno es una política permanente, el otro una sesión concreta. Ninguno de los dos afecta al inventario.

`biometric_lock` no se puede activar a ciegas. `_changeBiometricLock` comprueba primero `isDeviceSupported()` y después **exige una autenticación correcta** antes de guardar el ajuste. Si el dispositivo no lo admite o el usuario cancela, muestra «No se pudo activar el bloqueo en este dispositivo» y el ajuste no cambia. Es la dirección correcta: no se puede quedar bloqueado fuera de su propia aplicación.

`history_retention_days` se aplica en dos momentos: al arrancar (`AppBootstrapper.initialize` llama a `pruneOlderThan` con el valor guardado) y al cambiarlo (`_updateRetention` guarda y poda de inmediato). Cambiarlo a 30 días **borra ahí mismo** todo lo anterior a esa fecha, sin diálogo de confirmación. R-30.

Banderas de capacidad

Clave	Tipo	Por defecto	Qué cambia
`feature_flags`	`String` (JSON)	Nueve `false`	**Nada.** Ver abajo

`FeatureFlags` se serializa, se guarda, se lee y se restaura correctamente. Y **ningún código lo consulta**: las nueve banderas no aparecen fuera de `feature_flags.dart`. Tampoco hay control en Ajustes para cambiarlas, así que en la práctica la clave siempre contiene los nueve `false` por defecto.

Bandera	Capacidad reservada	Documentada en
`experimentalParsers`	Parsers experimentales	`../quality/FEATURE_FLAGS.md`
`secondaryScannerEngine`	Segundo motor de escaneo	idem
`ocr`	Reconocimiento óptico de texto	idem
`nfc`	Lectura NFC	idem
`urlReputation`	Reputación remota de URLs	idem
`productLookup`	Consulta de productos	idem
`encryptedSync`	Sincronización cifrada	idem
`enterpriseApi`	API empresarial	idem
`kioskMode`	Modo kiosco	idem

Si un `feature_flags` malformado llegara a preferencias, `SettingsRepository.load` lo captura y vuelve al valor por defecto en vez de fallar el arranque.

Restablecimiento parcial

`SettingsRepository.resetNonSensitive()` —accesible desde la pantalla de inicio seguro— borra **14 de las 17 claves**:

```
'theme_mode', 'language', 'sound_enabled', 'vibration_enabled', 'auto_torch',
'use_scan_window', 'confirm_before_open', 'hide_sensitive_values', 'high_contrast',
'large_controls', 'reduce_motion', 'clear_clipboard_seconds', 'history_retention_days',
'feature_flags',
```

Deja intactas `save_history`, `private_mode` y `biometric_lock`. Es deliberado: un botón de recuperación de emergencia no puede desactivar el bloqueo de la aplicación ni reactivar el guardado de historial que el usuario había apagado a propósito. La decisión **no está documentada en el repositorio** fuera del nombre del método.

---

2 · Constantes de compilación

Estas no son configurables en ejecución. Cambiarlas exige recompilar.

Límites operativos

Constante	Valor	Archivo	Consecuencia de subirla
Imágenes por lote	`20`	`scanner_screen.dart`	Más memoria y más tiempo de bloqueo del isolate
Páginas de PDF	`50`	`scanner_screen.dart`, `pdf_page_renderer_io.dart`	Idem, y más archivos temporales simultáneos
Píxeles objetivo del render	`2400` en el lado mayor	`_renderScale`	Memoria: un A4 a 4× son ~3400×4800 px
Escala mínima y máxima	`[1.5, 4.0]`	idem	Códigos pequeños ilegibles por debajo
Registros de historial	`5000`	`HistoryRepository.maxItems`	Tiempo de carga: `load` descifra todos
Registros por respaldo	`5000`	`ImportService.maxHistoryRecords`	Debe seguir igual que el anterior; ver R-15
Productos por sesión importada	`10000`	`ImportService.maxInventoryItems`	Memoria
Tamaño de archivo importado	`25 MB`	`ImportService.maxImportBytes`	Se decodifica completo en memoria
Profundidad de JSON	`32`	`ImportService.maxJsonDepth`	Defensa contra JSON hostil
Nodos de JSON	`100 000`	`ImportService.maxJsonNodes`	Idem
Entradas de diagnóstico	`100`	`AppDiagnostics.maxEntries`	Solo memoria

`docs/quality/PERFORMANCE.md` añade la regla de proceso: **estos límites no deben ampliarse sin medir memoria, tiempo y temperatura en dispositivos reales**.

Comportamiento de la interfaz

Constante	Valor	Archivo	Qué controla

`HandheldFrame.maxWidth`	`560`	`app.dart`	Umbral de centrado en pantall as anchas
Lado del marco	`0.68` del lado menor	`scanner_screen.dart`	Tamaño del cuadrado de lect ura
Radio del marco	`28`	`scanner_overlay.dart`	Esquinas redondeadas del re corte
Longitud de la esquina	`42`	idem	Las cuatro escuadras
Opacidad del velo	`0.58`	idem	Oscurecimiento fuera del mar co
Duración del barrido	`2200 ms` con inversión	idem	Velocidad de la línea
Antirrebote del escáner	`2 s` por combinación	`scanner_screen.dart`	Evita releer el mismo conjunto
Antirrebote del inventario	`1200 ms` por código	`inventory_screen.dart`	Permite recontar más rápido
Semilla del tema claro	`#006B66`	`app_theme.dart`	Color base
Semilla del tema oscuro	`#5BD7CD`	idem	Color base
Fondo del icono adaptativo	`#005854`	`tool/bootstrap.py`	Color del icono de Android

Criptografía

Constante	Valor	Consecuencia de cambiarla
`PayloadCipher.currentVersion`	`2`	**Datos existentes ilegibles** si se rompe l a compatibilidad
`PayloadCipher.currentAlgorithm`	`'AES-256-GCM'`	Idem
`PayloadCipher.currentKeyId`	`'v2'`	Se buscaría una llave que no existe
`PayloadCipher.legacyKeyId`	`'v2'`	Los sobres heredados dejarían de leerse
`SecureStorageKeyProvider._prefix`	`'ucs_database_key_'`	Se perdería el acceso a todas las llaves g uardadas
`SchemaMigrator.currentVersion`	`4`	Bajarla deja bases existentes por delante del código

Estas seis son las **más peligrosas del repositorio**. Cambiar cualquiera sin una migración explícita convierte los datos del usuario en ruido.

Señales de riesgo

Dos conjuntos literales en `scan_security_analyzer.dart`:

- **`_shorteners`**, 11 dominios: `bit.ly`, `tinyurl.com`, `t.co`, `goo.gl`, `ow.ly`, `buff.ly`, `is.gd`, `cutt.ly`, `rebrand.ly`, `shorturl.at`, `tiny.one`.
- **`_dangerousExtensions`**, 17 extensiones: `.apk .appx .bat .cmd .com .dmg .exe .hta .iso .jar .js .msi .pkg .ps1 .scr .vbs .zip`.

Ambas son listas cerradas y **no se actualizan sin recompilar**. Es la consecuencia directa de no consultar reputación remota: la aplicación no puede aprender de acortadores nuevos. Está declarado como límite en `README.md` y en `docs/SECURITY.md`.

Y dos listas más, en otros archivos:

- **Campos enmascarados**, en `scan_result_sheet.dart`: `Contraseña`, `Secreto`, `Dirección`, `IBAN`, `Carga`.
- **Palabras de inicio de sesión**, en `_looksLikeLogin`: `login`, `signin`, `verify`, `account`, `password`, `wallet`, `bank`.

---

3 · Configuración del repositorio

`pubspec.yaml`

Campo	Valor	Nota
`name`	`universal_code_scanner`	Determina el prefijo de todos los `import 'package:…'`
`version`	`1.1.0+2`	`validate_structure.py` exige la forma `X.Y.Z+N` y que coincida con `appVersion`
`publish_to`	`"none"`	Impide una publicación accidental en pub.dev
`environment.sdk`	`>=3.12.0 <4.0.0`	—
`environment.flutter`	`>=3.44.0`	El toolchain real se fija en `.fvmrc`
`flutter.assets`	`assets/sounds/`	**Única carpeta empaquetada**. Los iconos de `assets/launcher/` los copia `bootstrap.py`, no se empaquetan

Detalle no obvio: `assets/launcher/` **no está en** **`flutter.assets`**. Sus PNG no viajan dentro del APK como recursos de Flutter; los copia `bootstrap.py` a `android/app/src/main/res/` y a `web/icons/`, donde el sistema operativo los consume como iconos nativos.

Los tres anclajes de dependencia

Paquete	Fijado en	Motivo exacto
`share_plus`	`12.0.2`	Desde `13.0.0` exige `win32 ^6`, incompatible con el `win32 ^5.9` de `file_picker 11.x`
`pdfrx`	`2.4.5`	Desde `2.4.6` arrastra `archive ^4`, incompatible con el `archive ^3.6.1` de `excel 4.0.6`
`excel`	`4.0.6`	Última publicada; es la que ancla `archive` a la serie 3

Los dos primeros están además en `.github/dependabot.yml` con la razón escrita al lado, para que Dependabot no proponga subirlos. `flutter pub outdated` sí los muestra como actualizables: **subirlos rompe** **`pub get`**.

`analysis_options.yaml`

```
include: package:flutter_lints/flutter.yaml
linter:
 rules:
   always_declare_return_types: true
   avoid_print: true
   prefer_final_locals: true
   use_build_context_synchronously: true
```

Cuatro reglas sobre `flutter_lints 6.0.0`. `avoid_print` es un control de privacidad además de estilo: se combina con `check_sensitive_logging` del validador, que además prohíbe que cualquier `print`, `debugPrint` o `log` reciba `rawValue`, `password`, `secret`, `otp` o `payload`.

Comprobado: `grep -rn "print(" lib --include=*.dart | wc -l` devuelve `0`.

`.gitattributes`

```
* text=auto eol=lf
*.png binary
*.jpg binary
*.jpeg binary
*.zip binary
*.ttf binary
*.otf binary
*.ps1 text eol=crlf
```

El comentario que lo encabeza explica el porqué: **los SHA-256 de `SOURCE_MANIFEST.json` dependen de que los finales de línea sean estables tras un clon en Windows**. La excepción de `.ps1` a CRLF es para que PowerShell lo lea de forma nativa.

Consecuencia práctica: cualquier archivo de texto nuevo que se añada al repositorio **debe escribirse con LF**. Un CRLF de más rompe la verificación del manifiesto.

Variables de los workflows

Variable	Valor	Dónde	Para qué
`FLUTTER_VERSION`	`3.44.7`	Ambos workflows	`subosito/flutter-action@v2`
`APP_BASE_HREF`	`/universal-code-scanner/app/`	`deploy-landing.yml`	`flutter build web --base-href`

`APP_BASE_HREF` **debe empezar y terminar en `/`**, y hay un comentario en el workflow que lo dice. Es la ruta desde la que GitHub Pages sirve la PWA.

`.gitignore`

Además de los artefactos habituales (`.dart_tool/`, `build/`, `coverage/`, `.idea/`, `.vscode/`, `*.iml`), excluye las **cuatro carpetas de plataforma**:

```
# Se generan con tool/bootstrap.py
/android/
/ios/
/web/
/macos/
```

Es la decisión estructural del repositorio. Ver 02-installation-and-execution.md.

4 · Matriz de decisión: qué ajuste tocar para conseguir qué

Quiero…	Ajuste	Valor
Que no quede rastro de mis lecturas	Sesión privada	Activada
Que no quede rastro nunca	Guardar historial	Desactivado
Que el historial se limpie solo	Retención del historial	30, 90 o 365 días
Leer códigos fuera del marco central	Marco de lectura real	Desactivado
Que nadie más abra la aplicación	Bloqueo de la aplicación	Activado
Escanear en silencio	Sonido de confirmación	Desactivado
Escanear sin vibración	Vibración	Desactivada
Ver las contraseñas Wi-Fi sin pulsar el oj o	Ocultar valores sensibles	Desactivado
Abrir enlaces sin diálogo (salvo los de rie sgo)	Confirmar antes de abrir	Desactivado
Que lo copiado no se quede en el portap apeles	Borrar portapapeles	15 s
Que lo copiado se quede	Borrar portapapeles	Nunca
Menos animación	Reducir movimiento	Activado
Botones más grandes	Controles más grandes	Activado
Más contraste	Alto contraste	Activado
Cambiar la llave de cifrado	Rotar llave de cifrado	Acción, no ajuste

---

5 · Lo que no es configurable

Merece decirse, porque alguien lo buscará:

- **El idioma inglés.** Existe la infraestructura, existen las claves, y el delegado lo rechaza a propósito.
- **Las banderas de capacidad.** Se guardan pero no se leen ni se pueden cambiar.
- **La lista de acortadores y de extensiones peligrosas.** Literales de compilación.
- **Los límites de lote, de historial y de importación.** Constantes.
- **La ubicación de la base de datos.** La decide `path_provider`.
- **El nivel de riesgo a partir del cual se exige confirmación.** Está fijo en «cualquier cosa que no sea `low`».
- **Qué campos se enmascaran.** Lista literal de cinco nombres.

Continúa en 11 · Seguridad.

11 · Seguridad

Este documento enumera los controles de seguridad **presentes y ausentes**, y describe los hallazgos sin publicar cadenas explotables. No se ejecutó ninguna prueba destructiva ni ningún ataque contra sistemas externos: el análisis es de código y de configuración.

La documentación previa de seguridad sigue siendo normativa y este documento la complementa: `../SECURITY.md`, `../security/THREAT_MODEL.md`, `../security/MASVS_CHECKLIST.md` y `../security/SECURITY_TEST_PLAN.md`.

Conclusión primero

La superficie de ataque de esta aplicación es **pequeña por construcción**: no hay red saliente propia, no hay servidor, no hay cuentas y no hay ejecución de código de terceros. Lo que queda son cuatro superficies reales —el contenido de un código escaneado, un archivo importado, el almacenamiento local y las aplicaciones externas a las que se delega una acción— y las cuatro tienen controles explícitos.

Los tres controles criptográficos centrales están **bien construidos y probados**: la llave ausente no se regenera, la rotación limpia su propia basura, y la migración verifica antes de borrar. Las carencias principales no están en la criptografía sino en dos asimetrías de aplicación de política —el inventario no filtra contenido sensible; la exportación sale en claro sin advertencia en la propia interfaz— y en que las acciones de GitHub no están fijadas a SHA.

---

1 · Activos que se protegen

Tomados de `../security/THREAT_MODEL.md` y verificados contra el código:

Activo	Dónde vive	Protección
Historial de lecturas	Sembast `scan_history`	AES-256-GCM por registro
Sesiones de inventario	Sembast `inventory_sessions`	AES-256-GCM por registro
Contraseñas Wi-Fi leídas	En memoria, durante la sesión	No se persisten desde el escáner; enmascaradas en pantalla
Semillas OTP	Idem	Idem
Datos de pago (EMVCo, SEPA, Swiss QR)	Idem	Idem
Identificación AAMVA	Idem	Idem
Llave de cifrado	Keychain / Keystore	`flutter_secure_storage`; nunca se exporta
Archivos exportados	Fuera de la aplicación	**Ninguna**: salen en claro por decisión del usuario

---

2 · Criptografía

Lo que se usa

Aspecto	Valor	Evidencia
Algoritmo	AES-256-GCM (cifrado autenticado)	`AesGcm.with256bits()` en `payload_cipher.dart`
Biblioteca	`cryptography 2.9.0`	`pubspec.yaml`
Generación de llave	`algorithm.newSecretKey()` de la propia biblioteca	`SecureStorageKeyProvider.readOrCreate`
Nonce	Lo genera `cryptography` por operación y se guarda en el sobre	`SecretBox.nonce`
Autenticación	MAC de GCM, guardado en el sobre	`SecretBox.mac.bytes`
Codificación	Base64 para nonce, texto cifrado y MAC	`base64Encode`

No hay criptografía escrita a mano. `grep -rn "Random(" lib` no devuelve nada: no existe ninguna generación de aleatoriedad propia que pudiera ser débil. La única generación de material criptográfico pasa por `newSecretKey()`.

`crypto 3.0.7` se usa solo para **SHA-256 y SHA-1 de identificadores**, nunca para proteger nada:

Uso	Algoritmo	Para qué
`ScanRecord.id`	SHA-256 truncado a 20	Identificador único de registro
`InventorySession.id`	SHA-1 truncado a 16	Identificador único de sesión
`RecoveryIssue.id`	SHA-256 truncado a 24	Deduplicación determinista de incidencias

El SHA-1 de `InventorySession.id` **no es un problema de seguridad**: la entrada incluye `microsecondsSinceEpoch`, y el valor es un identificador local, no una firma ni un control de integridad. Aun así, usar dos funciones hash distintas para el mismo propósito en dos modelos hermanos es una inconsistencia. R-31.

Las tres propiedades que hay que proteger

Son las que sostienen todo lo demás. Las tres tienen prueba dedicada.

1 · Una llave ausente nunca se regenera en silencio.

`decryptJson` registra el `keyId` en `_missingKeyIds` al no encontrar la llave, y `encryptJson` **comprueba ese mismo conjunto antes de cifrar**. Sin la segunda mitad, un descifrado fallido seguido de cualquier escritura llamaría a `readOrCreate`, crearía una llave nueva bajo el mismo identificador y dejaría los datos antiguos irrecuperables **incluso si la llave original reapareciera**.

Prueba: `test/core/payload_cipher_test.dart`, «a missing decryption key is never replaced silently».

2 · Una rotación fallida no deja material huérfano.

`rotateEncryptionKey` precalcula todos los sobres, escribe registros y metadatos en una sola transacción, y solo entonces activa la llave en memoria. Si algo falla, el `catch` llama a `forgetKey(keyId)` dentro de un `try` anidado que no oculta el error original.

Prueba: `test/core/data_maintenance_test.dart`, «failed rotation removes a newly generated orphan key».

3 · La manipulación del texto cifrado se rechaza.

GCM autentica: alterar un solo bit del `cipherText` hace fallar la validación del MAC. Prueba: `test/core/payload_cipher_test.dart`, «tampered authenticated ciphertext is rejected».

Y dos rechazos explícitos más, ambos probados: una versión de sobre superior a la conocida (`FormatException`) y un algoritmo distinto de `AES-256-GCM` (`FormatException`). Es la política correcta —rechazar en vez de intentar interpretar— y está recogida como compromiso en `../quality/COMPATIBILITY_CONTRACT.md`.

Lo que queda en claro, y por qué

Dato	Dónde	Motivo
`id` del registro	Clave de Sembast	Necesario para direccionar
`scannedAt` / `createdAt`	Campo del registro	Necesario para ordenar (`SortOrder`) y po dar (`Filter.lessThan`) sin descifrar 5 00 0 registros
Cabecera del sobre (`version`, `algorithm`, `keyId`, `createdAt`)	Dentro del `payload`	Sin ella no se podría decidir cómo descifr ar
Todo `recovery_issues` salvo `encrypted Payload`	Almacén propio	Solo metadatos técnicos
Las 22 claves de preferencias	`SharedPreferences`	Son decisiones del usuario, no datos esc aneados

Consecuencia para una auditoría: con acceso al archivo de base de datos pero sin la llave, un atacante conoce **cuántas lecturas hubo y cuándo**, y nada más sobre su contenido. Es un metadato real y está documentado aquí porque no lo estaba en ningún otro sitio del repositorio.

Almacenamiento de la llave

`SecureStorageKeyProvider` usa `const FlutterSecureStorage()` **sin opciones explícitas**: no se pasan `AndroidOptions` ni `IOSOptions`. Comprobado con `grep`: no aparecen en `lib/`. Se aceptan por tanto los valores por defecto de `flutter_secure_storage 10.3.1`.

Inferencia basada en el código: en esa serie del plugin el respaldo por defecto de Android es `EncryptedSharedPreferences` y el de Apple el Keychain con la accesibilidad por defecto. **Requiere validación** en dispositivo, porque el comportamiento exacto depende de la versión del plugin y del sistema. Lo que sí está fijado desde el repositorio es `android:allowBackup="false"`, aplicado por `tool/bootstrap.py`, que impide que el sistema restaure una copia sin el material criptográfico compatible.

En la **web**, `flutter_secure_storage` no dispone de un almacén protegido por el sistema operativo: su implementación depende del origen del navegador. `docs/SECURITY.md` ya lo advierte. **Requiere validación** el comportamiento concreto de la PWA publicada; es la plataforma con la garantía más débil.

---

3 · Autenticación local

No hay autenticación remota porque no hay servidor. Lo que existe es un bloqueo local opcional.

Aspecto	Comportamiento
Activación	Ajustes → «Bloqueo de la aplicación»

Requisito para activar	`isDeviceSupported()` **y** una autenticación correcta en ese mismo momento
Momento de bloqueo	`inactive`, `paused`, `hidden` o `detached`
Momento de desbloqueo	Al volver, con `authenticate()`
Fallo de plataforma	`catch` → `false` → **no autenticado**
Bandera del plugin	`persistAcrossBackgrounding: true`

Dos decisiones acertadas:

No se puede activar a ciegas. `_changeBiometricLock` exige una autenticación correcta antes de guardar el ajuste. Si el dispositivo no lo admite o la persona cancela, el ajuste no cambia y se muestra «No se pudo activar el bloqueo en este dispositivo». Nadie se queda fuera de su propia aplicación.

Cualquier fallo se resuelve como «no autenticado». Ambos métodos de `BiometricService` capturan `on Object` y devuelven `false`. Es la dirección correcta para un control de seguridad.

Límites del bloqueo

- **No cifra nada adicional.** La llave AES vive en el almacén seguro independientemente de si el bloqueo está activo. Alguien con acceso al dispositivo desbloqueado y a las herramientas adecuadas no se topa con este control.
- **`didUpdateWidget` desbloquea si el ajuste se apaga** (`if (!widget.settings.value.biometricLock) _unlocked = true;`). Es coherente, pero significa que apagar el ajuste surte efecto inmediato sin reautenticar.
- **No hay reintento limitado ni bloqueo temporal**: eso lo gestiona el diálogo del sistema, no la aplicación.

---

4 · Validación de entrada

4.1 Contenido de un código escaneado

Es la entrada menos confiable de todas: un código lo puede escribir cualquiera.

Control	Dónde
Ninguna acción se ejecuta automáticamente	`_openRecord` exige pulsación explícita
Bloqueo de esquemas de aplicación no permitidos	`analyze`, rama 4: cualquier esquema que no sea `http`, `https`, `mailto`, `tel`, `sms`, `smsto` o `geo` produce `high` y `canOpen: false`
16 señales locales sobre URL	`analyze`, ramas 1 a 16
Confirmación obligatoria con riesgo distinto de `low`	`shouldConfirm` es una disyunción entre el ajuste y el nivel
El intérprete nunca lanza	Todas las ramas devuelven; la última es `ContentKind.text`
El TLV de EMVCo se detiene en vez de fallar	`_tlv` hace `break` ante un campo mal formado

La prueba `test/scan_security_analyzer_test.dart` incluye un caso con un esquema de aplicación activo y verifica `high` y `canOpen: false`. Esa es la defensa concreta contra un código que intente inyectar un esquema ejecutable.

Lo que el analizador no hace, y está declarado: no consulta reputación remota, no valida certificados, no sigue redirecciones y **nunca declara que un enlace sea seguro**. Su nivel más bajo se llama literalmente «Sin señales evidentes».

4.2 Archivo JSON importado

Seis puertas antes de que un byte llegue a la base de datos. Detalle en 08-data-flow.md:

Puerta	Límite	Ataque que mitiga
Tamaño	25 MB	Agotamiento de memoria
Profundidad	32 por rama	Desbordamiento de pila al recorrer
Nodos	100 000	Agotamiento de CPU y memoria
Procedencia	`application` y `type` exactos	Importar un JSON de otra aplicación
Versión de esquema	`≤ 2`	Interpretar un formato futuro desconocido
Cantidad	5 000 registros / 10 000 productos	Agotamiento

`_validateJsonShape` usa **una pila explícita, no recursión**: es lo que permite que la comprobación de profundidad se ejecute antes de que el propio recorrido desborde. Hay prueba dedicada en `test/services/import_service_test.dart` («excessively deep JSON is rejected before persistence»).

Además, **nada se escribe hasta que el usuario elige una estrategia**: la vista previa muestra válidos, duplicados y rechazados y la base sigue intacta.

4.3 Entrada de teclado

`trim()` en notas, etiquetas, nombres de sesión y notas de producto; las etiquetas se deduplican con `toSet()`. No hay inyección posible: **no hay SQL, no hay plantillas, no hay `eval`, no hay HTML renderizado desde datos del usuario**. Todo se muestra con widgets `Text` y `SelectableText`, que no interpretan marcado.

---

5 · Privacidad

Control	Implementación	Verificado por
Cero telemetría	Ninguna dependencia de analítica; ningún cliente de red	`grep` sobre `pubspec.yaml` y `lib/`
El diagnóstico no puede filtrar datos	`AppDiagnostics.record` acepta un `Object` y guarda `runtimeType`, no el mensaje	`test/core/diagnostics_test.dart`
El paquete de recuperación no lleva llaves	Los sobres salen cifrados	`test/core/recovery_service_test.dart`
OTP y Wi-Fi con contraseña fuera del historial	`_persistAndShow` filtra por `isSensitive`	Ver la excepción en el apartado 7
Enmascarado en pantalla	Cinco nombres de campo, con botón para revelar	—
Portapapeles con borrado programado	`ClipboardService.copy`, comprobando que el valor no haya cambiado	—

Sesión privada	`private_mode` desactiva toda persistencia del escáner	—
Prohibición de registrar cargas	`avoid_print` más `check_sensitive_logging` en el validador	`grep -rn "print(" lib` devuelve `0`

`check_sensitive_logging` de `tool/validate_structure.py` es un gate real: falla la validación si algún `print`, `debugPrint` o `log` recibe un argumento que contenga `rawValue`, `password`, `secret`, `otp` o `payload`.

---

6 · Permisos y superficie de plataforma

Android

Permiso	Declarado por	Por qué
`android.permission.CAMERA`	`tool/bootstrap.py`	Lectura con cámara
`android.permission.USE_BIOMETRIC`	idem	Bloqueo de la aplicación
`android.hardware.camera.any` con `required="false"`	idem	Permite instalar sin cámara

No se piden `INTERNET`, `READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`, `ACCESS_FINE_LOCATION` ni ningún otro. **La ausencia de `INTERNET` en el manifiesto principal es la comprobación más contundente de que no hay red saliente propia**: aunque alguien añadiera un cliente HTTP, la aplicación no tendría permiso para usarlo.

Nota: Flutter añade `INTERNET` al manifiesto de **depuración** (`android/app/src/debug/AndroidManifest.xml`) para el conector del depurador. El manifiesto principal, que es el que se compila en release, no lo declara.

`android:allowBackup="false"` bloquea la restauración automática.

Apple

Clave	Texto
`NSCameraUsageDescription`	«La cámara se utiliza para leer códigos QR y códigos de barras.»
`NSPhotoLibraryUsageDescription`	«La galería se utiliza para analizar códigos presentes en imágenes seleccionadas.»
`NSFaceIDUsageDescription`	«Face ID protege el acceso a la aplicación cuando el usuario activa el bloqueo.»

Entitlements de macOS: `com.apple.security.device.camera`, `com.apple.security.files.user-selected.read-only` y `keychain-access-groups` vacío. El acceso a archivos es **solo lectura y solo de lo que el usuario seleccione**.

Todo esto lo aplica `tool/bootstrap.py`. Como las carpetas nativas no se versionan, **la única fuente de verdad de estos permisos es ese script**.

Web y PWA

La PWA hereda el modelo de permisos del navegador: la cámara exige un gesto del usuario y un origen seguro. La lectura de PDF **no está disponible** (`pdf_page_renderer_stub.dart` lanza `UnsupportedError` y el menú se

deshabilita con `enabled: !kIsWeb`).

`docs/SECURITY.md` exige publicar la PWA solo por HTTPS con HSTS y cabeceras adecuadas. GitHub Pages sirve por HTTPS; **no se identificó** ninguna configuración de CSP ni de cabeceras de seguridad en `landing/` ni en el workflow. R-32.

---

7 · Hallazgos de seguridad

Descritos sin cadenas explotables. Todos están además en 15-risks-and-technical-debt.md con severidad, probabilidad y prioridad.

S-1 · El inventario persiste contenido sensible (R-03)

Severidad: media. **Evidencia**: `InventoryScreen._onDetect` llama a `widget.store.addScan(record)` sin comprobar `record.isSensitive`, y `InventoryStore.addScan` indexa el ítem por `record.rawValue`.

Un QR de Wi-Fi con contraseña, un OTP o un QR de pago escaneado dentro de una sesión de inventario **se persiste**, con su texto íntegro como clave del ítem. Va cifrado igual que todo lo demás, así que no es una fuga hacia el disco en claro, pero **contradice** el punto 5 de `docs/SECURITY.md` («No persistir automáticamente OTP ni Wi-Fi con contraseña») y la política de privacidad.

Además, ese texto sale **en claro** al exportar la sesión a CSV, XLSX o JSON.

S-2 · La exportación sale en claro sin advertencia en la interfaz (R-27)

Severidad: media. **Evidencia**: `ExportService` no cifra nada; las entradas del menú se llaman «Exportar CSV», «Exportar JSON» y «Exportar Excel XLSX», y no muestran ningún diálogo.

Compartir un registro **individual** que sea sensible sí abre un diálogo de advertencia. Exportar **todo el historial** no. La advertencia existe solo en `docs/SECURITY.md`, que quien usa la aplicación no lee.

S-3 · Las acciones de GitHub no están fijadas a SHA (R-33)

Severidad: media para la cadena de suministro. **Evidencia**: los 13 `uses:` de los dos workflows apuntan a etiquetas móviles (`actions/checkout@v7`, `subosito/flutter-action@v2` y similares), no a un SHA de commit.

Una etiqueta puede reapuntarse. Un compromiso de cualquiera de esos repositorios ejecutaría código arbitrario en el runner que compila los APK publicados. Mitigación parcial ya presente: `permissions: contents: read` en el trabajo de calidad, y `deploy-landing.yml` limita sus permisos a `pages: write` e `id-token: write`.

S-4 · Los APK publicados van firmados con la clave de depuración

Severidad: alta para la distribución, **ya declarada**. `README.md`, `docs/RELEASE.md` e `IMPLEMENTATION_STATUS.md` lo dicen con claridad, incluido que una versión futura firmada con clave propia **no podrá actualizarlos**.

Se registra aquí por completitud: cualquiera puede compilar un APK con esa misma clave conocida. La verificación real la aporta el SHA-256 publicado junto a cada artefacto, no la firma.

S-5 · Las llaves antiguas no se borran tras rotar (R-20)

Severidad: baja. **Evidencia**: `rotateEncryptionKey` no llama a `forgetKey` sobre la llave anterior en el camino de éxito.

Es prudente —permite recuperar un registro que se hubiera quedado atrás— pero significa que el almacén seguro acumula una entrada por rotación y que material criptográfico capaz de descifrar datos antiguos permanece en el dispositivo indefinidamente.

S-6 · Los PNG temporales del PDF son datos en claro en disco

Severidad: baja, **riesgo aceptado**. `PdfPageRenderer` escribe cada página como PNG en el directorio temporal para poder pasar una ruta al motor de lectura. La limpieza cubre los tres caminos de salida —éxito, error y cancelación— y es idempotente. La ventana de exposición es la duración del análisis.

S-7 · La lista de campos enmascarados es literal (R-19)

Severidad: baja. Cinco nombres: `Contraseña`, `Secreto`, `Dirección`, `IBAN`, `Carga`. Un parser nuevo que produjera un campo sensible con otro nombre **no se enmascararía**, aunque el registro estuviera marcado como sensible.

S-8 · `legacyKeyId` y `currentKeyId` valen lo mismo (R-13)

Severidad: baja hoy, **alta si alguien cambia `currentKeyId`**. Ambas constantes valen `'v2'`. Cambiar la primera sin cambiar la segunda rompería silenciosamente la lectura de los sobres heredados.

S-9 · Sin CSP ni cabeceras de seguridad declaradas para la PWA (R-32)

Severidad: baja. **No identificado**: no hay archivo de cabeceras, ni meta CSP en `landing/index.html`, ni paso en el workflow que las añada. GitHub Pages no permite cabeceras personalizadas, así que la mitigación tendría que ser una meta `Content-Security-Policy` en el propio HTML. `docs/SECURITY.md` lo pide y no está.

---

8 · Controles ausentes

Enumerarlos es tan importante como enumerar los presentes.

Control ausente	¿Es un problema?
Certificate pinning	**No**: no hay conexiones propias que fijar
Detección de root o jailbreak	No implementado. **No documentado en el repositorio** como de cisión
Ofuscación del código Dart	No se usa `--obfuscate`. Un APK release es descompilable
Detección de depurador o anti-tampering	No implementado
`FLAG_SECURE` contra capturas de pantalla	**No implementado**: una captura del selector de aplicaciones pue de mostrar una contraseña Wi-Fi en pantalla. `MASVS_CHECKLIST.md` lo lista como pendiente de validar en dispositivo
Bloqueo tras N intentos fallidos	Lo gestiona el sistema, no la aplicación
Borrado seguro de memoria	Dart no lo permite; las llaves quedan en el montón hasta la recol ección
Auditoría de seguridad independiente	**No realizada**. Declarado en `MASVS_CHECKLIST.md`
Escaneo de dependencias por CVE	No hay paso de CI que lo haga. Sí hay SBOM e inventario de lic encias
Firma de publicación	Fuera del alcance de 1.1.0

La ausencia de `FLAG_SECURE` merece atención: la aplicación muestra contraseñas Wi-Fi y secretos OTP en pantalla —tras pulsar el ojo— y sin esa bandera esas pantallas aparecen en la vista de aplicaciones recientes de Android.

---

9 · Cadena de suministro

Control	Estado
`pubspec.lock` versionado	**Sí**
Versiones exactas en `pubspec.yaml`	**Sí**: las 19 dependencias directas usan versión exacta, sin `^`
SBOM CycloneDX	**Sí**: `docs/security/sbom-source.cdx.json` versionado, más u no resuelto en cada CI
Inventario de licencias	**Sí**: `tool/generate_license_inventory.py`, en cada CI
Manifiesto de fuente con SHA-256	**Sí**: `SOURCE_MANIFEST.json`, 150 archivos
Checksums de los artefactos publicados	**Sí**: `checksums.txt` en el release
Dependabot	**Sí**: pub semanal, acciones mensual
Acciones fijadas a SHA	**No** (S-3)
Escaneo de CVE en CI	**No identificado**

El uso de versiones **exactas** en `pubspec.yaml` es una decisión de seguridad poco frecuente y merece reconocerse: `audioplayers: 6.8.1`, no `^6.8.1`. Combinado con el lockfile versionado, hace que dos compilaciones del mismo commit resuelvan el mismo grafo.

`tool/generate_sbom.py` declara honestamente su cobertura: si Flutter está disponible produce un SBOM `resolved-transitive`; si no, uno `declared-direct-only` marcado como tal en `metadata.properties`. No infla la cobertura.

---

10 · Barrido de secretos

Ejecutado sobre todo el árbol versionado, excluyendo `build/`:

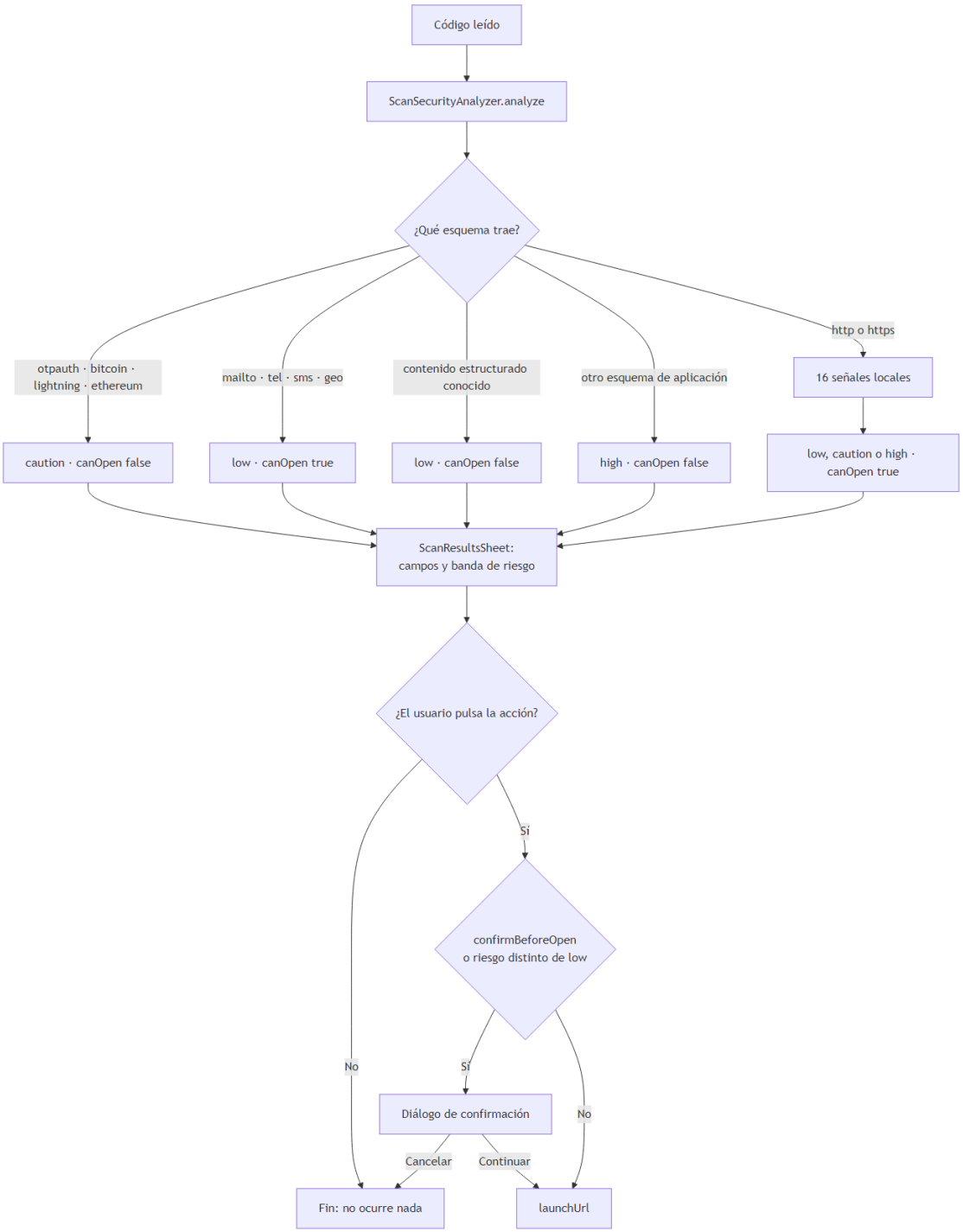
```
grep -rnEi "(sk-|ghp_|gho_|AKIA|BEGIN [A-Z ]*PRIVATE KEY)" \
 --include="*.dart" --include="*.md" --include="*.json" \
 --include="*.yaml" --include="*.yml" --include="*.py" .
```

Sin resultados. Tampoco hay archivos `.env`, `.p12`, `.jks`, `.keystore` ni `.pem` en el repositorio. Los dos workflows **no consumen ningún secreto de GitHub**: no aparece ninguna referencia a `secrets.` en ellos.

Los valores de ejemplo del código son ficticios y usan dominios reservados: `https://example.com` en el generador, `correo@example.com`, y números de teléfono con prefijo chileno inventados.

---

11 · Flujo de decisión de seguridad



Lo que muestra: que hay dos puertas antes de cualquier acción externa —una decisión del usuario y, casi siempre, un diálogo— y que el nivel de riesgo puede imponer el diálogo aunque el usuario lo haya desactivado.

Lo que no muestra: que el paso `launchUrl` delega en el sistema operativo, y que a partir de ahí la seguridad depende de la aplicación que el sistema elija. Tampoco muestra el camino de compartir, que exige diálogo solo para registros sensibles.

---

12 · Recomendaciones, por prioridad

Informativo. **Ninguna se aplicó en este trabajo**: ver la nota de alcance en 15-risks-and-technical-debt.md.

#	Recomendación	Esfuerzo estimado

1	Aplicar el filtro `isSensitive` también en `InventoryScreen._onDetect`, o declarar la excepción en `docs/SECURITY.md`	1 línea más una decisión de producto
2	Añadir un diálogo de advertencia antes de exportar historial o inventario	~30 líneas
3	Fijar las 13 acciones de GitHub a SHA de commit	30 minutos
4	Añadir `FLAG_SECURE` en Android para las pantallas que muestran secretos	Requiere canal de plataforma
5	Borrar la llave anterior tras una rotación confirmada, o documentar por qué no	3 líneas más una decisión
6	Añadir una meta `Content-Security-Policy` a `landing/index.html`	1 línea
7	Añadir un paso de CI que compare el SBOM contra una base de CVE	Media jornada
8	Separar `legacyKeyId` en una constante distinta y documentada	2 líneas

---

Continúa en 12 · Pruebas y calidad.

12 · Pruebas y calidad

Conclusión primero

La suite es **pequeña pero bien elegida**: 57 casos que cubren con precisión los puntos donde un error sería costoso —criptografía, migración, importación, cancelación y estado del escáner— y dejan sin cubrir las pantallas grandes y el camino real de la cámara, que no se pueden probar sin hardware.

Hay un hallazgo que merece nombrarse antes que nada: **el catálogo de regresión de `test_assets/` no lo ejecuta ninguna prueba.** Ocho imágenes con su carga esperada, un manifiesto, y un validador que comprueba únicamente que los archivos existan. Ninguna prueba Dart los decodifica ni los compara. Es el hallazgo R-01.

---

1 · Qué se ejecutó, con su salida real

Sobre el commit `c1f9878`, con Flutter 3.44.7 y Dart 3.12.2 en Windows 11:

`flutter analyze --fatal-infos`

```
Analyzing universal-code-scanner...
No issues found! (ran in 100.8s)
```

Código de salida: `0`.

`flutter test`

```
00:24 +57: All tests passed!
```

Código de salida: `0`. 57 de 57 en verde, en 24 segundos.

`python tool/validate_structure.py`

Su resultado se registra al final de este documento, en el apartado 9, junto con la verificación completa de la entrega.

Estas dos primeras ejecuciones son la **línea base previa a cualquier cambio de este trabajo**. Coinciden con lo que declara `../../VALIDATION.md` y con la insignia del `README.md`.

---

2 · Inventario de la suite

20 archivos de prueba, 58 casos declarados. `flutter test` ejecuta los 57 de `test/`; el de `integration_test/` necesita un dispositivo.

Archivo	Casos	Qué protege
`test/core/payload_cipher_test.dart`	5	Sobre versionado, sobre heredado, llave ausente, manipulación del texto cifrado, v ersión futura
`test/core/data_maintenance_test.da rt`	3	Rotación correcta, aborto ante carga inco mpleta, limpieza de llave huérfana

`test/scan_security_analyzer_test.dart`	9	HTTPS normal, HTTP, correo en la consulta, credenciales, Punycode, esquema no permitido, `tel:`, Wi-Fi, OTP
`test/content_interpreter_test.dart`	6	Wi-Fi con escapes, vCard, GS1, EMVCo, `www.`, ISBN
`test/scan_record_test.dart`	3	Serialización completa, contenido binario, detección de sensibles
`test/services/import_service_test.dart`	4	Vista previa, esquema futuro, duplicados internos, JSON profundo
`test/features/scan_status_bar_test.dart`	5	Barra en movimiento, movimiento reducido, pausa, cámara no disponible, barrido del marco
`test/core/handheld_frame_test.dart`	3	Móvil, umbral exacto, pantalla ancha centrada
`test/core/localization_test.dart`	4	Solo español, etiquetas resueltas, rechazo de inglés, claves inglesas conservadas
`test/services/scan_feedback_test.dart`	4	Tono y vibración, canales independientes, degradación al sonido del sistema, liberación
`test/core/async_write_queue_test.dart`	2	Orden de envío, un fallo no bloquea la cola
`test/core/cancellation_token_test.dart`	2	Cancelación determinista, fracción acotada
`test/services/history_repository_test.dart`	1	`replaceAll` atómico
`test/services/content_parser_registry_test.dart`	1	Parser propio aislado y removible
`test/core/recovery_service_test.dart`	1	El paquete no lleva llaves ni texto claro
`test/core/schema_migrator_test.dart`	1	Migración idempotente y no destructiva
`test/core/diagnostics_test.dart`	1	La exportación excluye mensajes y valores sensibles
`test/core/accessibility_test.dart`	1	Centro de recuperación con texto al 200 %
`test/inventory_session_test.dart`	1	Unidades y serialización
`integration_test/app_launch_test.dart`	1	Arranque normal o seguro. **No la ejecuta `flutter test`**

Distribución por tipo:

Tipo	Casos	Porcentaje

Unitarias puras	39	68 %
De widget (`testWidgets`)	18	32 %
De integración	1	Fuera de `flutter test`

---

3 · Las pruebas que más valen

No todas las pruebas valen lo mismo. Estas cinco son las que impedirían un desastre.

1 · «a missing decryption key is never replaced silently» (`payload_cipher_test.dart`). Comprueba que tras olvidar la llave, tanto `decryptJson` como `encryptJson` lanzan, y que **el proveedor sigue sin la llave** después de ambos intentos. Sin esta propiedad, un descifrado fallido seguido de una escritura destruiría el historial para siempre.

2 · «failed rotation removes a newly generated orphan key» (`data_maintenance_test.dart`). Usa un `_TrackingKeyProvider` que envuelve al de memoria para observar qué llave se creó y cuál se olvidó. Verifica que la llave nueva se creó, se olvidó, ya no se puede leer, y que el metadato de llave activa **no cambió**.

3 · «tampered authenticated ciphertext is rejected» (`payload_cipher_test.dart`). Invierte un bit del texto cifrado y comprueba que el descifrado falla. Es la prueba de que GCM está haciendo su trabajo.

4 · «diagnostic export excludes error messages and sensitive values» (`diagnostics_test.dart`). Registra un `StateError` cuyo mensaje contiene literalmente una contraseña, un código OTP y un dominio, y verifica que ninguno aparece en la exportación —pero que `StateError` sí. Prueba una propiedad de privacidad, no un comportamiento.

5 · «excessively deep JSON is rejected before persistence» (`import_service_test.dart`). Construye un JSON con 34 niveles de anidamiento y comprueba que se rechaza. Protege contra un archivo hostil que agotara la pila.

Una sexta merece mención por lo que revela del cuidado del equipo: `test/core/accessibility_test.dart` levanta el Centro de recuperación con `TextScaler.linear(2)` y comprueba las guías `labeledTapTargetGuideline` y `textContrastGuideline` de Flutter. Y lo hace con `tester.runAsync` porque —según su propio comentario— la base de datos usa temporizadores reales que no funcionan bajo el reloj falso de `testWidgets`. Ese detalle costó un defecto real, registrado en `VALIDATION.md`: la prueba se colgaba diez minutos.

---

4 · Patrones de prueba del repositorio

Merece documentarlos porque quien añada una prueba tendrá que seguirlos.

Base de datos en memoria

```
final AppDatabase database = await AppDatabase.openTemporary();
addTearDown(database.close);
```

Nunca se toca el disco. `openTemporary` genera un nombre único con `microsecondsSinceEpoch`, así que dos pruebas del mismo archivo no comparten estado.

Llaves en memoria

```
final PayloadCipher cipher = PayloadCipher(keyProvider: MemoryEncryptionKeyProvider());
```

`MemoryEncryptionKeyProvider` existe para esto y para el modo temporal. Sin él, las pruebas de cifrado necesitarían un Keychain.

Preferencias en memoria

```
setUp(() {
 TestWidgetsFlutterBinding.ensureInitialized();
 SharedPreferencesAsyncPlatform.instance = InMemorySharedPreferencesAsync.empty();
});
```

Requiere la dependencia de desarrollo `shared_preferences_platform_interface 2.4.2`, que está en `pubspec.yaml` exactamente por esto. Cuatro casos fallaban por no registrarla; el defecto está en `VALIDATION.md`.

Nunca `pumpAndSettle` sobre una animación infinita

```
// Never pumpAndSettle: while it is scanning the bar animates forever, which
// is exactly the state under test.
await tester.pump();
```

El comentario está en el propio `scan_status_bar_test.dart`. `pumpAndSettle` espera a que no queden animaciones; la barra indeterminada nunca para.

Comprobar la animación por el contador de callbacks

```
await pumpOverlay(active: true, animate: true);
expect(tester.binding.transientCallbackCount, greaterThan(0));

await pumpOverlay(active: false, animate: true);
await tester.pump(const Duration(milliseconds: 16));
expect(tester.binding.transientCallbackCount, 0);
```

Es la forma correcta de verificar que una animación **está parada**: contar los callbacks transitorios en vez de inspeccionar píxeles.

Dobles por interfaz, no por biblioteca de mocks

No hay `mockito` ni `mocktail` en `pubspec.yaml`. Los dobles se escriben a mano implementando la interfaz: `_FakeTonePlayer`, `_TrackingKeyProvider`, `_TestParser`. Es coherente con las fronteras estrechas descritas en 09-apis-and-integrations.md: interfaces de dos a cuatro miembros no necesitan generación de código.

---

5 · El catálogo de regresión que nadie ejecuta

Este es el hallazgo principal de este documento.

`test_assets/` contiene ocho imágenes PNG y un `manifest.json` que declara, para cada una, su **carga esperada**, su formato y su escenario:

Archivo	Escenario	Carga esperada
`qr/url_https.png`	`qr`	Una URL de ejemplo
`qr/wifi.png`	`qr`	Un QR de Wi-Fi con contraseña
`qr/vcard.png`	`qr`	Una vCard completa
`gs1/gs1_text.png`	`gs1`	Identificadores GS1 con GTIN, vencimien to y lote

`damaged/url_partially_damaged.png`	`damaged`	La misma URL, con daño parcial
`low_light/wifi_low_light.png`	`low_light`	El mismo Wi-Fi, con poca luz
`multiple_codes/url_and_vcard.png`	`multiple_codes`	**Dos** cargas en una imagen
`curved/url_warped.png`	`curved_simulation`	La misma URL, deformada

`tool/validate_structure.py::check_test_assets` comprueba tres cosas: que el manifiesto exista, que tenga `schemaVersion: 1` y una lista `items`, y que **cada archivo declarado exista en disco**.

Lo que **no** comprueba: nada del contenido. No decodifica ninguna imagen, no compara con `expectedPayload`, no verifica el formato.

Y la verificación decisiva:

```
grep -rn "test_assets" test/ integration_test/ lib/ | wc -l
# → 0
```

Ninguna prueba Dart lee ese directorio. Los ocho escenarios —daño parcial, poca luz, deformación, varios códigos en una imagen— están preparados, versionados y documentados, y **nunca se ejecutan**. Su única función actual es alimentar la matriz manual de `../quality/DEVICE_TEST_MATRIX.md`, que los menciona como «recursos reproducibles».

Además, seis carpetas contienen únicamente un `README.md` sin imágenes: `aztec/`, `datamatrix/`, `ean/`, `micro_qr/`, `pdf417/` y `upc/`. Esto **sí está declarado** y es una decisión defendible: `IMPLEMENTATION_STATUS.md` dice que las demás simbologías «requieren capturas reales verificadas antes de incorporarse», y `test_assets/README.md` añade que «no se inventan muestras que puedan entregar falsos positivos».

Inferencia basada en el código: una prueba que use `MobileScannerController.analyzeImage` sobre estas imágenes necesitaría el motor de plataforma, que no existe en `flutter test`. Sería una prueba de integración con dispositivo, no unitaria. Esa es probablemente la razón de que no exista; **no está documentada en el repositorio**.

---

6 · Qué queda sin cubrir

Sin cobertura de prueba automatizada

Área	Líneas aproximadas	Por qué no está cubierta	¿Es aceptable?
`ScannerScreen`	638	Necesita cámara de plataform a	Sí, salvo la lógica pura
`InventoryScreen`	491	Idem	Sí, salvo la lógica pura
`SettingsScreen`	378	Widget grande, sin lógica críti ca propia	Parcialmente
`ScanResultsSheet` y `ScanRe cordCard`	318	**Contiene la decisión de en mascarado y la de confirma ción**	**No**
`GeneratorScreen`	310	`_escape` y `_normalizeUrl` so n puras y no se prueban	**No**
`HistoryScreen`	284	Filtrado en memoria, probable con widget test	Parcialmente

`ExportService`	164	`_encodeCsv` y `_encodeXlsx` son funciones puras	**No**
`PdfPageRenderer`	115	Necesita `pdfrx` y sistema de archivos	Sí
`RecoveryScreen`	139	Cubierto indirectamente por la prueba de accesibilidad	Parcialmente
`FormatsScreen`	170	Puramente informativa	Sí
`InventoryStore` y `ScanStore`	248	Solo `AsyncWriteQueue` se prueba por separado	**No**
`EncryptionMetadataRepository`	30	La migración desde preferencias no tiene prueba propia	**No**

Casos límite conocidos sin prueba

Cada uno corresponde a una decisión real del código:

1. **La migración del historial desde preferencias.** Es el flujo con más precauciones del repositorio —respaldo cifrado, verificación dentro de la transacción, borrado solo al final— y **no tiene ninguna prueba**. R-34.
2. **La actualización perezosa de sobres al leer.** `load` reescribe los sobres antiguos; ninguna prueba lo comprueba.
3. **`_trim()` al superar 5 000 registros.** No hay prueba de que borre los más antiguos.
4. **`pruneOlderThan`.** No hay prueba del filtro por fecha.
5. **El filtro de contenido sensible en `_persistAndShow`.** Se prueba que `isSensitive` sea `true` para OTP y Wi-Fi, pero **no que el registro quede fuera del historial**.
6. **La deduplicación de dos segundos del escáner.**
7. **`_escape` y `_normalizeUrl` del generador**, ambas puras y triviales de probar.
8. **`_encodeCsv`**: el BOM, el `\r\n` y el entrecomillado universal son decisiones concretas y comprobables sin plataforma.
9. **`resetNonSensitive`**: que deje intactas exactamente tres claves.
10. **`ContentInterpreter`**: 6 casos para 20 ramas. Sin cobertura para MeCard, vEvent, `mailto:`, `tel:`, `sms:`, `geo:`, `otpauth:`, cripto, Swiss QR, EPC/SEPA, AAMVA, binario ni producto.

---

7 · Cobertura

`flutter test --coverage` genera `coverage/lcov.info`, y la CI lo sube como artefacto. **No hay umbral mínimo declarado en ningún sitio**: ni en el workflow, ni en `validate_structure.py`, ni en la documentación. Nada falla por bajar la cobertura. R-35.

No se ejecutó `--coverage` en este análisis, así que **no se dispone de un porcentaje medido**. Lo que sí se puede afirmar contando líneas:

Capa	Líneas	Con prueba directa
`core/security`	435	Sí, extensiva
`core/database`	91	Solo `SchemaMigrator`

`core/performance`	58	Sí, ambas clases
`core/recovery`	197	Solo `exportBundle`
`models`	398	Parcial
`services`	1 401	Parcial: importación, feedback, historial
`state`	270	No
`features`	3 072	Solo `ScanStatusBar` y `ScannerOverlay`

Inferencia: la cobertura de línea estará dominada por `features/`, que es el 45 % del código y casi no tiene pruebas. Un porcentaje global sería engañoso: lo relevante es que **el 100 % de la criptografía tiene prueba y el 0 % de los stores también**.

---

8 · Análisis estático y gates

`analysis_options.yaml`

```
include: package:flutter_lints/flutter.yaml
linter:
 rules:
   always_declare_return_types: true
   avoid_print: true
   prefer_final_locals: true
   use_build_context_synchronously: true
```

Cuatro reglas sobre `flutter_lints 6.0.0`. Dos merecen comentario:

- **`avoid_print`** es un control de privacidad, no solo de estilo. Se combina con `check_sensitive_logging` del validador.
- **`use_build_context_synchronously`** es el que obliga al patrón `if (!mounted) return;` y `if (context.mounted)` que aparece en todas las pantallas. Veintidós avisos de esta clase se corrigieron durante la validación de 1.0.0, según `VALIDATION.md`.

La CI usa `--fatal-infos`, que convierte incluso los avisos informativos en fallo. Es el nivel más estricto disponible.

Discrepancia real: `tool/run_quality_gate.sh` ejecuta `flutter analyze` **sin** `--fatal-infos`. Un desarrollador que use la puerta de calidad local puede pasar en verde y fallar en la CI. R-06.

Los diez gates de `validate_structure.py`

Ya enumerados en 04-code-map.md. Lo que revelan sobre las prioridades del proyecto:

Gate	Qué considera irrompible
`check_interface_version`	Que la versión mostrada al usuario sea la real. Nació de un defe cto concreto
`check_sensitive_logging`	Que nunca se registre una carga escaneada
`check_absolute_paths`	Que no se filtre una ruta del entorno de origen
`check_markdown_links`	Que ningún enlace de la documentación esté roto

`check_imports`	Que ningún `import` interno apunte a un archivo inexistente
`check_test_assets`	Que los recursos declarados existan
`check_source_sbom`	Que el SBOM de fuente exista y sea CycloneDX

`check_markdown_links` es especialmente útil para este trabajo: **valida los enlaces relativos de los 20 documentos nuevos gratis**, sin escribir un verificador aparte.

Los dos workflows

`flutter-ci.yml` corre en `push` a `main` y `develop`, en cada pull request y a petición. Dos trabajos:

Trabajo	Runner	Pasos
`quality`	`ubuntu-latest`	Validación offline con `--require-lock`, `bootstrap.py`, `pub deps`, `analyze --fatal-infos`, `test --coverage`, SBOM, licencias, `build web --release`, `build apk --debug`, checksums, subida de artefactos
`apple`	`macos-latest`	`bootstrap.py --platforms ios,macos`, `build ios --release --no-codesign`, `build macos --release`

El trabajo `apple` **no ejecuta la prueba de integración**, con el motivo escrito como comentario al final del archivo: los runners macOS de GitHub no tienen sesión gráfica interactiva, así que la aplicación arranca sin llegar a establecer la conexión de depuración. Es un ejemplo de degradación declarada en vez de silenciosa.

---

9 · Verificación de esta entrega

Ejecutado tras escribir la documentación y añadir los comentarios de código, para comprobar que nada de este trabajo rompió la base verde. La salida real está en el informe final; el resumen es:

Comprobación	Antes de los cambios	Después
`flutter analyze --fatal-infos`	`No issues found!`	`No issues found!`
`flutter test`	`+57: All tests passed!`	`+57: All tests passed!`
`python tool/validate_structure.py`	Sin errores	Sin errores
Enlaces relativos de la documentación	—	Validados por `check_markdown_links`
Mojibake	—	`grep -rl $'\xc3\x83'` sin resultados

---

10 · Propuesta priorizada

Informativo. **No se implementó ninguna** en este trabajo.

#	Propuesta	Valor	Esfuerzo

1	Ejecutar el catálogo de `test_assets/` en una prueba de integración con dispositivo, comparando contra `expectedPayload`	**Alto**: hoy ocho escenarios están preparados y sin usar	Media jornada
2	Probar la migración del historial desde preferencias, incluidos el respaldo cifrado y el fallo	**Alto**: es el flujo con más riesgo de pérdida de datos	3 horas
3	Probar que un registro sensible **no** entra en el historial	**Alto**: es una garantía de privacidad enunciada públicamente	1 hora
4	Probar `_trim` y `pruneOlderThan`	Medio	2 horas
5	Alinear `run_quality_gate.sh` con la CI añadiendo `--fatal-infos`	Medio, coste cero	1 minuto
6	Ampliar `ContentInterpreter` a las 20 ramas	Medio	3 horas
7	Probar `_encodeCsv`, `_escape` y `_normalizeUrl`	Medio, son funciones puras	1 hora
8	Probar `ScanRecordCard`: enmascarado y confirmación	Medio	2 horas
9	Declarar un umbral de cobertura y hacerlo fallar la CI	Bajo hoy, alto a largo plazo	1 hora
10	Añadir un paso de CI que compare el SBOM contra una base de CVE	Medio	Media jornada

---

11 · Lo que está bien y conviene proteger

- **La suite entera corre en 24 segundos** y no toca disco, red ni plataforma. Es una suite que la gente ejecutará.
- **Las pruebas de criptografía prueban propiedades, no implementaciones.** «Una llave ausente nunca se reemplaza en silencio» seguirá siendo válida aunque se reescriba `PayloadCipher`.
- **Los comentarios dentro de las pruebas explican el porqué**, no el qué. El de `pumpAndSettle` y el de `tester.runAsync` ahorran horas a quien los lea.
- **La CI compila cuatro plataformas**, no solo la de desarrollo.
- **El validador estructural es un gate propio y barato** que corre offline y en menos de un segundo.
- **La degradación está declarada, no oculta**: `generate_sbom.py` marca su cobertura real y el workflow explica por qué omite la prueba de integración.

Continúa en 13 · Despliegue y operación.

13 · Despliegue y operación

Qué se despliega, exactamente

Este proyecto produce **tres artefactos distintos**, y solo uno de ellos se despliega automáticamente.

Artefacto	Cómo se publica	Automático
APK de Android	GitHub Releases, con `checksums.txt`	**No**: proceso manual
Landing + PWA	GitHub Pages	**Sí**: `deploy-landing.yml`
Documentación	El propio repositorio	No aplica

No hay servidor que operar, no hay base de datos que administrar y no hay secretos de producción que rotar: **los dos workflows no consumen ningún secreto de GitHub**.

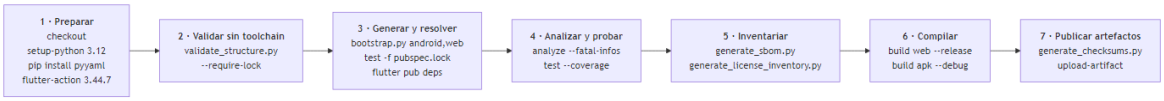
---

1 · Integración continua

`.github/workflows/flutter-ci.yml`

Aspecto	Valor
Nombre	`Flutter quality and compatibility`
Disparadores	`push` a `main` y `develop`, cualquier `pull_request`, `workflow_dispatch`
Permisos	`contents: read`
Variable	`FLUTTER_VERSION: 3.44.7`
Trabajos	`quality` (ubuntu-latest) y `apple` (macos-latest)

Trabajo `quality`



Lo que muestra: el orden real de los pasos y que la validación estructural va **antes** de generar plataformas, para fallar rápido y sin toolchain.

Lo que no muestra: que `bootstrap.py` es imprescindible aquí porque `android/` y `web/` no están versionados. Sin ese paso, `flutter build` no tendría a qué apuntar. Tampoco muestra que el APK que se compila es **de depuración**: los APK de release se compilan a mano al publicar.

Artefactos subidos: el APK de depuración, `build/web`, el SBOM, el inventario de licencias, `build/checksums.txt`, `coverage/lcov.info`, `pubspec.lock` y `test_assets/manifest.json`.

Trabajo `apple`

Tres pasos reales: `bootstrap.py --platforms ios,macos`, `flutter build ios --release --no-codesign` y `flutter build macos --release`.

La prueba de integración **se omite a propósito**, con el motivo escrito como comentario en el propio archivo: los runners macOS de GitHub no tienen sesión gráfica interactiva, así que la aplicación arranca sin llegar a establecer la conexión de depuración. Para ejecutarla en local:

```
flutter test integration_test/app_launch_test.dart -d macos
```

Es un ejemplo de **degradación declarada en vez de silenciosa**, y conviene conservarlo.

---

2 · Publicación de la landing y la PWA

`.github/workflows/deploy-landing.yml`

Aspecto	Valor
Disparadores	`push` a `main` **filtrado por rutas**, más `workflow_dispatch`
Permisos	`contents: read`, `pages: write`, `id-token: write`
Concurrencia	Grupo `pages`, sin cancelar en curso
Tiempo máximo	30 minutos
`APP_BASE_HREF`	`/universal-code-scanner/app/`

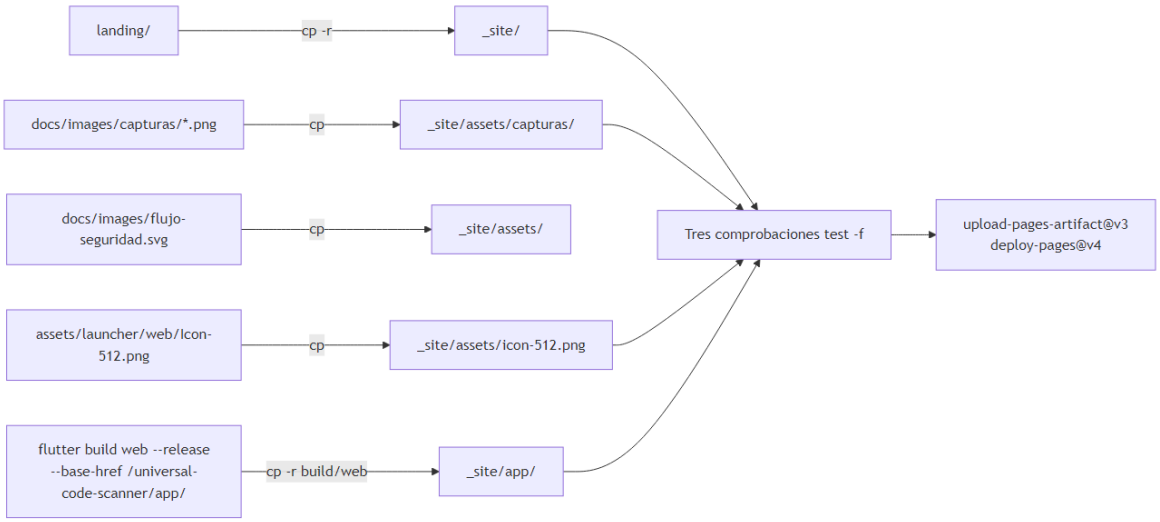
Rutas que lo disparan: `landing/**`, `lib/**`, `web/**`, `assets/**`, `docs/images/**`, `pubspec.yaml`, `pubspec.lock`, `tool/bootstrap.py` y el propio workflow.

Hallazgo: el filtro `web/**` **no puede dispararse nunca**, porque `/web/` está en `.gitignore` y no hay ningún archivo versionado bajo esa ruta. Es un filtro inerte. R-12.

Consecuencia práctica de esta lista: un commit que solo toque `docs/system-documentation/**` o `README.md` **no dispara la publicación de la landing**; solo `flutter-ci.yml`. Pero basta con que toque un archivo de `lib/` —aunque el cambio sea únicamente un comentario— para que el filtro `lib/**` coincida y la PWA se vuelva a compilar y a publicar.

Es exactamente lo que ocurrió con el commit que introdujo esta documentación: además de los veinte Markdown, añadió comentarios a 24 archivos de `lib/`, y eso disparó **los dos** workflows. Conviene tenerlo presente antes de suponer que un cambio «solo de documentación» no toca la publicación.

Cómo se monta el sitio



Lo que muestra: que las imágenes **no se duplican** en `landing/`. Viven una sola vez en el repositorio y se copian al artefacto durante la publicación.

Lo que no muestra: la consecuencia de esa decisión, que sí explica `../../landing/README.md`: al abrir `landing/index.html` directamente desde el disco, **las capturas, el diagrama y el enlace a `/app/` aparecen rotos**. Para verlo en local hay que reproducir el montaje, y ese archivo trae el guion exacto.

Las tres comprobaciones `test -f` al final del paso de montaje son un gate real: si falta `_site/app/index.html`, `_site/assets/capturas/escanear.png` o `_site/assets/icon-512.png`, el workflow falla antes de publicar. Es lo que impide publicar un sitio a medias.

Direcciones publicadas

Ruta	Contenido
`https://vladimiracunadev-create.github.io/universal-code-scanner/`	La landing
`.../universal-code-scanner/app/`	La PWA en release

`touch _site/.nojekyll` desactiva el procesado de Jekyll, necesario porque el build de Flutter genera carpetas que empiezan por guion bajo.

---

3 · Publicación de un release de Android

Proceso manual. No hay workflow que lo haga. La lista completa de lo que haría falta para llegar a las tiendas está en `../RELEASE.md` y **está fuera del alcance de 1.1.0**.

El procedimiento que se siguió para 1.1.0, reconstruido desde `../../VALIDATION.md` y `RELEASE_SOURCE.json`:

```
# 1 · Puerta de estabilidad completa
./tool/finalize_stable.sh

# 2 · Los cuatro APK
flutter build apk --release --split-per-abi
flutter build apk --release

# 3 · Checksums
python3 tool/generate_checksums.py \
 build/app/outputs/flutter-apk/*.apk > checksums.txt
```

```
# 4 · Manifiesto de fuente
python3 tool/generate_source_manifest.py

# 5 · Publicación
gh release create v1.1.0 build/app/outputs/flutter-apk/*.apk checksums.txt
```

`tool/finalize_stable.sh` encadena: comprobar que `flutter` existe, `flutter --version`, `pub get`, comprobar `pubspec.lock`, `analyze`, `test`, `build apk --debug`, `build web --release`, `validate_structure.py --require-lock`, SBOM, inventario de licencias, manifiesto de fuente, y termina con un `git diff -- pubspec.lock pubspec.yaml` y el recordatorio de revisar y confirmar el lockfile antes de etiquetar.

Artefactos de 1.1.0 y sus tamaños

Archivo	Para quién	Tamaño
`app-arm64-v8a-release.apk`	Prácticamente cualquier móvil actual	32,8 MB
`app-armeabi-v7a-release.apk`	Móviles antiguos de 32 bits	26,8 MB
`app-x86_64-release.apk`	Emuladores y equipos x86	35,3 MB
`app-release.apk`	Universal	88,1 MB

Valores tomados de `RELEASE_SOURCE.json`, campo `runtimeValidation.androidReleaseApk`.

La advertencia de firma

Los cuatro APK van firmados con la clave de depuración de Android (`CN=Android Debug`). Sirven para instalar y probar. **No** sirven para Google Play, y una versión futura firmada con una clave propia **no podrá actualizarlos**: habrá que desinstalar antes.

Está declarado en `README.md`, en `docs/RELEASE.md` y en `IMPLEMENTATION_STATUS.md`. La verificación real de un artefacto descargado es su **SHA-256**, publicado en `checksums.txt`, no la firma.

---

4 · Generación de los PDF de esta documentación

`tool/build_system_documentation_pdf.py` produce un PDF por documento más uno consolidado, todos en `docs/system-documentation/pdf/`. **El Markdown es la fuente única**: el script no mantiene una segunda versión del texto.

Uso

```
# Todos los documentos más el consolidado
python tool/build_system_documentation_pdf.py

# Solo uno, para iterar rápido sobre el formato
python tool/build_system_documentation_pdf.py --only 03-architecture.md

# Sin el consolidado
python tool/build_system_documentation_pdf.py --skip-combined
```

Dependencias

Dependencia	Obligatoria	Si falta

`markdown`	**Sí**	El script se detiene con la instrucción de instalación
`xhtml2pdf`	**Sí**	Idem
`mmdc` (`@mermaid-js/mermaid-cli`)	No	Los diagramas se incluyen como bloque de texto monoespaciado y **se avisa por consola**

```
python -m pip install markdown xhtml2pdf
npm install -g @mermaid-js/mermaid-cli   # opcional, para los diagramas
```

Los diagramas se rasterizan a PNG y **se cachean por hash de su código fuente** en `docs/system-documentation/pdf/.mermaid-cache/`, así que una segunda ejecución solo rehace lo que cambió. Esa carpeta está en `.gitignore`.

Qué lee del repositorio

Dato de la portada	De dónde sale
Versión	`version:` de `pubspec.yaml`
Commit	`git rev-parse --short HEAD`
Fecha	`date.today()`

Ninguno se escribe a mano. Una portada con números manuales queda obsoleta en el primer commit.

Salida

Al terminar informa cuántos diagramas se renderizaron y cuántos degradaron a texto, y falla con código distinto de cero si algún PDF quedó de 0 bytes. La degradación **nunca es silenciosa**.

---

5 · Otras herramientas de operación

Guion	Cuándo se ejecuta	Qué produce
`tool/bootstrap.py`	Tras clonar y en ambos workflows	`android/`, `ios/`, `web/`, `macos/` parcheados
`tool/validate_structure.py`	CI, puerta de calidad, publicación	Diez comprobaciones offline
`tool/run_quality_gate.sh`	Manual, antes de un commit grande	Validación, análisis, pruebas, SBOM
`tool/finalize_stable.sh`	Antes de etiquetar	Todo lo anterior más compilaciones y manifiesto
`tool/generate_sbom.py`	CI y publicación	`build/reports/sbom.cdx.json`
`tool/generate_license_inventory.py`	CI y publicación	`build/reports/dependency-licenses.json`
`tool/generate_source_manifest.py`	Publicación	`SOURCE_MANIFEST.json`
`tool/generate_checksums.py`	Publicación	SHA-256 de los artefactos

`tool/generate_launcher_icons.py`	Solo al cambiar la identidad visual	Los PNG de `assets/launcher/`
`tool/generate_scan_beep.py`	Solo al cambiar el tono	`assets/sounds/scan_success.wav`

Los dos últimos son **regeneradores de recursos versionados**: su salida está en el repositorio, así que compilar no los necesita. `generate_launcher_icons.py` requiere Pillow; `generate_scan_beep.py` usa solo la biblioteca estándar.

`generate_source_manifest.py` **debe ejecutarse al final de cualquier cambio que añada o modifique archivos versionados**, incluido este trabajo de documentación: `SOURCE_MANIFEST.json` guarda el tamaño y el SHA-256 de cada fuente, y quedaría desactualizado.

---

6 · Mantenimiento

Dependencias

`.github/dependabot.yml` abre hasta 5 PR semanales para `pub` y actualiza las acciones de GitHub mensualmente. Ignora explícitamente:

Paquete	Versiones ignoradas	Motivo escrito en el propio archivo
`share_plus`	`>=13.0.0`	Exige `win32 ^6`, incompatible con el `win32 ^5.9` de `file_picker 11.x`
`pdfrx`	`>=2.4.6`	Arrastra `archive ^4`, incompatible con el `archive ^3.6.1` de `excel 4.0.6`

Antes de levantar cualquiera de esos dos topes hay que leer `../quality/LOCKFILE.md`. Subirlos sin resolver antes el conflicto rompe `pub get` por completo.

El punto 14 del contrato de compatibilidad exige actualizar las dependencias **de forma individual**, con pruebas de regresión y posibilidad de reversión.

Versión

Subir la versión implica **dos archivos a la vez**:

1. `pubspec.yaml`, campo `version`, con la forma `X.Y.Z+N`;
2. `lib/core/app_info.dart`, constante `appVersion`, con la parte `X.Y.Z`.

`tool/validate_structure.py::check_interface_version` **falla la validación si divergen**. Esa comprobación nació de un defecto real: la pantalla «Acerca de» mostraba `1.0.0` en la versión `1.1.0` porque el número estaba escrito como literal en `settings_screen.dart` y nadie lo subió al publicar.

Cadena de compilación de Android

`tool/bootstrap.py` fija AGP `8.11.1`, Kotlin `2.2.20` y Gradle `8.14.3`. El motivo está escrito como comentario en el script y explicado en `../quality/LOCKFILE.md`. Revisar cuando los plugins del conjunto publiquen versiones compatibles con el Kotlin integrado de AGP 9.

---

7 · Reversión

Situación	Cómo se revierte	Riesgo
Un commit rompe la CI	`git revert <sha>` y push	Bajo
La landing publicada está rota	Revertir el commit y volver a lanzar `deploy-landing.yml` a mano	Bajo. La versión anterior sigue servida hasta que termine el despliegue nuevo
Un APK publicado tiene un fallo	Publicar un release nuevo. **No se puede retirar un APK ya descargado**	Medio
Una dependencia rompe la resolución	`git checkout -- pubspec.yaml pubspec.lock` y `flutter pub get`	Bajo
Un cambio del formato de cifrado deja datos ilegibles	**No hay reversión automática.** Los datos ya reescritos con el formato nuevo no vuelven solos	**Crítico**

La última fila es la que importa. Un cambio en `PayloadCipher.currentVersion`, `currentAlgorithm`, `currentKeyId` o `SecureStorageKeyProvider._prefix` **no es reversible** una vez que la aplicación ha reescrito registros: la actualización perezosa de sobres los habrá convertido al formato nuevo en la primera lectura.

Por eso el contrato de compatibilidad dedica sus ocho primeros puntos a los datos del usuario, y por eso la rotación de llave verifica y limpia. Antes de tocar esas cuatro constantes hay que leer `../quality/COMPATIBILITY_CONTRACT.md` y 07-database.md.

---

8 · Operación en el dispositivo

No hay servidor, así que «operar» significa lo que el usuario puede hacer cuando algo va mal. Las cuatro herramientas de recuperación:

Herramienta	Dónde	Para qué
Inicio seguro	Automático si el arranque falla	Reintentar, abrir en modo temporal, restablecer preferencias visuales, copiar diagnóstico
Modo temporal	Desde el inicio seguro	Usar la aplicación con base en memoria, sin tocar los datos persistentes
Centro de recuperación	Ajustes → Datos y compatibilidad	Reintentar el descifrado, descartar un registro, marcar revisado, exportar el paquete
Rotar llave de cifrado	Ajustes → Datos y compatibilidad	Recifrar todo con una llave nueva

El diagnóstico privado contiene **solo** tipo de error, área y huella de pila; el texto bajo el botón lo enumera literalmente. El paquete de recuperación contiene los sobres **todavía cifrados** y ninguna llave.

Detalle de cada síntoma en 14-troubleshooting.md.

---

9 · Lista de comprobación antes de publicar

Reconstruida de `tool/finalize_stable.sh`, `docs/RELEASE.md` y la práctica observable en `VALIDATION.md`:

- [ ] `flutter pub get` resuelve y `pubspec.lock` no cambia inesperadamente

- [ ] `python tool/validate_structure.py --require-lock` sin errores
- [ ] `flutter analyze --fatal-infos` sin hallazgos
- [ ] `flutter test` en verde
- [ ] `flutter build web --release` compila
- [ ] `flutter build apk --release --split-per-abi` y `--release` compilan
- [ ] La versión de `pubspec.yaml` y la de `app_info.dart` coinciden
- [ ] `CHANGELOG.md` describe los cambios de la versión
- [ ] `VALIDATION.md` y `RELEASE_SOURCE.json` reflejan lo verificado de verdad
- [ ] `python tool/generate_source_manifest.py` regenerado y confirmado
- [ ] `python tool/build_system_documentation_pdf.py` regenerado si la documentación cambió
- [ ] Checksums generados y publicados junto a los artefactos
- [ ] El tono de lectura está **dentro** del APK, no solo en el repositorio
- [ ] El APK descargado del release coincide con su SHA-256 publicado

Las dos últimas no son teóricas: se comprobaron para 1.1.0 y están registradas en `VALIDATION.md`. Verificar que un recurso empaquetado llegó de verdad al artefacto es el tipo de comprobación que separa un release correcto de uno que compila y sale vacío.

`docs/RELEASE.md` es un documento de proceso del propio proyecto. **Se propone** añadirle una línea para regenerar los PDF de esta documentación antes de etiquetar; no se editó en este trabajo por ser un documento de decisión del usuario.

Continúa en 14 · Solución de problemas.

14 · Solución de problemas

Cada entrada sigue la misma estructura: **síntoma** → **causa** → **diagnóstico** → **solución** → **archivos** → **riesgo de aplicar la solución**.

Las entradas de la sección 1 son para quien usa la aplicación; las de la 2 y la 3, para quien la desarrolla o la publica.

---

1 · Problemas de uso

1.1 · «Escaneando» y la barra se mueve, pero el código no entra

Aspecto	Detalle
Síntoma	La barra de estado dice «Escaneando» y se mueve, la imagen se ve, y ningún código se lee
Causa	El motor **sí** está analizando cuadros: la barra en movimiento lo demuestra. El problema es el encuadre, la distancia, el enfoque o la luz. Con el marco activado, solo se analiza el cuadrado central del 68 %
Diagnóstico	Si la barra se mueve, descarta un fallo de arranque. Prueba a acercar o alejar el teléfono y a usar el deslizador de zoom
Solución	Acercar o alejar, usar el zoom, encender la linterna, tocar la imagen para enfocar. Si sigue sin entrar, desactivar **Marco de lectura real** en Ajustes para analizar toda la imagen
Archivos	`lib/features/scanner/scanner_screen.dart` (cálculo de `window`), `lib/models/app_settings.dart` (`useScanWindow`)
Riesgo	Ninguno. Desactivar el marco solo aumenta el trabajo por cuadro; puede reducir la velocidad de detección en teléfonos modestos

1.2 · «Escaneando» y ningún código de ningún tipo entra

Aspecto	Detalle
Síntoma	Con el marco activado, ni siquiera un QR grande y bien iluminado se lee
Causa	El recorte de lectura podría no coincidir con lo que se ve. La ventana se calcula desde el tamaño de la cámara y la orientación, datos que pueden llegar después del primer cuadro
Diagnóstico	Desactivar **Marco de lectura real**. Si con la imagen completa sí lee, era la ventana
Solución	Pulsar **Reiniciar cámara**, que reconstruye el controlador y la vista previa con una clave nueva. O desactivar el marco
Archivos	`scanner_screen.dart`, método `_restartCamera`
Riesgo	Ninguno. `_restartCamera` libera el controlador antiguo antes de crear el nuevo y reinicia el zoom a 0

Este es exactamente el defecto que originó la versión 1.1.0. La causa técnica —un umbral de actualización de la ventana fijado en `4` sobre una magnitud que se expresa entre 0 y 1, es decir, «no recalcular nunca»— está en `../quality/SCANNER_UX.md`. Ya está corregida: el umbral se eliminó.

1.3 · «Iniciando cámara…» que no avanza

Aspecto	Detalle
Síntoma	La barra dice «Iniciando cámara…» y se queda ahí
Causa	`MobileScannerState.isRunning` sigue en `false`: la cámara no llegó a entregar imagen. Suele ser otra aplicación ocupando el s ensor
Diagnóstico	La fase `starting` significa que no hubo error, solo ausencia de cuadros. Si hubiera error, la fase sería `unavailable`
Solución	**Reiniciar cámara**. Si persiste, cerrar otras aplicaciones que la u sen
Archivos	`scanner_screen.dart`, método `_phaseFor`
Riesgo	Ninguno

1.4 · «Cámara no disponible»

Aspecto	Detalle
Síntoma	Barra en color de error, con un motivo concreto debajo
Causa	Tres posibles, y el mensaje las distingue: falta el permiso (`permi ssionDenied`), el dispositivo no lo admite (`unsupported`), o cual quier otro fallo de arranque
Diagnóstico	Leer el mensaje. «Falta el permiso de cámara» es inequívoco
Solución	Conceder el permiso en los ajustes del sistema y pulsar **Reinten tar**. Si dice que el dispositivo no lo permite, usar la importación d esde galería
Archivos	`scanner_screen.dart`, método `_describe`; permisos en `tool/b ootstrap.py`
Riesgo	Ninguno

1.5 · Lee, pero no suena

Aspecto	Detalle
Síntoma	El código se lee y aparece el resultado, pero no hay tono
Causa	Tres posibles: el ajuste **Sonido de confirmación** está apagado; el volumen multimedia está a cero; o el reproductor de audio fall ó y `ScanFeedback` degradó al sonido del sistema, que Android sil encia si los sonidos táctiles están apagados
Diagnóstico	Revisar el ajuste y el volumen multimedia. La tercera causa no e s observable desde la interfaz

Solución	Activar **Sonido** en Ajustes y subir el volumen multimedia
Archivos	`lib/services/scan_feedback.dart`; el recurso es `assets/sounds/scan_success.wav`
Riesgo	Ninguno

El tono es un **recurso propio empaquetado**, no el efecto de sonido del sistema. El motivo está en `../quality/SCANNER_UX.md`: `SystemSound.play` se traduce al efecto de la interfaz, que Android silencia en cuanto se apagan los sonidos táctiles, que es el ajuste habitual.

1.6 · «Inicio seguro» al abrir la aplicación

Aspecto	Detalle
Síntoma	En vez de la aplicación aparece una tarjeta con «Inicio seguro», un diagnóstico y cuatro botones
Causa	`AppBootstrapper.initialize` lanzó. Puede ser la apertura de la base, la migración de esquema, la lectura de la llave activa o la inicialización de un store
Diagnóstico	El texto muestra `errorType` y una huella de pila. Copiar el diagnóstico privado da hasta 100 entradas con área y tipo, **sin ningún dato escaneado**
Solución	En orden: **Reintentar**; si falla, **Restablecer preferencias visuales** y reintentar; si sigue fallando, **Abrir sin historial ni inventarios persistentes**
Archivos	`lib/bootstrap.dart`, `lib/bootstrap_host.dart`, `lib/core/diagnostics/`
Riesgo	**Reintentar** y **modo temporal** no tocan datos. **Restablecer preferencias visuales** borra 14 de las 17 claves de configuración y **no se puede deshacer**; deja intactas `save_history`, `private_mode` y `biometric_lock`

1.7 · Banner permanente de «Modo temporal»

Aspecto	Detalle
Síntoma	Un aviso fijo sobre las pestañas: «el historial y los inventarios se eliminarán al cerrar la aplicación»
Causa	Se arrancó desde el inicio seguro con **Abrir sin historial ni inventarios persistentes**. La base es de memoria y la llave también
Diagnóstico	Ajustes muestra además «Modo temporal activo», y **Rotar llave de cifrado** aparece deshabilitado
Solución	Cerrar y volver a abrir la aplicación para intentar el arranque normal
Archivos	`bootstrap.dart` (rama `temporary`), `app.dart` (el banner)
Riesgo	**Todo lo escaneado o inventariado en modo temporal se pierde al cerrar.** Exportarlo antes si interesa conservarlo

1.8 · Faltan registros en el historial

Aspecto	Detalle
Síntoma	Se escanearon códigos y no aparecen en Historial
Causa	Cinco posibles: **Guardar historial** apagado; **Sesión privada** acti va; el contenido era **sensible** (OTP, Wi-Fi con contraseña, pago, AAMVA, cripto); la **retención** los podó; o se superaron los 5 00 0 registros y se recortaron los más antiguos
Diagnóstico	Revisar los dos interruptores y el valor de retención en Ajustes. Un registro sensible se muestra al leerlo pero nunca se guarda
Solución	Ajustar las preferencias. Lo ya no guardado **no se puede recup erar**: nunca se escribió
Archivos	`scanner_screen.dart` (`_persistAndShow`), `history_reposito ry.dart` (`pruneOlderThan`, `_trim`)
Riesgo	Ninguno al cambiar los ajustes; afectan solo a lecturas futuras

El caso del contenido sensible es **deliberado y está documentado** en `docs/SECURITY.md` y en la política de privacidad. El del inventario es distinto: ahí sí se guarda. Ver el hallazgo R-03 de 15-risks-and-technical-debt.md.

1.9 · Aparecen elementos en el Centro de recuperación

Aspecto	Detalle
Síntoma	Ajustes → Centro de recuperación muestra incidencias con un c ódigo como `decrypt_StateError` o `missing_payload`
Causa	Al cargar, un registro no se pudo descifrar o no tenía carga. La c ausa más frecuente es que **la llave ya no está** en el almacén se guro: reinstalación, restauración del sistema o cambio de disposi tivo
Diagnóstico	El código lo dice. `decrypt_StateError` corresponde casi siempr e a `encryption_key_missing`. `missing_payload` es un registro sin `payload`, más raro
Solución	**Reintentar recuperación** si aparece la opción; funciona si la lla ve volvió. Si no, **Descartar registro afectado** elimina solo ese r egistro. **Marcar revisado** lo oculta sin borrar nada
Archivos	`lib/core/recovery/`, `lib/services/history_repository.da rt`, `lib/services/inventory_repository.dart`
Riesgo	**Descartar borra ese registro de forma definitiva.** Marcar revis ado no borra nada, pero si el problema se repite la incidencia **vu elve a aparecer**: su identificador es determinista

Los registros dañados **nunca se borran automáticamente**, y una llave ausente **nunca se reemplaza en silencio**: el descifrado informa y **bloquea nuevas escrituras** con ese identificador hasta que la persona recupere, rote o descarte. Es la propiedad descrita en `../quality/RECOVERY.md`.

1.10 · Ajustes ofrece «Reintentar migración del historial»

Aspecto	Detalle
Síntoma	Una fila extra en Ajustes con un código como `legacy_FormatException`
Causa	La conversión del historial guardado en preferencias a la base cifrada falló
Diagnóstico	El subtítulo muestra el código. La fila solo aparece si `migrationStatus.completed == false`
Solución	Pulsarla. El mensaje dirá si se completó o si sigue pendiente
Archivos	`history_repository.dart`, método `_migrateLegacyHistory`
Riesgo	**Ninguno.** El origen se conserva intacto y hay un respaldo cifrado. La conversión no se marca completada hasta que se verifica dentro de la transacción

1.11 · La rotación de llave falló

Aspecto	Detalle
Síntoma	«No se modificaron los datos porque la rotación no pudo completarse»
Causa	Un registro sin `payload`, un fallo de descifrado con la llave actual, o un error de la transacción
Diagnóstico	El mensaje es genérico a propósito. Revisar el Centro de recuperación por si aparecieron incidencias
Solución	Resolver primero las incidencias del Centro de recuperación y volver a intentarlo
Archivos	`lib/core/security/data_maintenance_service.dart`
Riesgo	**Ninguno.** Es una operación todo o nada: registros y metadatos se escriben en una única transacción, y la llave nueva se borra del almacén seguro si algo falla. Hay prueba dedicada

1.12 · «No existe una aplicación compatible para esta acción»

Aspecto	Detalle
Síntoma	Al pulsar «Abrir», «Llamar», «Correo», «SMS» o «Mapa» aparece ese aviso
Causa	`launchUrl` devolvió `false`: el sistema no encontró ninguna aplicación que atienda ese esquema
Diagnóstico	Frecuente en emuladores y en macOS sin cliente de correo configurado
Solución	Instalar o configurar una aplicación para ese esquema. Alternativa: **Copiar** y pegarlo donde haga falta
Archivos	`scan_result_sheet.dart`, método `_openRecord`

Riesgo	Ninguno

1.13 · El contenido copiado desapareció del portapapeles

Aspecto	Detalle
Síntoma	Se copió algo y al pegarlo más tarde no está
Causa	Es el comportamiento previsto. **Borrar portapapeles** vale 30 se gundos por defecto
Diagnóstico	El mensaje al copiar lo dice: «se borrará en N segundos»
Solución	Ajustes → **Borrar portapapeles** → `Nunca`, o un valor mayor
Archivos	`lib/services/clipboard_service.dart`
Riesgo	Poner `Nunca` deja información posiblemente sensible en el porta papeles indefinidamente

El borrado **comprueba antes** que el portapapeles siga conteniendo el mismo valor: si se copió otra cosa entre medias, no se borra nada.

1.14 · El QR generado no se lee al escanearlo

Aspecto	Detalle
Síntoma	Se genera un código y otro lector no lo interpreta
Causa	El contenido no es válido para el formato elegido —letras en un EAN-13, longitud incorrecta en un UPC-A— o la corrección de er rores es insuficiente
Diagnóstico	Si el contenido es inválido, la vista previa muestra «El contenido no es válido para \<formato>» con el error concreto
Solución	Elegir un formato compatible (QR o Code 128 admiten casi cual quier texto) y activar **Corrección de errores alta**
Archivos	`lib/features/generator/generator_screen.dart`
Riesgo	Ninguno: el generador no persiste nada

1.15 · El Excel exportado muestra los acentos rotos

Aspecto	Detalle
Síntoma	Un CSV abierto en Excel muestra caracteres extraños en lugar d e acentos
Causa	**No debería ocurrir**: `_encodeCsv` antepone el BOM UTF-8. Si oc urre, la aplicación que abre el archivo lo está ignorando
Diagnóstico	Abrir el CSV en un editor de texto: si los acentos se ven bien, el archivo es correcto

Solución	Usar la exportación **XLSX** en lugar de CSV, o importar el CSV in dicando UTF-8 explícitamente
Archivos	`lib/services/export_service.dart`, función `_encodeCsv`
Riesgo	Ninguno

El XLSX escribe **todas las celdas como texto** (`TextCellValue`), así que las cantidades aparecerán como texto, no como números.

---

2 · Problemas de desarrollo

2.1 · `flutter run` no encuentra nada que ejecutar

Aspecto	Detalle
Síntoma	Tras clonar, `flutter run` falla o no ofrece dispositivos
Causa	`android/`, `ios/`, `web/` y `macos/` **no están versionados**
Diagnóstico	`cat .gitignore` muestra las cuatro rutas bajo el comentario «S e generan con tool/bootstrap.py»
Solución	`python tool/bootstrap.py --platforms android,web`
Archivos	`.gitignore`, `tool/bootstrap.py`
Riesgo	Ninguno sin `--force`. **Con `--force` se borran y regeneran** las carpetas, perdiendo cualquier cambio manual hecho en ellas

2.2 · `cannot find symbol: class FilePickerPlugin`

Aspecto	Detalle
Síntoma	`flutter build apk` falla con ese error de Java
Causa	Se está usando AGP 9. Varios plugins —`file_picker` entre ello s— dejan de aplicar el Kotlin Gradle Plugin al detectar AGP 9, d ando por hecho que el Kotlin integrado está activo; pero la plantil la entrega `android.builtInKotlin=false`, así que nadie compi la su Kotlin
Diagnóstico	Comprobar la versión en `android/settings.gradle.kts`; debe ser `8.11.1`
Solución	Volver a ejecutar `python tool/bootstrap.py`, que reescribe A GP a `8.11.1`, Kotlin a `2.2.20` y Gradle a `8.14.3`
Archivos	`tool/bootstrap.py`, función `patch_android_toolchain`
Riesgo	Ninguno. Activar el Kotlin integrado en su lugar **rompe el caso c ontrario**, en los plugins que sí aplican el plugin de Kotlin

2.3 · `pub get` falla al resolver

Aspecto	Detalle
Síntoma	Conflicto de versiones, casi siempre alrededor de `win32` o `archive`
Causa	Se subió `share_plus`, `pdfrx` o `excel` por encima de su tope. `share_plus >= 13` exige `win32 ^6`, incompatible con el `^5.9` de `file_picker 11.x`; `pdfrx >= 2.4.6` arrastra `archive ^4`, incompatible con el `^3.6.1` de `excel 4.0.6`
Diagnóstico	`git diff pubspec.yaml`
Solución	Revertir esas versiones a `12.0.2`, `2.4.5` y `4.0.6`
Archivos	`pubspec.yaml`, `.github/dependabot.yml`, `../quality/LOCKFILE.md`
Riesgo	Ninguno al revertir

2.4 · `ModuleNotFoundError: No module named 'yaml'`

Aspecto	Detalle
Síntoma	`validate_structure.py` o `generate_sbom.py` fallan al importar
Causa	Falta `pyyaml`
Diagnóstico	`python -c "import yaml"`
Solución	`python -m pip install --upgrade pyyaml`
Archivos	`tool/validate_structure.py` línea 11, `tool/generate_sbom.py`
Riesgo	Ninguno. `tool/bootstrap.py` **no** necesita `yaml`

2.5 · `La versión mostrada en la interfaz no coincide con pubspec.yaml`

Aspecto	Detalle
Síntoma	`validate_structure.py` falla con ese mensaje
Causa	Se cambió `version` en `pubspec.yaml` sin cambiar `appVersion` en `lib/core/app_info.dart`, o al revés
Diagnóstico	El mensaje muestra ambos valores
Solución	Igualar `appVersion` a la parte `X.Y.Z` de `version`
Archivos	`lib/core/app_info.dart`, `pubspec.yaml`
Riesgo	Ninguno. Este gate existe porque la aplicación llegó a mostrar `1.0.0` siendo `1.1.0`

2.6 · `Ruta absoluta del entorno encontrada en …`

Aspecto	Detalle

Síntoma	`validate_structure.py` falla señalando un archivo
Causa	Ese archivo contiene una de las tres rutas de entorno que prohíb e la variable `bad` de `check_absolute_paths`: dos directorios de t rabajo de entornos remotos y el prefijo del directorio de perfiles d e Windows. Suele ser una traza pegada desde una máquina con creta
Diagnóstico	El mensaje da la ruta del archivo
Solución	Sustituir la ruta absoluta por una relativa o por un marcador gen érico
Archivos	`tool/validate_structure.py`, función `check_absolute_path s`
Riesgo	Ninguno

Afecta también a **esta documentación**: no se pueden citar rutas absolutas de la máquina de análisis.

2.7 · `Enlace Markdown local inexistente en …`

Aspecto	Detalle
Síntoma	`validate_structure.py` falla señalando un `.md` y un destino
Causa	Un enlace relativo apunta a un archivo que no existe
Diagnóstico	El mensaje da el archivo y el destino exactos
Solución	Corregir el enlace
Archivos	`tool/validate_structure.py`, función `check_markdown_link s`
Riesgo	Ninguno

Este gate valida **toda** la documentación del repositorio, incluida la de `docs/system-documentation/`. Es el verificador de enlaces del proyecto.

2.8 · Una prueba de widget se cuelga

Aspecto	Detalle
Síntoma	`flutter test` no termina, o tarda minutos en un solo caso
Causa	Dos posibles: `pumpAndSettle` sobre una animación infinita —la barra indeterminada nunca para—, o abrir la base de datos dentr o del reloj falso de `testWidgets`, que usa temporizadores reales
Diagnóstico	Comentar el caso sospechoso y volver a ejecutar
Solución	Usar `await tester.pump()` en lugar de `pumpAndSettle`, y envo lver el trabajo con temporizadores reales en `tester.runAsync(. ..)`
Archivos	`test/features/scan_status_bar_test.dart` y `test/core/ac cessibility_test.dart` traen el patrón correcto comentado

Riesgo	Ninguno

Ambos casos son defectos reales ya corregidos, registrados en `../../VALIDATION.md`.

2.9 · La CI falla en un paso que pasa en local

Aspecto	Detalle
Síntoma	`flutter analyze` en verde localmente, `analyze` en rojo en la CI
Causa	`tool/run_quality_gate.sh` ejecuta `flutter analyze` **sin** `--fatal-infos`; la CI lo ejecuta **con**
Diagnóstico	Ejecutar `flutter analyze --fatal-infos` en local
Solución	Corregir también los avisos informativos
Archivos	`tool/run_quality_gate.sh`, `.github/workflows/flutter-ci.yml`
Riesgo	Ninguno

Registrado como R-06.

---

3 · Problemas de publicación

3.1 · La landing publicada no se actualiza

Aspecto	Detalle
Síntoma	Se hizo push a `main` y GitHub Pages sigue igual
Causa	`deploy-landing.yml` solo se dispara si el push toca `landing/**`, `lib/**`, `web/**`, `assets/**`, `docs/images/**`, `pubspec.yaml`, `pubspec.lock`, `tool/bootstrap.py` o el propio workflow. Un cambio en `docs/system-documentation/` o en `README.md` **no lo dispara**
Diagnóstico	Comparar los archivos del commit con la lista `paths:`
Solución	Lanzarlo a mano con `workflow_dispatch` desde la pestaña Actions
Archivos	`.github/workflows/deploy-landing.yml`
Riesgo	Ninguno. La concurrencia usa el grupo `pages` sin cancelar en curso

3.2 · Las capturas de la landing salen rotas en local

Aspecto	Detalle
Síntoma	Al abrir `landing/index.html` desde el disco, las imágenes y `/app/` no cargan

Causa	Es el comportamiento previsto. Las imágenes **no se duplican** e n `landing/`: se copian al artefacto durante la publicación
Diagnóstico	`ls landing/assets/` muestra solo `favicon.svg` y `style.css`
Solución	Reproducir el montaje. El guion exacto está en `../../landing/README.md`
Archivos	`landing/`, `.github/workflows/deploy-landing.yml`
Riesgo	Ninguno

3.3 · El SHA-256 del APK descargado no coincide

Aspecto	Detalle
Síntoma	El hash del archivo descargado difiere del de `checksums.txt`
Causa	Descarga incompleta o corrupta. **O un artefacto que no es el p ublicado**
Diagnóstico	`python tool/generate_checksums.py <archivo>` y comparar
Solución	Volver a descargar. Si vuelve a no coincidir, **no instalar** y report arlo
Archivos	`tool/generate_checksums.py`
Riesgo	Instalar un artefacto cuyo hash no coincide es exactamente lo qu e la publicación de checksums existe para evitar

3.4 · Android no deja instalar el APK

Aspecto	Detalle
Síntoma	«Aplicación no instalada» o un aviso de origen desconocido
Causa	Dos posibles: falta permitir la instalación desde esa fuente, o **ya hay instalada una versión firmada con otra clave**
Diagnóstico	El mensaje de conflicto de firma es explícito
Solución	Permitir la instalación desde esa fuente. Si es conflicto de firma, **desinstalar la versión anterior**
Archivos	Declarado en `README.md` y en `docs/RELEASE.md`
Riesgo	**Desinstalar borra el historial y los inventarios de forma defi nitiva.** Están cifrados con una llave que también se borra, y no h ay copia remota porque no hay servidor. **Exportar antes**

3.5 · `SOURCE_MANIFEST.json` no cuadra tras clonar en Windows

Aspecto	Detalle
Síntoma	Los SHA-256 de archivos de texto no coinciden con los del mani fiesto

Causa	Finales de línea. El manifiesto se calcula sobre bytes, y CRLF pr oduce otro hash
Diagnóstico	Comprobar que `.gitattributes` está presente y que Git lo resp etó al clonar
Solución	Volver a clonar con `.gitattributes` en vigor. Es lo que `* text =auto eol=lf` garantiza
Archivos	`.gitattributes`, `tool/generate_source_manifest.py`
Riesgo	Ninguno. El comentario que encabeza `.gitattributes` explica exactamente este motivo

Cualquier archivo de texto nuevo **debe escribirse con LF**.

3.6 · Los PDF de la documentación salen sin diagramas

Aspecto	Detalle
Síntoma	El script avisa de que `mmdc` no está instalado y los diagramas ap arecen como bloques de texto
Causa	Falta `@mermaid-js/mermaid-cli`. Ningún motor de PDF de Pyth on ejecuta JavaScript, así que por sí solos no renderizan Mermai d
Diagnóstico	La salida del script informa cuántos diagramas se renderizaron y cuántos degradaron
Solución	`npm install -g @mermaid-js/mermaid-cli` y volver a ejecuta r
Archivos	`tool/build_system_documentation_pdf.py`
Riesgo	Ninguno. La degradación es deliberada: un PDF con un diagram a en texto es mejor que ningún PDF

En Windows, `mmdc` es un `.cmd` y Node ≥ 20.12 se niega a lanzarlo sin shell por el endurecimiento de CVE-2024-27980; el script lo contempla.

---

4 · Índice rápido por síntoma

Lo que ves	Entrada
«Escaneando» y no lee	1.1, 1.2
«Iniciando cámara…» eterno	1.3
«Cámara no disponible»	1.4
Lee pero no suena	1.5
«Inicio seguro»	1.6
Banner de modo temporal	1.7

Faltan registros	1.8
Elementos en el Centro de recuperación	1.9
«Reintentar migración» en Ajustes	1.10
La rotación de llave falló	1.11
«No existe una aplicación compatible»	1.12
Se borró el portapapeles	1.13
El QR generado no se lee	1.14
Acentos rotos en el CSV	1.15
`flutter run` no encuentra nada	2.1
`FilePickerPlugin` no encontrado	2.2
`pub get` no resuelve	2.3
Falta `yaml`	2.4
Versión de interfaz distinta	2.5
Ruta absoluta detectada	2.6
Enlace Markdown roto	2.7
Prueba de widget colgada	2.8
CI roja y local verde	2.9
La landing no se actualiza	3.1
Capturas rotas en local	3.2
SHA-256 que no coincide	3.3
Android no instala	3.4
Manifiesto de fuente descuadrado	3.5
PDF sin diagramas	3.6

Continúa en 15 · Riesgos y deuda técnica.

15 · Riesgos y deuda técnica

Alcance de este documento

Informativo. No se corrigió ninguno de estos hallazgos. El encargo de esta documentación excluye expresamente modificar el comportamiento del código: los únicos cambios aplicados a `lib/` son comentarios y documentación de símbolos, que no alteran la ejecución.

Cada hallazgo lleva severidad, impacto, probabilidad, evidencia verificable, ubicación, recomendación y prioridad. Todos se comprobaron con `grep` o leyendo el código; ninguno es una sospecha.

Antes de la lista: lo que está bien

Un informe que solo enumera problemas da una imagen falsa y se descarta por sesgado. Estas propiedades del repositorio son **sólidas** y alguien las romperá si nadie dice que importan:

1. **Ninguna acción externa sin confirmación.** `launchUrl` aparece **una sola vez** en las 6 821 líneas de `lib/`, y está detrás de un diálogo cuya condición es una disyunción entre el ajuste del usuario y el nivel de riesgo.
2. **Una llave ausente nunca se regenera en silencio.** Doble barrera en `PayloadCipher`, con prueba dedicada.
3. **La rotación de llave limpia su propia basura.** Con prueba dedicada.
4. **La migración verifica antes de borrar**, dentro de la transacción.
5. **La importación se valida completa antes de tocar la base**, con seis puertas y vista previa.
6. **El diagnóstico es estructuralmente incapaz de filtrar datos**: acepta un `Object` y guarda su `runtimeType`, no el mensaje.
7. **Cero telemetría, cero red saliente propia, cero permiso `INTERNET`.**
8. **Las 19 dependencias directas usan versión exacta**, sin `^`, con lockfile versionado.
9. **La degradación está declarada, nunca silenciosa**: el SBOM marca su cobertura real, la CI explica por qué omite la prueba de integración, y `ScanFeedback` avisa por bandera cuando el audio dejó de funcionar.
10. **El repositorio tiene gates propios y baratos**: diez comprobaciones offline que corren en menos de un segundo.
11. **La documentación previa declara lo que no se ha verificado**, sin adornos.

Resumen por severidad

Severidad	Cantidad	Identificadores
Alta	3	R-01, R-02, R-03
Media	9	R-04, R-06, R-13, R-15, R-19, R-27, R-32, R-33, R-34
Baja	23	El resto
Total	35	R-01 a R-35

Ninguno es crítico: no hay pérdida de datos, fuga de secretos ni fallo de seguridad explotable en el estado actual del código.

Severidad alta

R-01 · El catálogo de regresión no lo ejecuta ninguna prueba

Aspecto	Detalle
Severidad	Alta
Impacto	Ocho escenarios de lectura —daño parcial, poca luz, deformación, varios códigos en una imagen— están preparados, versionados y documentados, y **nunca se ejecutan**. Una regresión en la lectura no la detecta nada
Probabilidad	Certeza: ya ocurre
Evidencia	`grep -rn "test_assets" test/ integration_test/ lib/` devuelve **0 resultados**. `tool/validate_structure.py::check_test_assets` comprueba únicamente que el manifiesto exista, tenga `schemaVersion: 1` y que **cada archivo declarado exista en disco**: no decodifica ninguna imagen ni compara con `expectedPayload`
Ubicación	`test_assets/manifest.json`, `tool/validate_structure.py` líneas 116-129
Recomendación	Añadir una prueba de integración con dispositivo que use `analyzeImage` sobre las ocho imágenes y compare con `expectedPayload`. Mientras no exista, decir en `test_assets/README.md` que el catálogo es material para la matriz manual, no una suite automatizada
Prioridad	1

Inferencia basada en el código: una prueba que use `MobileScannerController.analyzeImage` necesita el motor de plataforma, que no existe bajo `flutter test`. Esa es probablemente la razón de que no se haya escrito. **No está documentada en el repositorio.**

R-02 · Nueve banderas de capacidad que nadie consulta

Aspecto	Detalle
Severidad	Alta como deuda de coherencia; nula como riesgo de ejecución
Impacto	`FeatureFlags` se serializa, se persiste y se restaura correctamente, y **ningún código de la aplicación la lee**. `docs/quality/FEATURE_FLAGS.md`, `IMPLEMENTATION_STATUS.md` y el contrato de compatibilidad la presentan como una frontera de extensión activa. Un lector concluye que existe un mecanismo de activación que no existe
Probabilidad	Certeza
Evidencia	`grep` sobre los nueve nombres (`experimentalParsers`, `secondaryScannerEngine`, `ocr`, `nfc`, `urlReputation`, `productLookup`, `encryptedSync`, `enterpriseApi`, `kioskMode`) devuelve **0 apariciones fuera de `lib/core/feature_flags/feature_flags.dart`**. Tampoco hay control en Ajustes para cambiarlas

Ubicación	`lib/core/feature_flags/feature_flags.dart`, `lib/models/app_settings.dart`, `lib/services/settings_repository.dart`
Recomendación	Documentar en `docs/quality/FEATURE_FLAGS.md` que en 1.1.0 las banderas son una **reserva de forma persistida**, no un mecanismo activo, y que activar una no tiene efecto
Prioridad	2

R-03 · El inventario persiste contenido sensible; el escáner no

Aspecto	Detalle
Severidad	Alta
Impacto	Un QR de Wi-Fi con contraseña, un OTP, un QR de pago o un documento AAMVA escaneado **dentro de una sesión de inventario** se persiste con su texto íntegro como clave del ítem. Va cifrado, pero contradice una garantía enunciada públicamente. Además, ese texto sale **en claro** al exportar la sesión
Probabilidad	Media: requiere que alguien escanee un código sensible en modo inventario
Evidencia	`InventoryScreen._onDetect` llama a `widget.store.addScan(record)` sin comprobar `record.isSensitive`. `InventoryStore.addScan` indexa por `record.rawValue`. Compárese con `ScannerScreen._persistAndShow`, que sí filtra: `records.where((r) => !r.isSensitive)`
Contradice	`docs/SECURITY.md` punto 5 («No persistir automáticamente OTP ni Wi-Fi con contraseña»), `docs/PRIVACY_POLICY.md` y `README.md`
Ubicación	`lib/features/inventory/inventory_screen.dart` líneas 389-405; `lib/state/inventory_store.dart` líneas 89-106
Recomendación	Decidir explícitamente: o filtrar también aquí, o documentar la excepción. Un inventario de códigos de producto nunca contiene secretos, así que filtrar no rompería el caso de uso real
Prioridad	1

Severidad media

R-04 · `InventoryScreen` no usa `ScannerEngine`

Aspecto	Detalle
Severidad	Media
Impacto	La frontera de extensión no cubre las dos pantallas que usan cámara. Un segundo motor exigiría reescribir también el inventario. Además, `autoTorch` **no se aplica** allí: es una diferencia de comportamiento observable

Probabilidad	Certeza
Evidencia	`inventory_screen.dart` construye `MobileScannerController` directamente en `_createController()` y llama al controlador sin pasar por la interfaz. Sus opciones omiten `torchEnabled` y `returnImage`
Contradice	`docs/ARCHITECTURE.md`: «`ScannerEngine` es la frontera estable alrededor del paquete de captura»
Ubicación	`lib/features/inventory/inventory_screen.dart` líneas 41-48
Recomendación	Migrar `InventoryScreen` a `ScannerEngine`, o documentar que la frontera cubre solo la pestaña Escanear
Prioridad	3

R-06 · La puerta de calidad local es más laxa que la CI

Aspecto	Detalle
Severidad	Media
Impacto	Un desarrollador ejecuta `tool/run_quality_gate.sh`, lo ve en verde, hace push, y la CI falla
Probabilidad	Alta cuando aparece un aviso informativo nuevo
Evidencia	`tool/run_quality_gate.sh` línea 6: `flutter analyze`. `.github/workflows/flutter-ci.yml` línea 44: `flutter analyze --fatal-infos`. `README.md` y `VALIDATION.md` documentan la forma estricta
Ubicación	`tool/run_quality_gate.sh`
Recomendación	Añadir `--fatal-infos`. Coste: un cambio de una línea
Prioridad	3

R-13 · `legacyKeyId` y `currentKeyId` valen lo mismo

Aspecto	Detalle
Severidad	Media en potencia, baja hoy
Impacto	Ambas constantes valen `'v2'`. Cambiar `currentKeyId` sin cambiar `legacyKeyId` rompería **silenciosamente** la lectura de los sobres heredados: se buscaría una llave que no existe
Probabilidad	Baja hoy; alta si alguien introduce un `v3` como identificador por defecto
Evidencia	`payload_cipher.dart` líneas 82-83: `static const String currentKeyId = 'v2'; static const String legacyKeyId = 'v2';`
Ubicación	`lib/core/security/payload_cipher.dart`

Recomendación	Comentar por qué coinciden, o definir `legacyKeyId` con un literal propio para que un cambio en `currentKeyId` no lo arrastre
Prioridad	4

R-15 · `replaceAll` no recorta el historial

Aspecto	Detalle
Severidad	Media
Impacto	`upsert` y `upsertAll` llaman a `_trim()`; `replaceAll` no. Hoy no puede desbordar porque `ImportService.maxHistoryRecords` vale también `5000`, pero **la protección depende de que dos constantes de archivos distintos sigan siendo iguales**
Probabilidad	Baja hoy; certeza si alguien sube el límite de importación
Evidencia	`history_repository.dart` líneas 99-114: `replaceAll` no invoca `_trim()`. `maxItems = 5000` en `history_repository.dart` línea 27; `maxHistoryRecords = 5000` en `import_service.dart` línea 46
Ubicación	`lib/services/history_repository.dart`
Recomendación	Llamar a `_trim()` también desde `replaceAll`, o derivar `ImportService.maxHistoryRecords` de `HistoryRepository.maxItems`
Prioridad	4

R-19 · La lista de campos enmascarados es literal

Aspecto	Detalle
Severidad	Media
Impacto	Solo cinco nombres se enmascaran: `Contraseña`, `Secreto`, `Dirección`, `IBAN`, `Carga`. Un parser nuevo que produjera un campo sensible con otro nombre —`Clave`, `Semilla`, `PIN`— **se mostraría en claro** aunque el registro estuviera marcado como sensible
Probabilidad	Baja hoy; certeza si se añade un parser
Evidencia	`scan_result_sheet.dart` línea 113
Ubicación	`lib/features/result/scan_result_sheet.dart`
Recomendación	Mover la marca de sensibilidad al nivel de campo dentro de `ParsedContent`, en lugar de deducirla del nombre
Prioridad	4

R-27 · La exportación sale en claro sin advertencia en la interfaz

Aspecto	Detalle

Severidad	Media
Impacto	Compartir **un registro** sensible abre un diálogo de advertencia. Exportar **todo el historial** a CSV, XLSX o JSON no muestra ninguno, y el archivo resultante no está cifrado
Probabilidad	Alta: es la vía normal de exportar
Evidencia	`ExportService` no cifra nada; `HistoryScreen` invoca los tres métodos de exportación desde un `PopupMenuButton` sin diálogo previo. Compárese con `ScanRecordCard._shareRecord`, que sí lo tiene
Ubicación	`lib/features/history/history_screen.dart` líneas 76-81; `lib/services/export_service.dart`
Recomendación	Añadir un diálogo antes de exportar, con el número de registros y la advertencia de que el archivo va en claro
Prioridad	3

R-32 · Sin CSP ni cabeceras de seguridad para la PWA

Aspecto	Detalle
Severidad	Media
Impacto	`docs/SECURITY.md` exige publicar la PWA «con HSTS y cabeceras de seguridad apropiadas». No hay ninguna configuración que las aplique
Probabilidad	Certeza
Evidencia	**No identificado**: no existe archivo de cabeceras, ni meta `Content-Security-Policy` en `landing/index.html`, ni paso en `deploy-landing.yml` que las añada. GitHub Pages no permite cabeceras personalizadas
Ubicación	`landing/`, `.github/workflows/deploy-landing.yml`
Recomendación	Añadir una meta CSP en el HTML, que es lo único que GitHub Pages permite, o rebajar la exigencia de `docs/SECURITY.md` a lo que la plataforma soporta
Prioridad	4

R-33 · Las acciones de GitHub no están fijadas a SHA

Aspecto	Detalle
Severidad	Media
Impacto	Riesgo de cadena de suministro. Un compromiso de cualquiera de esos repositorios ejecutaría código arbitrario en el runner que compila los APK publicados
Probabilidad	Baja, pero de impacto alto

Evidencia	Los 13 `uses:` de los dos workflows apuntan a etiquetas móviles: `actions/checkout@v7`, `actions/setup-python@v7`, `subosito/flutter-action@v2`, `actions/upload-artifact@v7`, `actions/configure-pages@v5`, `actions/upload-pages-artifact@v3`, `actions/deploy-pages@v4`
Mitigación presente	`permissions: contents: read` en el trabajo de calidad; permisos acotados en el de publicación; ningún secreto consumido
Ubicación	`.github/workflows/flutter-ci.yml`, `.github/workflows/deploy-landing.yml`
Recomendación	Fijar cada acción a un SHA de commit y dejar que Dependabot los actualice
Prioridad	3

R-34 · La migración del historial no tiene prueba

Aspecto	Detalle
Severidad	Media
Impacto	Es el flujo con más precauciones del repositorio —respaldo cifrado, verificación dentro de la transacción, borrado solo al final— y el que más daño haría si se rompiera: es el único que puede **perder datos del usuario**. No tiene ninguna prueba
Probabilidad	Baja de que se rompa; alta de que nadie lo note si se rompe
Evidencia	Ningún archivo de `test/` menciona `scan_history_v1`, `_migrateLegacyHistory` ni `HistoryMigrationStatus`. `history_repository_test.dart` cubre solo `replaceAll`
Ubicación	`lib/services/history_repository.dart` líneas 154-221
Recomendación	Tres casos: instalación limpia, migración correcta con verificación del respaldo cifrado, y fallo que conserva el origen y crea la incidencia
Prioridad	2

Severidad baja

R-05 · Código muerto en `BarcodeLabels`

`BarcodeLabels.contentType` y `_fallbackType` (39 líneas) **no se llaman desde ningún sitio**: `grep -rn "BarcodeLabels\." lib test` devuelve una sola línea, `scan_record.dart:44`, que usa `format`. Fueron sustituidos por `ParsedContent.title`. **Recomendación**: eliminarlos. **Prioridad**: 6.

R-07 · Un modelo importa una carpeta de `features`

`lib/models/scan_record.dart` línea 9 importa `features/formats/domain/content_parser_registry.dart`. No es un ciclo, pero invierte la dirección declarada de las capas. El registro de parsers pertenece conceptualmente a `services`. **Recomendación**: mover `features/formats/domain/` a `lib/services/parsers/`. **Prioridad**: 6.

R-08 · Línea inalcanzable en `ContentParserRegistry.parse`

La reserva final `return ContentInterpreter.parse(rawValue)` no se alcanza nunca: `LegacyContentParser.canParse` devuelve siempre `true` y `unregister` protege `builtin-v2`. Es defensa redundante, no un error. **Recomendación**: comentarla como defensa deliberada. **Prioridad**: 7.

R-09 · Dos valores de enumeración que nunca se emiten

`RecoveryEntityType.database` y `.startup` están declarados y **nunca se pasan a `RecoveryRepository.record`**: `grep -rn "entityType: RecoveryEntityType\." lib` muestra solo `history` (2), `inventory` (2) y `migration` (1). Sus ramas en `retry` y `discard` son inalcanzables. **Recomendación**: emitirlos desde el manejador de fallo de arranque, o eliminarlos. **Prioridad**: 6.

R-10 · `FormatsScreen` lista formatos a mano

Tres grupos con 20 nombres escritos como literales, sin relación con `BarcodeLabels.format` ni con lo que declare el motor. Si `mobile_scanner` añadiera o quitara una simbología, la pantalla no lo reflejaría. **Prioridad**: 6.

R-11 · El parámetro `compact` nunca se usa

`ScanRecordCard.compact` tiene valor por defecto `false` y **ningún llamante lo pasa como `true`**: `grep -rn "compact" lib test` muestra solo la declaración, el campo y su uso interno. La rama que oculta la banda de riesgo y los botones es inalcanzable. **Prioridad**: 7.

R-12 · Un filtro de ruta inerte en el workflow de publicación

`deploy-landing.yml` incluye `web/**` en su lista `paths:`, pero `/web/` está en `.gitignore` y no hay ningún archivo versionado bajo esa ruta. Ese filtro **no puede dispararse nunca**. **Recomendación**: eliminarlo. **Prioridad**: 7.

R-14 · Cualquier cadena de 10 dígitos se clasifica como ISBN

`ContentInterpreter._isIsbn` devuelve `true` si, tras eliminar todo lo que no sea dígito o `X`, la longitud es exactamente 10. Como esa comprobación va **antes** de la rama de producto (`^\d{8,14}$`), un código de producto de 10 dígitos aparece como «ISBN». **Recomendación**: validar el dígito de control del ISBN-10. **Prioridad**: 5.

R-16 · El CSV del historial no incluye las notas; el XLSX sí

El CSV tiene 8 columnas y omite `notas`; el XLSX tiene 9 y la incluye. Los dos del inventario tienen 8 y ambos incluyen `notas`. Es una asimetría sin motivo aparente. **Recomendación**: añadir `notas` al CSV del historial. **Prioridad**: 6.

R-17 · `_failure` se asigna fuera de `setState`

En `_BootstrapHostState._retry`, el `catchError` asigna `_failure` sin envolver en `setState`. Funciona porque el `setState` que asigna `_future` ya programó una reconstrucción y el `FutureBuilder` se reconstruye al resolverse el `Future`, pero **depende de ese orden**. **Recomendación**: envolver la asignación. **Prioridad**: 5.

R-18 · Los formatos de pago se interpretan por posición de línea

Swiss QR Bill y EPC/SEPA leen campos por índice fijo (`lines[3]`, `lines[5]`, `lines[18]`, `lines[19]`). Es lo que especifican ambos estándares, pero una variante con líneas insertadas produciría campos desplazados **sin ningún aviso**: los `if (lines.length > N)` evitan el desbordamiento, no el desalineamiento. **Recomendación**: validar la versión del formato antes de mapear. **Prioridad**: 5.

R-20 · Las llaves antiguas no se borran tras rotar

`rotateEncryptionKey` no llama a `forgetKey` sobre la llave anterior en el camino de éxito. El almacén seguro acumula una entrada por rotación, y material capaz de descifrar datos antiguos permanece en el dispositivo. Es prudente, pero **no está documentado**. **Prioridad**: 5.

R-21 · `RecoveryService.retry` pierde la causa del fallo

Su `catch` es `on Object { return false; }`: cualquier error se convierte en un `false` que la interfaz traduce a «No fue posible recuperar el elemento». **No se registra en `AppDiagnostics`**, así que la causa se pierde justo en la pantalla diseñada para diagnosticar. **Recomendación**: registrar antes de devolver `false`. **Prioridad**: 4.

R-22 · Mezcla de fechas locales y UTC

`ScanRecord.scannedAt` e `InventorySession.createdAt` usan `DateTime.now()` (**local**) y se serializan sin sufijo de zona. `RecoveryIssue.detectedAt`, el `createdAt` del sobre de cifrado y los `exportedAt` usan `toUtc()`. Comparar registros creados en zonas horarias distintas —por ejemplo tras importar un respaldo ajeno— produciría un orden incorrecto. **Recomendación**: unificar en UTC y mostrar en local. **Prioridad**: 4.

R-23 · Cuatro marcas de esquema que nadie lee

`v2MigratedAt`, `recoveryStoreEnabled`, `cipherEnvelopeVersion` y `encryptionMetadataInDatabase` se escriben en `_schema_meta` y **no se leen nunca**: `grep` sobre los cuatro nombres devuelve solo `schema_migrator.dart`. Son registro histórico, no configuración. **Recomendación**: documentarlo como tal. **Prioridad**: 7.

R-24 · `SchemaMigrationResult` se descarta

`AppDatabase.open()` y `openTemporary()` hacen `await SchemaMigrator(...).migrate();` sin usar el resultado. `fromVersion`, `toVersion` y `applied` solo los consume la prueba. **Recomendación**: registrar el resultado en `AppDiagnostics` cuando `applied` no esté vacío. **Prioridad**: 6.

R-25 · `normalizedHost` se calcula y se descarta

`SecurityAssessment.normalizedHost` se rellena en `analyze` y **`ScanRecord` no lo guarda**: no aparece en `toJson`. Sería útil para filtrar el historial por dominio. **Prioridad**: 7.

R-26 · El temporizador del portapapeles no se puede cancelar

`ClipboardService.copy` crea un `Timer` cuya referencia no se guarda. Sobrevive al `dispose` del widget y se dispara aunque se haya salido de la pantalla. Es intencional —el objetivo es limpiar pase lo que pase— y está mitigado por la comprobación de que el valor no haya cambiado. **No está documentado. Prioridad**: 6.

R-28 · El paquete de recuperación no tiene importador

`RecoveryService.exportBundle` produce un JSON con `schemaVersion: 1` que **nada consume**. No hay importador ni herramienta que lo procese. Su utilidad real es entregarlo a soporte. **Recomendación**: decirlo en `docs/quality/RECOVERY.md`. **Prioridad**: 6.

R-29 · `ScannerEngine` filtra tipos de `mobile_scanner`

Dos de sus nueve miembros —`controller` y `state`— devuelven `MobileScannerController` y `ValueListenable<MobileScannerState>`. Un motor alternativo tendría que producir esos tipos, lo que la hace inservible como frontera independiente del proveedor. Es una abstracción de conveniencia sobre el paquete actual. **Recomendación**: rediseñarla con tipos propios si alguna vez se añade un segundo motor. **Prioridad**: 5.

R-30 · Cambiar la retención poda sin confirmar

`_updateRetention` guarda el ajuste y llama de inmediato a `pruneOlderThan`. Elegir «30 días» **borra ahí mismo** todo lo anterior, sin diálogo y sin posibilidad de deshacer. Compárese con «Borrar todo el historial», que sí confirma. **Recomendación**: añadir un diálogo que indique cuántos registros se eliminarán. **Prioridad**: 4.

R-31 · Dos funciones hash distintas para el mismo propósito

`ScanRecord.id` usa SHA-256 truncado a 20; `InventorySession.id` usa SHA-1 truncado a 16. No es un problema de seguridad —son identificadores locales con marca de tiempo en la entrada— pero es una inconsistencia entre dos modelos hermanos. **Prioridad**: 7.

R-35 · Sin umbral de cobertura

`flutter test --coverage` genera `lcov.info` y la CI lo sube como artefacto, pero **nada falla si la cobertura baja**: no hay umbral en el workflow, ni en `validate_structure.py`, ni en la documentación. **Recomendación**: declarar un mínimo y hacerlo fallar la CI. **Prioridad**: 5.

Deuda de documentación

Contradicciones o vacíos entre lo que la documentación previa afirma y lo que el código hace. Se registran aquí, **no se corrigieron**.

#	Documento	Afirma	Realidad
D-1	`docs/SECURITY.md` punto 5	«No persistir automáticament e OTP ni Wi-Fi con contraseñ a»	Cierto en el escáner; **falso en el inventario** (R-03)
D-2	`docs/ARCHITECTURE.md`	«`ScannerEngine` es la fronter a estable alrededor del paque te de captura»	`InventoryScreen` no la usa (R-04)
D-3	`docs/quality/FEATURE_FLA GS.md`	Presenta las nueve banderas como funciones aisladas activ ables	Ninguna se consulta (R-02)
D-4	`docs/quality/DEVICE_TEST _MATRIX.md`	«Los recursos reproducibles e stán en `test_assets/manife st.json`»	Correcto, pero ninguna prueb a automatizada los usa (R-01)
D-5	`docs/SECURITY.md`	Exige HSTS y cabeceras de s eguridad para la PWA	No hay configuración que las aplique (R-32)
D-6	Ningún documento	—	El **momento** de cada lectura queda en claro en la base, au nque el contenido no. No esta ba dicho en ningún sitio
D-7	Ningún documento	—	`resetNonSensitive` deja inta ctas exactamente tres claves. Solo lo dice el nombre del m étodo
D-8	Ningún documento	—	`HistoryRepository.load` **es cribe** en la base al actualizar sobres antiguos

Los ocho quedan documentados en este directorio: D-6, D-7 y D-8 en 07-database.md y 06-deep-code-explanation.md.

Referencia histórica correcta

No es deuda y conviene no «corregirlo»: `docs/quality/SCOPE_1_0.md` y el `CHANGELOG.md` mencionan **48 pruebas**. Es correcto: se refieren a la versión 1.0.0. El marcador de estado actual —57— aparece en `README.md`, `VALIDATION.md`, `IMPLEMENTATION_STATUS.md` y `RELEASE_SOURCE.json`, y los cuatro coinciden con lo medido.

Igual ocurre con `COMPATIBILITY_CONTRACT.md`, que habla de «los compromisos que la versión 1.0.0 adquiere»: es una referencia histórica deliberada.

Priorización sugerida

Informativo. **Nada de esto se aplicó.**

Prioridad	Hallazgos	Esfuerzo conjunto estimado
1	R-01, R-03	1 jornada
2	R-02, R-34	Media jornada
3	R-04, R-06, R-27, R-33	1 jornada
4	R-13, R-15, R-19, R-21, R-22, R-30, R-32	1 jornada
5	R-14, R-17, R-18, R-20, R-29, R-35	Media jornada
6	R-05, R-07, R-09, R-10, R-16, R-24, R-26, R-28	Media jornada
7	R-08, R-11, R-12, R-23, R-25, R-31	2 horas

Los dos de prioridad 1 son los únicos que afectan a una garantía enunciada públicamente (R-03) o a una capacidad de detección que se cree tener y no se tiene (R-01).

Continúa en 16 · Glosario.

16 · Glosario

Definiciones pensadas para que las entienda alguien sin formación técnica. Los términos marcados con ★ significan algo **específico en este sistema**: su definición general no basta para entender el código.

A

AAMVA — Estándar estadounidense para el código de barras del reverso de una licencia de conducir. Este sistema **reconoce siete campos**: número de licencia, apellido, nombre, fecha de nacimiento, vencimiento, dirección y ciudad. Se marca siempre como contenido sensible.

AES-256-GCM — El método de cifrado que protege el historial y los inventarios. «AES-256» es la cerradura; «GCM» añade un sello de garantía que delata si alguien manipuló el contenido. Si se altera un solo bit, el descifrado falla en vez de devolver basura.

Application Identifier (AI) ★ — En un código GS1, un número de dos a cuatro dígitos que dice qué significa el dato que viene detrás. `01` es el GTIN del producto, `10` el lote, `17` la fecha de vencimiento. Este sistema traduce nueve de ellos a nombre legible; el resto se muestra como «AI» seguido del número.

Aztec — Un tipo de código bidimensional, parecido a un QR pero con un cuadrado en el centro en vez de tres esquinas. Común en billetes de tren y tarjetas de embarque.

B

Base64 — Una forma de escribir datos binarios usando solo letras, números y un par de símbolos. Este sistema la usa para dos cosas: guardar el resultado del cifrado, y representar un código cuyo contenido no es texto legible.

`binary-base64:` ★ — Prefijo interno. Cuando un código contiene datos que no son texto, este sistema lo guarda como `binary-base64:` seguido del contenido en Base64. Tres partes del código reconocen ese prefijo —el intérprete, el analizador de riesgo y el generador de registros— y hay que cambiarlas a la vez si se toca.

Bootstrap ★ — En este proyecto tiene **dos significados distintos**:

1. `tool/bootstrap.py`, el script que genera las carpetas de Android, iOS, web y macOS, que no están en el repositorio.
2. `lib/bootstrap.dart`, el arranque interno de la aplicación, que construye la base de datos, el cifrador y los almacenes de estado.

`builtin-v2` ★ — El identificador del intérprete de contenido integrado. Es el único que **no se puede quitar** del registro de intérpretes.

C

Cancelación en dos niveles ★ — Al analizar un PDF, el botón «Cancelar» actúa sobre dos cosas a la vez: el bucle que recorre las páginas y el propio motor que está dibujando la página actual. Sin la segunda, cancelar esperaría a que terminara de dibujar la página en curso.

`canOpen` ★ — Campo de cada lectura que dice si existe una acción externa posible. **No significa «es seguro»**: un enlace de riesgo alto tiene `canOpen` verdadero, y lo que impide abrirlo sin más es el diálogo de confirmación.

Catálogo de regresión ★ — Las ocho imágenes de `test_assets/` con su resultado esperado. Sirven para comprobar que el lector sigue leyendo lo mismo tras un cambio. **En esta versión ninguna prueba automática las usa**; ver el hallazgo R-01.

Cifrado autenticado — Cifrado que, además de ocultar, detecta manipulación. Es lo que aporta la parte «GCM» de AES-256-GCM.

`ContentKind` ★ — La lista de los diecisiete tipos de contenido que este sistema sabe reconocer: enlace, Wi-Fi, contacto, evento, correo, teléfono, SMS, ubicación, OTP, GS1, ISBN, producto, pago, criptomoneda, identidad, binario y texto.

D

Data Matrix — Código bidimensional cuadrado, muy usado en industria y etiquetado de componentes pequeños.

Deduplicación en dos niveles ★ — Este sistema evita registrar el mismo código dos veces de dos maneras: dentro de una misma captura descarta repeticiones del mismo texto, y entre capturas ignora la misma **combinación** de códigos durante dos segundos. El inventario usa una ventana distinta —1,2 segundos y por código individual— porque ahí escanear dos veces sí debe sumar dos unidades.

Diagnóstico privado ★ — El informe técnico que la aplicación permite copiar. Contiene **solo** el tipo de error, la zona donde ocurrió y una huella de la pila de llamadas. Nunca contiene el mensaje del error, ni códigos escaneados, ni enlaces, ni contraseñas, ni notas. Hay una prueba que lo verifica.

E

EAN-13 / EAN-8 — Los códigos de barras de los productos del supermercado. El de 13 dígitos es el habitual; el de 8, para envases pequeños.

EMVCo — El estándar de los QR de pago. Este sistema los reconoce, extrae moneda, monto, comercio, ciudad y país, y los marca como contenido sensible.

EPC / SEPA — El QR europeo de transferencia bancaria. Se identifica porque empieza por las letras `BCD` seguidas de un salto de línea.

Esquema — La primera parte de una dirección, antes de los dos puntos: `https:`, `mailto:`, `tel:`. Este sistema **solo permite abrir siete**: `http`, `https`, `mailto`, `tel`, `sms`, `smsto` y `geo`. Cualquier otro se marca como riesgo elevado y no se abre.

F

FeatureFlags ★ — Nueve interruptores para capacidades futuras: parsers experimentales, segundo motor, OCR, NFC, reputación de enlaces, consulta de productos, sincronización, API empresarial y modo kiosco. **Se guardan y se leen del almacenamiento, pero ningún código los consulta**: en esta versión son una declaración de intención, no un mecanismo activo. Hallazgo R-02.

Flutter — El marco de trabajo con el que está hecha la aplicación. Permite escribir una vez y compilar para Android, iOS, macOS y navegador.

G

GS1 — La organización que define los códigos de producto y logística de todo el mundo. Un «código GS1» agrupa varios datos con etiqueta: producto, lote, vencimiento, cantidad.

GTIN — El número que identifica un producto de forma única en todo el mundo. Es lo que hay detrás de un código de barras de supermercado.

H

`HandheldFrame` ★ — La capa que impide que la aplicación se estire de borde a borde en una pantalla ancha. Por debajo de 560 puntos **no envuelve nada**: en un móvil, la aplicación es exactamente la misma que si esta capa no existiera.

Huella de pila ★ — Un número corto de ocho dígitos que resume dónde ocurrió un error, sin revelar nada del contenido. Dos errores con la misma huella son probablemente el mismo problema.

I

Inicio seguro ★ — La pantalla que aparece si la aplicación no consigue arrancar. Ofrece cuatro salidas: reintentar, abrir en modo temporal, restablecer las preferencias visuales o copiar el diagnóstico privado.

Isolate — En Flutter, un hilo de ejecución con su propia memoria. Este sistema usa uno aparte para generar los archivos CSV y Excel, para que la interfaz no se congele.

ISBN — El número que identifica un libro. Este sistema lo reconoce cuando tiene 13 dígitos empezando por 978 o 979, **o cuando tiene exactamente diez caracteres**, lo que hace que algunos códigos de producto de diez dígitos se etiqueten como ISBN. Hallazgo R-14.

L

Llave activa ★ — El identificador de la llave con la que se está cifrando ahora mismo. Empieza siendo `v2` y cambia a `v3_` seguido de una marca de tiempo cada vez que se rota. Se guarda **dentro de la base de datos**, en la misma transacción que los datos recifrados.

Llave ausente ★ — Situación en la que la aplicación encuentra datos cifrados pero no la llave para descifrarlos. Ocurre tras una reinstalación o una restauración del sistema. La respuesta del sistema es deliberada: **no crea una llave nueva**, informa del problema y **bloquea también las escrituras** con ese identificador. Sin esa segunda mitad, la primera escritura posterior destruiría la posibilidad de recuperar los datos antiguos.

M

MaxiCode — El código hexagonal que usa UPS en sus paquetes.

MeCard — Un formato de tarjeta de contacto más compacto que vCard, popular en Japón.

Micro QR — Un QR pequeño, con menos capacidad, para etiquetas diminutas.

ML Kit — El motor de reconocimiento de Google que este sistema usa en Android. En Apple se usa Vision; en el navegador, el motor web del paquete de lectura.

Modo temporal ★ — Forma de arrancar cuando la base de datos no se puede abrir. Todo funciona igual, pero **el historial y los inventarios viven solo en memoria y se pierden al cerrar**. Un aviso permanente lo indica, y la

rotación de llave se deshabilita.

N

Nonce — Un número aleatorio que se usa una sola vez en cada operación de cifrado. Es lo que hace que cifrar dos veces el mismo texto produzca resultados distintos. Se guarda junto al texto cifrado porque hace falta para descifrar; no es secreto.

O

OTP — «One-Time Password», el código de seis dígitos que cambia cada treinta segundos para verificación en dos pasos. Un QR de OTP contiene la **semilla** que genera esos códigos, así que es tan sensible como una contraseña. Este sistema lo muestra pero **nunca lo guarda en el historial** desde el escáner.

P

PDF417 — Código de barras rectangular y apilado. Se usa en tarjetas de embarque y en el reverso de las licencias de conducir.

Punycode — Una forma de escribir dominios con caracteres no latinos usando solo letras ASCII. Se reconoce porque el dominio contiene `xn--`. Es legítimo, pero también es la técnica con la que se fabrican dominios que **parecen** otro. Este sistema lo marca como riesgo elevado.

PWA — «Progressive Web App»: la aplicación funcionando dentro de un navegador, instalable como si fuera nativa. Es la forma en que este proyecto cubre Windows y Linux, que no tienen versión nativa.

R

Recuperación (Centro de) ★ — La pantalla donde aparecen los registros que no se pudieron leer. Permite reintentar el descifrado, descartar solo el registro afectado, marcarlo revisado o exportar un paquete para soporte. **Los registros dañados nunca se borran solos.**

Registro sensible ★ — Una lectura que el sistema clasifica como demasiado delicada para guardarla sola: contraseña de Wi-Fi, semilla OTP, datos de pago, dirección de criptomoneda o documento de identidad. Se muestra en pantalla, con los campos delicados tapados, pero **no entra en el historial** cuando se escanea desde la pestaña Escanear. En el inventario sí entra; hallazgo R-03.

Rotación de llave ★ — Cambiar la llave de cifrado y volver a cifrar todo con la nueva. Es una operación de todo o nada: si algo falla a mitad, nada cambia y la llave nueva se borra para que no quede material suelto.

S

SBOM — «Software Bill of Materials»: la lista completa de las piezas de terceros que componen el programa, con su versión. Sirve para saber si una vulnerabilidad publicada afecta a este proyecto.

`ScanPhase` ★ — Los cuatro estados que la barra superior del escáner puede mostrar: iniciando, escaneando, en pausa y no disponible. **La barra solo se mueve cuando el motor está analizando imágenes de verdad**, que es lo que distingue una cámara esperando un código de una cámara que no llegó a arrancar.

Sembast — La base de datos que usa este proyecto. No es SQL: guarda documentos en un archivo local, o en el almacenamiento del navegador cuando se usa como PWA.

Sobre de cifrado ★ — La caja donde viaja cada dato cifrado. Por fuera lleva cuatro etiquetas legibles —versión, algoritmo, identificador de llave y fecha— y por dentro el contenido cifrado, el nonce y el sello de garantía. Las etiquetas van visibles porque sin ellas nadie sabría cómo abrir la caja.

Sobre heredado ★ — El formato anterior, sin etiquetas por fuera. Se sigue aceptando, y **se actualiza al formato nuevo la primera vez que se lee**. Eso significa que leer el historial escribe en la base de datos.

Swiss QR Bill — La factura con QR del sistema bancario suizo. Se identifica porque empieza por las letras `SPC` seguidas de un salto de línea.

T

Tono de lectura ★ — El doble pitido que suena al leer un código. Es un **recurso propio empaquetado dentro de la aplicación**, no el sonido del sistema. El motivo es concreto: Android silencia el sonido del sistema en cuanto el usuario apaga los sonidos táctiles, así que la confirmación no se oía en la mayoría de los teléfonos.

U

UPC-A / UPC-E — Los códigos de barras de producto usados sobre todo en Estados Unidos y Canadá.

V

Validador estructural ★ — `tool/validate_structure.py`. Diez comprobaciones que se ejecutan sin necesidad de compilar: que los YAML sean válidos, que ningún `import` interno apunte a un archivo inexistente, que no haya rutas de máquina personal, que la versión mostrada coincida con la real, que nada registre datos sensibles, que los recursos de prueba existan, que los JSON sean válidos, que **ningún enlace de la documentación esté roto**, y que el SBOM exista.

vCard — El formato estándar de tarjeta de contacto. Un QR de vCard contiene nombre, teléfono, correo y organización.

vEvent — El formato estándar de evento de calendario, dentro de un archivo iCalendar.

Ventana de lectura ★ — El cuadrado central donde se busca el código. Ocupa el 68 % del lado menor de la vista. Se puede desactivar en Ajustes para analizar la imagen completa. **Un defecto en su recálculo fue la causa del fallo que originó la versión 1.1.0.**

Z

Zona horaria ★ — Detalle a tener presente: la fecha de cada lectura se guarda en **hora local**, mientras que las fechas internas del sistema de recuperación y de los sobres de cifrado se guardan en UTC. Hallazgo R-22.

Abreviaturas usadas en esta documentación

Abreviatura	Significado

AGP	Android Gradle Plugin
API	Interfaz de programación. En Android, también el número de versión del sistema
CI	Integración continua: las comprobaciones automáticas en cada cambio
CSP	Content-Security-Policy, una cabecera de seguridad web
CVE	Identificador público de una vulnerabilidad conocida
D-N	Identificador de una deuda de documentación en 15
FVM	Flutter Version Management, gestor de versiones de Flutter
KGP	Kotlin Gradle Plugin
MASVS	Estándar de verificación de seguridad de aplicaciones móviles de OWASP
PR	Pull request: propuesta de cambio
R-N	Identificador de un hallazgo en 15
S-N	Identificador de un hallazgo de seguridad en 11
SDK	Kit de desarrollo
SHA-256	Función que produce una huella única de un archivo. Sirve para verificar que una descarga es exactamente la publicada
TLV	«Tag-Length-Value», la forma en que se estructuran los datos de un QR de pago EMVCo
UTC	Hora universal coordinada, sin ajuste de zona

Continúa en 17 · Resumen ejecutivo.

17 · Resumen ejecutivo

Documento de decisión. Sin detalle técnico innecesario y con el esfuerzo de cada recomendación cuantificado.

En una frase

Universal Code Scanner es una **aplicación móvil de lectura de códigos QR y de barras que interpreta el contenido y evalúa su riesgo antes de permitir cualquier acción**, funcionando íntegramente en el dispositivo, sin servidores, sin cuentas y sin telemetría.

El producto

Aspecto	Estado
Versión analizada	`1.1.0+2`, commit `c1f9878`, rama `main`
Licencia	MIT
Tecnología	Flutter 3.44.7 · Dart 3.12.2
Tamaño del código propio	6 821 líneas de Dart en 59 archivos
Dependencias de terceros	19 paquetes directos, con versión exacta y lockfile versionado
Plataformas verificadas	Android (compilado y ejecutado en emulador) y web/PWA (compilada y publicada)
Plataformas que compilan sin ejecución verificada	iOS y macOS, en la CI
Plataformas sin motor nativo	Windows y Linux, cubiertas por la PWA
Distribución actual	APK en GitHub Releases y PWA en GitHub Pages
Estado de tiendas	Fuera del alcance de 1.1.0

La propuesta de valor, en una comparación

Pregunta	Un lector de códigos habitual	Universal Code Scanner
¿Qué hace al leer un enlace?	Lo abre	Muestra dominio, protocolo, ruta y consulta, y **espera confirmación**
¿Avisa si el enlace parece falso?	No	Sí: 16 señales locales, incluidas Punycode, alfabetos mezclados y credenciales embebidas
¿Dónde se procesa?	Con frecuencia, en un servidor	**Siempre en el dispositivo**
¿Qué sabe el fabricante de lo que escaneas?	Depende	**Nada**: no hay red saliente propia ni permiso de internet
¿Se guarda el historial?	A veces, en claro	Sí, **cifrado con AES-256-GCM**

¿Guarda contraseñas Wi-Fi y códigos de verificación?	Sí, habitualmente	**No** desde la pestaña Escanear
¿Sirve para inventariar?	Rara vez	Sí: sesiones de conteo con exportación a Excel
¿Sirve para generar códigos?	A veces	Sí: 10 simbologías a PNG y SVG

Qué se verificó ejecutando

Sobre el commit analizado, con el toolchain fijado por el propio proyecto:

Comprobación	Resultado
Resolución de dependencias	Correcta, sin cambios en el lockfile
Análisis estático estricto (`--fatal-infos`)	**0 hallazgos**
Suite de pruebas	**57 de 57 en verde**, en 24 segundos
Validación estructural offline	Sin errores

Estos cuatro resultados coinciden exactamente con lo que el repositorio declara en `VALIDATION.md` y en las insignias del `README.md`. **La documentación previa del proyecto es fiable**: no se encontró ninguna cifra inflada.

Estado de salud: cinco señales

Señal	Valoración	Base
Calidad del código	Buena	0 hallazgos con el nivel más estricto del a nalizador; 4 reglas adicionales sobre `flu tter_lints`
Corrección criptográfica	Muy buena	AES-256-GCM autenticado, sin criptograf ía escrita a mano, con las tres propiedad es críticas cubiertas por prueba
Cobertura de pruebas	Desigual	El 100 % de la criptografía tiene prueba; el 0 % de los almacenes de estado y de l as pantallas grandes
Honestidad de la documentación	Muy buena	Declara sin adornos lo que no se ha verifi cado, incluidos los dispositivos físicos
Automatización	Buena	CI para cuatro plataformas, SBOM, inven tario de licencias, checksums, Dependab ot y un validador propio

---

Lo que hay que saber antes de decidir

Tres fortalezas que sostienen el producto

La promesa central está implementada de verdad. `launchUrl` —la única forma de abrir algo— aparece **una sola vez** en las 6 821 líneas del proyecto, y está detrás de un diálogo. No es una afirmación de marketing.

La criptografía está bien hecha y bien probada. Tres propiedades difíciles y valiosas están implementadas y verificadas: una llave ausente nunca se regenera en silencio, una rotación fallida no deja material huérfano, y la migración verifica antes de borrar. Cualquiera de las tres, mal resuelta, produce pérdida irreversible de datos del usuario.

El proyecto declara sus límites. `VALIDATION.md` e `IMPLEMENTATION_STATUS.md` enumeran lo no verificado con el mismo detalle que lo verificado. Eso es infrecuente y reduce el riesgo de sorpresas.

Tres carencias que conviene atender

Ocho escenarios de regresión preparados y nunca ejecutados. `test_assets/` contiene imágenes con daño parcial, poca luz, deformación y varios códigos por imagen, cada una con su resultado esperado. El validador comprueba que los archivos existen; **ninguna prueba los decodifica**. Es una capacidad de detección que se cree tener y no se tiene.

Una garantía pública que el inventario no cumple. La documentación afirma que las contraseñas Wi-Fi y los códigos de verificación no se guardan automáticamente. Es cierto en la pestaña Escanear y **falso en la de Inventario**, que no aplica el filtro.

Nueve interruptores de capacidad que nadie consulta. `FeatureFlags` se presenta en tres documentos como una frontera de extensión activa. Se persiste y se restaura correctamente, y **ningún código de la aplicación la lee**.

Ninguna de las tres es un fallo de seguridad explotable. Las tres son divergencias entre lo que la documentación afirma y lo que el código hace.

Riesgos, resumidos

35 hallazgos registrados. Ninguno crítico: **no hay pérdida de datos, fuga de secretos ni vulnerabilidad explotable** en el estado actual.

Severidad	Cantidad	Qué implican
Alta	3	Divergencias entre documentación y código, y una capacidad de prueba no ejercida
Media	9	Deuda que crecerá: acciones sin fijar a SHA, gate local más laxo que la CI, exportación sin advertencia
Baja	23	Código muerto, inconsistencias menores, decisiones no documentadas

Detalle completo en 15-risks-and-technical-debt.md.

El riesgo que no está en esa lista

Ninguna versión se ha probado en dispositivos físicos de forma sistemática. Está declarado por el propio proyecto en tres documentos. La aplicación se instaló y arrancó en un emulador de Android API 36, y la matriz obligatoria de `../quality/DEVICE_TEST_MATRIX.md` sigue sin completar.

Es el mayor riesgo del producto y no es técnico: cámara real, biometría, PDF grandes, TalkBack y VoiceOver solo se pueden validar en hardware. Y la corrección principal de 1.1.0 —el fallo de lectura— **la reportó un teléfono real y solo un teléfono real puede cerrarla**.

Recomendaciones, con esfuerzo cuantificado

Ordenadas por relación entre valor y coste. **Ninguna se aplicó en este trabajo**, que es exclusivamente documental.

#	Recomendación	Esfuerzo	Por qué
1	Completar la matriz de pruebas en dispositivos físicos	**2 a 3 jornadas** con 3 teléfonos	Es el único riesgo del producto que no se puede reducir desde el código
2	Decidir sobre el contenido sensible en el inventario: filtrarlo o documentar la excepción	**1 hora**	Hoy la documentación afirma algo que el código no cumple
3	Ejecutar el catálogo de regresión en una prueba de integración	**Media jornada**	El material ya está preparado y versionado
4	Probar la migración del historial desde preferencias	**3 horas**	Es el único flujo que puede perder datos del usuario
5	Aclarar en la documentación que las banderas de capacidad son una reserva, no un mecanismo activo	**30 minutos**	Elimina una expectativa falsa
6	Añadir `--fatal-infos` a la puerta de calidad local	**1 minuto**	Elimina la discrepancia entre local y CI
7	Fijar las 13 acciones de GitHub a SHA de commit	**30 minutos**	Cadena de suministro de los artefactos publicados
8	Añadir un diálogo antes de exportar historial o inventario	**2 horas**	El archivo sale en claro y la interfaz no lo advierte
9	Declarar un umbral mínimo de cobertura en la CI	**1 hora**	Hoy nada falla si la cobertura baja
10	Añadir un paso de CI que contraste el SBOM contra una base de CVE	**Media jornada**	El SBOM ya se genera; falta consultarlo

Total de las nueve recomendaciones técnicas (2 a 10): aproximadamente 2 jornadas. La primera —dispositivos físicos— es la más cara y la más valiosa.

Qué está fuera del alcance de esta versión

Declarado por el propio proyecto, no una carencia detectada:

- Firma de publicación, cuentas de desarrollador, fichas de tienda, capturas comerciales, AAB e IPA.

- Publicación en Google Play o App Store.
- Motor nativo para Windows y Linux.
- OCR, NFC, reputación remota de enlaces, consulta de productos, sincronización cifrada, API empresarial y modo kiosco.
- Interfaz en inglés: la infraestructura existe y el idioma **no se expone a propósito** hasta que todas las pantallas lean sus textos del sistema de traducción, para no entregar una interfaz a medio traducir.

Conclusión

Universal Code Scanner es un proyecto **maduro para su tamaño y honesto sobre sus límites**. Su promesa central —interpretar antes de actuar, sin enviar nada a ningún sitio— está implementada de forma verificable, y la parte más delicada del sistema, la criptografía, está bien construida y bien probada.

Las tres carencias de severidad alta son **divergencias entre lo que la documentación afirma y lo que el código hace**, no defectos de funcionamiento, y las tres se resuelven en menos de una jornada de trabajo entre las tres.

La decisión relevante no es técnica: **el producto no se ha probado en teléfonos reales de forma sistemática**, y esa es la brecha entre «compila, analiza y pasa sus pruebas» y «funciona en manos de una persona». Cerrarla cuesta dos o tres jornadas con tres dispositivos, y es el paso que separa esta versión de una candidata a publicación en tiendas.

Si necesitas…	Ve a
Entender qué hace, sin tecnicismos	01 · Descripción general, sección final
El detalle de los 35 hallazgos	15 · Riesgos y deuda técnica
La postura de seguridad completa	11 · Seguridad
El estado real de las pruebas	12 · Pruebas y calidad
Poner a alguien a trabajar en esto	18 · Guía para nuevos desarrolladores

18 · Guía para nuevos desarrolladores

Itinerario de incorporación. Está pensado para que alguien que llega hoy pueda hacer un cambio útil y seguro **al final del segundo día**.

Antes de nada: la sorpresa número uno

Un clon recién hecho no compila. No es un fallo: `.gitignore` excluye `/android/`, `/ios/`, `/web/` y `/macos/` con el comentario «Se generan con tool/bootstrap.py».

```
python tool/bootstrap.py --platforms android,web
```

Si te saltas ese paso perderás media hora buscando qué falta. Todo lo demás de esta guía asume que ya lo ejecutaste.

Día 1 · Entorno y primer arranque

Instalar

Requisito	Versión	Cómo comprobarlo
Flutter	**3.44.7**	`flutter --version`
Dart	Viene con Flutter	`dart --version`
Python	3.x	`python --version`
`pyyaml`	Cualquiera reciente	`python -c "import yaml"`

La versión de Flutter está fijada en `../../.fvmrc` y replicada en la variable `FLUTTER_VERSION` de ambos workflows. **Nada la comprueba automáticamente**: si usas otra, `pub get` podría resolver distinto y `bootstrap.py` generar plantillas distintas.

Los cinco comandos que vas a repetir

```
flutter pub get                              # resolver dependencias
python tool/bootstrap.py --platforms android,web
python tool/validate_structure.py            # diez comprobaciones offline, < 1 s
flutter analyze --fatal-infos                # el nivel que usa la CI
flutter test                                 # 57 casos, ~24 s
```

Memoriza el tercero: es el gate propio del repositorio, corre sin toolchain y te avisa de enlaces rotos, versiones descuadradas y rutas de tu máquina antes de que lleguen a la CI.

Aviso: `tool/run_quality_gate.sh` ejecuta `flutter analyze` **sin** `--fatal-infos`. Puede pasar en verde y fallar en la CI. Usa siempre la forma estricta a mano. Es el hallazgo R-06.

Arrancar

```
flutter run              # dispositivo o emulador
flutter run -d chrome    # PWA en local, sin --base-href
```

Día 1 · Qué leer, en este orden

No leas 6 821 líneas de golpe. Este recorrido cubre las decisiones importantes en unas dos horas.

#	Archivo	Líneas	Por qué
1	`lib/main.dart`	14	Tres decisiones en tres líneas
2	`lib/bootstrap.dart`	77	Todo el grafo de servicios, en orden
3	`lib/models/scan_record.dart`	161	El modelo central; `fromBarcode` es la confluencia de todo
4	`lib/core/security/payload_cipher.dart`	168	La pieza más delicada del sistema
5	`lib/core/security/scan_security_analyzer.dart`	165	Las 16 señales, todas en una función
6	`lib/features/scanner/widgets/scan_status_bar.dart`	137	Ejemplo del estilo de comentarios del proyecto
7	`lib/services/history_repository.dart`	223	Persistencia, migración y recorte

Y después, dos documentos del repositorio que valen más que cualquier resumen:

- `../quality/SCANNER_UX.md` — explica un defecto real y por qué se resolvió así. Es la mejor pieza de documentación del proyecto.
- `../quality/COMPATIBILITY_CONTRACT.md` — los 14 compromisos que **no puedes romper**.

Truco para leer un archivo grande

Extrae primero el mapa de símbolos:

```
grep -n "^\s*\(class \|abstract \|Future<\|static \|void \|String \|bool \)" \
 lib/features/scanner/scanner_screen.dart
```

Después lee en profundidad solo lo que importe.

Día 2 · El modelo mental

Cinco capas, dependencia hacia abajo

```
features/  pantallas por caso de uso
  ↓
state/     ChangeNotifier: ScanStore, InventoryStore, SettingsStore
  ↓
services/  interpretación, importación, exportación, repositorios
  ↓
models/    entidades inmutables y serializables
  ↓
core/      base de datos, cifrado, recuperación, tema, i18n, diagnóstico
```

Dos excepciones reales que verás en cuanto abras el código, y que están registradas como hallazgos: `models/scan_record.dart` importa `features/formats/domain/` (R-07), y `core/recovery/recovery_service.dart` importa `models/` para validar cargas descifradas.

El flujo que hay que tener en la cabeza

```
Cámara / Imagen / PDF
       ↓
ScanRecord.fromBarcode
       ■■■ ScanSecurityAnalyzer.analyze  → nivel de riesgo, canOpen
       ■■■ ContentParserRegistry.parse   → tipo, campos, sensible
       ↓
ScanResultsSheet  ← siempre se muestra todo
       ↓
¿saveHistory && !privateMode && !isSensitive?
       ↓ sí
PayloadCipher.encryptJson → Sembast
```

Interioriza esto: interpretación y análisis ocurren **antes** de que exista la posibilidad de acción, y la persistencia es una rama independiente de la acción.

Sin gestor de estado, sin router, sin DI

No hay Riverpod, Bloc, Provider, GetIt ni GoRouter. Los servicios se construyen una vez en `AppBootstrapper.initialize` y se pasan **por constructor** hasta las pantallas. La navegación entre pestañas es un índice entero sobre una lista de cinco widgets.

Puede parecer poco sofisticado. Es deliberado y funciona para el tamaño del proyecto: no hay indirección que rastrear.

Día 2 · Convenciones que hay que seguir

Estilo de Dart

`analysis_options.yaml` añade cuatro reglas a `flutter_lints`:

Regla	Qué te obliga a hacer
`always_declare_return_types`	Declarar el tipo de retorno siempre
`avoid_print`	**No usar `print`**; es control de privacidad, no de estilo
`prefer_final_locals`	`final` en toda variable local que no se reasigne
`use_build_context_synchronously`	`if (!mounted) return;` tras cada `await` antes de tocar el cont exto

La última es la que más aparece. El patrón del repositorio es:

```
await algoAsincrono();
if (!mounted) return;
setState(() { … });
```

y, dentro de un callback con `BuildContext` propio:

```
if (context.mounted) ScaffoldMessenger.of(context).showSnackBar(…);
```

Tipos explícitos

El proyecto escribe los tipos completos incluso donde `var` bastaría:

```
final List<RecordSnapshot<String, Map<String, Object?>>> snapshots = await _store.find(…);
```

Es verboso a propósito y hace el código buscable con `grep`. Síguelo.

Comentarios: el porqué, no el qué

Esta es la convención más importante. El repositorio tiene **130 líneas de comentario en 6 821 de código** —densidad baja— pero las que hay son de calidad alta. Todas explican una decisión.

Buen ejemplo real, de `scanner_screen.dart`:

```
// The old controller is released before the new one exists: both share a
// single platform camera session, and disposing it afterwards would tear
// down the session the new controller has just claimed.
```

Contraejemplo de lo que **no** hay que escribir: `// asigna el valor`.

Los comentarios existentes están en inglés; el texto de la interfaz, en español. Sigue esa separación.

Textos de interfaz

Solo doce cadenas viven en `AppLocalizations` —las cinco de navegación, título y subtítulo del escáner y algunas más—. El resto son literales en español dentro de cada pantalla.

Si añades una cadena nueva a la interfaz, añádela a `AppLocalizations`, no como literal. Es lo que exige `docs/quality/ACCESSIBILITY.md` y es el requisito para poder exponer el inglés algún día.

Serialización

Toda entidad persistida implementa `toJson()` y `fromJson()` a mano. **No hay generación de código**: ni `json_serializable`, ni `freezed`, ni `build_runner`.

Las enumeraciones se serializan **por nombre** (`.name` / `.byName`). Renombrar un valor de `ContentKind`, `RiskLevel`, `RecoveryEntityType` o `RecoveryIssueState` **rompe los datos ya guardados y los respaldos exportados**.

Pruebas

Los dobles se escriben a mano implementando la interfaz. No hay `mockito` ni `mocktail`. Ejemplos a copiar: `_FakeTonePlayer`, `_TrackingKeyProvider`, `_TestParser`.

Patrones obligatorios:

```
// Base en memoria
final AppDatabase database = await AppDatabase.openTemporary();
addTearDown(database.close);

// Llaves en memoria
PayloadCipher(keyProvider: MemoryEncryptionKeyProvider());

// Preferencias en memoria
SharedPreferencesAsyncPlatform.instance = InMemorySharedPreferencesAsync.empty();
```

Y dos prohibiciones aprendidas a base de defectos reales:

1. **Nunca `pumpAndSettle` sobre una animación infinita.** La barra indeterminada del escáner nunca para: `pumpAndSettle` se cuelga. Usa `await tester.pump()`.

2. **Nunca abras la base dentro del reloj falso de `testWidgets`.** Sembast usa temporizadores reales. Envuelve en `tester.runAsync(...)`.

Ambas están comentadas en el propio código de las pruebas.

Finales de línea

`.gitattributes` fuerza `* text=auto eol=lf` porque **los SHA-256 de `SOURCE_MANIFEST.json` dependen de ello**. Cualquier archivo de texto nuevo debe escribirse con LF. La única excepción es `*.ps1`, que va con CRLF para que PowerShell lo lea de forma nativa.

Lo que NO puedes tocar sin leer antes

Seis constantes cuya modificación **convierte los datos del usuario en ruido**, sin reversión posible:

Constante	Archivo
`PayloadCipher.currentVersion`	`lib/core/security/payload_cipher.dart`
`PayloadCipher.currentAlgorithm`	idem
`PayloadCipher.currentKeyId`	idem
`PayloadCipher.legacyKeyId`	idem
`SecureStorageKeyProvider._prefix`	idem
`SchemaMigrator.currentVersion`	`lib/core/database/schema_migrator.dart`

No es reversible porque la **actualización perezosa de sobres** reescribe los registros antiguos la primera vez que se leen. Antes de tocar cualquiera de las seis, lee 07-database.md y `../quality/COMPATIBILITY_CONTRACT.md`.

Y tres versiones de dependencia que **rompen `pub get`** si las subes:

Paquete	Tope	Motivo
`share_plus`	`12.0.2`	Desde `13.0.0` exige `win32 ^6`, incompatible con `file_picker 11.x`
`pdfrx`	`2.4.5`	Desde `2.4.6` arrastra `archive ^4`, incompatible con `excel 4.0.6`
`excel`	`4.0.6`	Es la que ancla `archive` a la serie 3

`flutter pub outdated` te dirá que se pueden subir. **No los subas.** El motivo está en `../quality/LOCKFILE.md` y replicado en `.github/dependabot.yml`.

Cómo hacer los cambios más habituales

Añadir un intérprete de contenido nuevo

El camino limpio, sin tocar el intérprete integrado:

```
class MiParser implements ContentParser {
 @override String get id => 'mi-parser';
 @override int get priority => 100;          // el integrado tiene -1000
 @override bool canParse(String raw) => raw.startsWith('MIFORMATO:');
 @override ParsedContent parse(String raw) => ParsedContent(
       kind: ContentKind.text,
       title: 'Mi formato',
       fields: <String, String>{ 'Campo': raw },
       sensitive: false,
     );
}

ContentParserRegistry.instance.register(MiParser());
```

Cuidado con `sensitive`: decide si el registro entra o no en el historial. Y si tu parser produce un campo con datos delicados, **debe llamarse exactamente `Contraseña`, `Secreto`, `Dirección`, `IBAN` o `Carga`** para que se enmascare. Es una lista literal en `scan_result_sheet.dart`, hallazgo R-19.

No hay ningún punto en la aplicación desde el que se registre un parser: la extensibilidad es en tiempo de compilación, para quien bifurque el repositorio.

Añadir un ajuste

Cinco archivos, en este orden:

1. `lib/models/app_settings.dart` — campo, valor por defecto y `copyWith`;
2. `lib/services/settings_repository.dart` — leerlo en `load` y escribirlo en `save`;
3. decidir si va en la lista de `resetNonSensitive()` —los de seguridad **no** van: `save_history`, `private_mode` y `biometric_lock` quedan fuera a propósito;
4. `lib/features/settings/settings_screen.dart` — el control;
5. donde se consuma.

Añadir una señal de riesgo

Una sola línea en `ScanSecurityAnalyzer.analyze`:

```
if (miCondicion(uri)) caution('Texto en español que explica qué se detectó.');
```

`caution` solo eleva desde `low`; `high` es absorbente. **Añade también la prueba en `test/scan_security_analyzer_test.dart`** —es la única defensa antes de una acción externa— y actualiza el número «16» en `README.md`, `CHANGELOG.md`, `IMPLEMENTATION_STATUS.md` y en 05-technical-reference.md.

Añadir un campo persistido

1. Añádelo al modelo, a `toJson` y a `fromJson` **con valor por defecto** en la lectura: es lo que hace que un registro antiguo siga cargando.
2. **No subas `PayloadCipher.currentVersion`**: el sobre no cambia, cambia su contenido.
3. Si el campo es obligatorio y no puede tener valor por defecto, hace falta una migración explícita.

Subir la versión

Dos archivos, siempre juntos:

1. `pubspec.yaml`, campo `version`, forma `X.Y.Z+N`;
2. `lib/core/app_info.dart`, constante `appVersion`, parte `X.Y.Z`.

`validate_structure.py` **falla si divergen**. Ese gate nació de un defecto real: la pantalla «Acerca de» mostró `1.0.0` durante toda la versión `1.1.0`.

Primeras tareas sugeridas

Ordenadas por dificultad creciente. Las cuatro primeras son cambios reales y útiles que no tocan nada delicado.

#	Tarea	Dificultad	Qué te enseña
1	Añadir `--fatal-infos` a `tool/run_quality_gate.sh` (R-06)	Trivial	El sistema de gates
2	Añadir la columna `notas` al CSV del historial (R-16)	Baja	`ExportService` y el isolate de `compute`
3	Escribir pruebas para `_escape` y `_normalizeUrl` del generador	Baja	Las convenciones de prueba
4	Eliminar `BarcodeLabels.contentType` y `_fallbackType` (R-05)	Baja	Verificar que algo es código muerto antes de borrarlo
5	Escribir la prueba de que un registro sensible **no** entra en el historial	Media	El filtro de privacidad y los dobles
6	Probar `_trim` y `pruneOlderThan`	Media	Sembast y los `Finder`
7	Probar la migración del historial desde preferencias (R-34)	**Alta**	El flujo más delicado del proyecto
8	Resolver R-03: filtrar contenido sensible en el inventario	**Alta**	Requiere decisión de producto, no solo código

Antes de la 7 y la 8, habla con quien mantenga el proyecto: la 7 toca el único flujo que puede perder datos, y la 8 cambia comportamiento observable.

Antes de cada commit

```
python tool/validate_structure.py
flutter analyze --fatal-infos
flutter test
```

Si tocaste archivos versionados nuevos, además:

```
python tool/generate_source_manifest.py
```

Y si tocaste la documentación de `docs/system-documentation/`:

```
python tool/build_system_documentation_pdf.py
```

Dónde buscar cuando algo no cuadra

Pregunta	Documento
¿Qué hace esta función?	05 · Referencia técnica
¿Por qué está escrita así?	06 · Explicación profunda
¿Dónde vive esto?	04 · Mapa del código
¿Cómo se guarda?	07 · Base de datos
¿Por dónde entra y sale este dato?	08 · Flujo de datos
¿Qué ajuste controla esto?	10 · Configuración
Me sale este error	14 · Solución de problemas
¿Esto es un defecto conocido?	15 · Riesgos y deuda técnica
¿Qué significa esta palabra?	16 · Glosario
¿Dónde se prueba esta funcionalidad?	19 · Matriz de trazabilidad

---

Diez cosas que descubrirías a golpes

Aquí están, para que no tengas que descubrirlas así:

1. **Un clon no compila** sin `python tool/bootstrap.py`.
2. **Leer el historial escribe en la base**, si algún sobre está desactualizado.
3. **El momento de cada lectura queda en claro** en la base; el contenido no.
4. **Un registro que no descifra no rompe la carga**: se omite y se registra como incidencia.
5. **El inventario sí guarda contenido sensible**; el escáner no.
6. **`autoTorch` no se aplica en el inventario**, solo en la pestaña Escanear.
7. **El temporizador del portapapeles sobrevive** al widget que lo creó.
8. **Los diagnósticos se pierden al cerrar**: viven solo en memoria.
9. **Las llaves antiguas no se borran** tras una rotación exitosa.
10. **Restablecer preferencias visuales no toca** `save_history`, `private_mode` ni `biometric_lock`.

Continúa en 19 · Matriz de trazabilidad.

19 · Matriz de trazabilidad

Cada fila permite seguir una funcionalidad **desde la interfaz hasta la persistencia y sus pruebas, sin salir de la fila**.

Convención de la columna «Prueba»: se cita el archivo y el nombre del caso. Un – significa **sin prueba automatizada**, y es un dato, no un descuido de esta tabla.

---

1 · Lectura de códigos

Funcionalidad	Interfaz	Orquestación	Lógica	Datos	Prueba
Leer con la cámara	`ScannerScreen` · pestaña Escanear	`_handleCapture`	`ScanRecord.fromBarcode`	`scan_history` si procede	—
Ver el estado del escáner	`ScanStatusBar` + `ScannerOverlay`	`_phaseFor`	`ScanPhase`	Ninguno	`scan_status_bar_test.dart`: 5 casos
Pausar y reanudar	Botón «Pausar»/«Escanear»; toque en la vista	`_togglePause`, `_resumeScanning`	`MobileScannerEngine.stop/start`	Ninguno	`scan_status_bar_test.dart`: «a paused camera is announced and offers the way back»
Reiniciar la cámara	Botón «Reiniciar cámara»	`_restartCamera`	Nuevo `MobileScannerEngine` + nueva `_previewKey`	Ninguno	—
Linterna	Botón de linterna	`_engine.toggleTorch`	`MobileScannerController`	Preferencia `auto_torch`	—
Zoom	Deslizador	`_engine.setZoomScale`	`MobileScannerController`	Ninguno	—
Cambiar de cámara	Botón	`_engine.switchCamera`	`MobileScannerController`	Ninguno	—
Enfocar al tocar	Toque en la vista	`tapToFocus: true`	`MobileScanner`	Ninguno	—
Marco de lectura	Ajustes → «Marco de lectura real»	`_settings.useScanWindow`	`scanWindow` o `null`	Preferencia `use_scan_window`	—
Varios códigos a la vez	Automático	`_handleCapture` con mapa `unique`	`payloadForBarcode`	Un registro por código	—
Antirrebote de 2 s	Automático	`_lastSignature`, `_lastDetectedAt`	Firma ordenada de la captura	Ninguno	—
Tono y vibración	Automático tras leer	`ScanFeedback.success`	`AssetScanTonePlayer` · `HapticFeedback`	`assets/sounds/scan_success.wav`	`scan_feedback_test.dart`: 4 casos

Movimiento reducido	Ajustes → «Reducir movimiento»	`animate: !reduceMotion`	`_ScanProgressBar`, `ScannerOverlay`	Preferencia `reduce_motion`	`scan_status_bar_test.dart`: «reduced motion keeps the bar readable without movement»

2 · Lectura desde archivo

Funcionalidad	Interfaz	Orquestación	Lógica	Datos	Prueba
Importar imágenes	Menú → «Varias imágenes»	`_scanFromGallery`	`ImagePicker.pickMultiImage(limit: 20)` · `analyzeImage`	`scan_history`, `source: 'Imagen'`	—
Importar PDF	Menú → «Documento PDF»	`_scanFromPdf`	`PdfPageRenderer.pickAndRender(maxPages: 50)`	`scan_history`, `source: 'PDF · página N'`	—
Progreso del lote	`_BatchProgressDialog`	`_runBatchDialog`	`BatchProgress`	Ninguno	`cancellation_token_test.dart`: «batch progress reports a bounded fraction»
Cancelar un lote	Botón «Cancelar»	`token.cancel()`	`CancellationToken` · `PdfPageRendererCancellationToken`	**Nada se modifica**	`cancellation_token_test.dart`: «cancellation token stops batch work deterministically»
Limpieza de temporales	Automática	`PdfPageRenderer.cleanup`	Borra PNG y directorio	Directorio temporal	—

3 · Interpretación del contenido

Tipo interpretado	Detección	Función	`sensitive`	Prueba
Enlace web	`http`/`https` o `www.`	`ContentInterpreter.parse` rama 18	no	`content_interpreter_test.dart`: «normaliza un enlace que comienza con www»
Wi-Fi	`wifi:`	`_wifi` + `_segments`	**sí si trae contraseña**	«interpreta Wi-Fi con campos escapados»
Contacto vCard	`begin:vcard`	`_vcard`	no	«interpreta una vCard»
Contacto MeCard	`mecard:`	`_mecard`	no	—
Evento	`begin:vevent`	`_event`	no	—
Correo	`mailto:` o `matmsg:`	`_email`	no	—

Teléfono	`tel:`	Rama en línea	no	—
SMS	`sms:` o `smsto:`	`_sms`	no	—
Ubicación	`geo:`	`_geo`	no	—
OTP	`otpauth:`	`_otp`	**sí**	`scan_record_test.dart`: «identifica OTP y Wi-Fi con contraseña como sensibles»
Criptomoneda	`bitcoin:`, `lightning:`, `ethereum:`	`_crypto`	**sí**	—
Swiss QR Bill	Empieza por `SPC\n`	`_swissQr`	**sí**	—
EPC / SEPA	Empieza por `BCD\n`	`_epc`	**sí**	—
EMVCo	`000201` y contiene `6304`	`_emv` + `_tlv`	**sí**	«interpreta datos EMVCo»
AAMVA	`@` y contiene `ANSI`	`_aamva`	**sí**	—
GS1	`(NN)` o `<GS>`	`_gs1` + `_aiLabel`	no	«interpreta identificadores GS1»
ISBN	13 dígitos con `978`/`979`, o 10 caracteres	`_isIsbn`	no	«identifica ISBN»
Producto	8 a 14 dígitos	Rama en línea	no	—
Binario	`binary-base64:`	Rama 1	no	`scan_record_test.dart`: «conserva un contenido binario como Base64»
Texto	Todo lo demás	Rama final	no	—
Parser propio	`register()`	`ContentParserRegistry`	Lo decide el parser	`content_parser_registry_test.dart`: «custom parser is isolated and removable»

Cobertura: 6 de 20 ramas tienen prueba. Ver R-10 y la propuesta 6 de 12-testing-and-quality.md.

---

4 · Análisis de riesgo

Señal	Nivel	Prueba
HTTPS normal, sin señales	`low`	`scan_security_analyzer_test.dart`: «acepta un enlace HTTPS común»
`http` sin TLS	`caution`	«advierte cuando el enlace usa HTTP»
Host vacío	`high`	—

Credenciales antes del dominio	`high`	«marca credenciales antes del dominio como riesgo elevado»
Correo en la consulta, no credencial	`low`	«no marca un correo dentro de la consulta como credencial»
Punycode (`xn--`)	`high`	«marca Punycode como riesgo elevado»
Alfabetos mezclados	`high`	—
Caracteres internacionales en el host	`caution`	—
IP literal	`caution`	—
Red privada, `localhost` o `.local`	`high`	—
Acortador conocido (11 dominios)	`caution`	—
Más de 5 subdominios	`caution`	—
Puerto inusual	`caution`	—
Longitud mayor de 240	`caution`	—
Caracteres de control o `%00`	`high`	—
Descarga ejecutable (17 extensiones)	`high`	—
Ruta que aparenta inicio de sesión	`caution`	—
Tres o más parámetros de seguimiento	`caution`	—
Esquema de aplicación no permitido	`high`, `canOpen: false`	«bloquea esquemas desconocidos»
Esquema de sistema (`tel:`)	`low`, `canOpen: true`	«permite acciones telefónicas explícitas»
Contenido estructurado (Wi-Fi)	`low`, `canOpen: false`	«trata un QR Wi-Fi como contenido estructurado»
Pago o secreto (`otpauth:`)	`caution`, `canOpen: false`	«advierte que una clave OTP es sensible»

Todo en `lib/core/security/scan_security_analyzer.dart`. **9 de 22 casos tienen prueba.**

---

5 · Acciones sobre un resultado

Funcionalidad	Interfaz	Orquestación	Condición	Salida	Prueba
Abrir un enlace	Botón «Abrir»	`_openRecord`	Diálogo si `confirmBeforeOpen` **o** riesgo distinto de `low`	`launchUrl`	—
Llamar, correo, SMS, mapa	Botón contextual	`_openRecord` + `_actionUri`	Misma condición	`launchUrl`	—

Copiar	Botón «Copiar»	`ClipboardService.copy`	Ninguna	Portapapeles, con borrado programado	—
Compartir texto	Botón «Compartir»	`_shareRecord`	**Diálogo si el registro es sensible**	`share_plus`	—
Compartir contacto	Idem, tipo `contact`	`_shareRecord`	Idem	Archivo `.vcf`, MIME `text/vcard`	—
Compartir evento	Idem, tipo `event`	`_shareRecord`	Idem	Archivo `.ics`, MIME `text/calendar`	—
Enmascarar valores sensibles	Botón del ojo	`_revealed` local	`isSensitive && hideSensitiveValues`	Ninguna: solo pantalla	—
Banda de riesgo	`_RiskBanner`	—	Siempre visible salvo `compact`	Ninguna	—

Ninguna fila de esta tabla tiene prueba automatizada. Es el hueco más relevante de la suite: aquí viven la decisión de confirmación y la de enmascarado. Ver la propuesta 8 de 12-testing-and-quality.md.

---

6 · Historial

Funcionalidad	Interfaz	Orquestación	Persistencia	Prueba
Guardar una lectura	Automático	`_persistAndShow` → `ScanStore.addAll`	`HistoryRepository.upsertAll` → `scan_history`	—
No guardar contenido sensible	Automático	`records.where((r) => !r.isSensitive)`	Nada se escribe	`scan_record_test.dart` prueba `isSensitive`, **no el filtro**
No guardar con sesión privada	Ajustes → «Sesión privada»	`!privateMode`	Nada se escribe	—
No guardar nunca	Ajustes → «Guardar historial»	`saveHistory`	Nada se escribe	—
Listar y filtrar	`HistoryScreen`	Filtro en memoria	Lectura de `scan_history`	—
Buscar	Campo de búsqueda	`rawValue`, `contentType`, `format`, etiquetas, notas	En memoria	—
Filtrar por riesgo	`ChoiceChip`	`_risk`	En memoria	—
Favoritos	Icono de estrella	`ScanStore.toggleFavorite`	`upsert` recifrando	—
Notas y etiquetas	Menú → «Notas y etiquetas»	`_edit` → `ScanStore.update`	`upsert` recifrando	—

Eliminar un registro	Menú → «Eliminar»	`ScanStore.remove`	`_store.record(id).delete`	—
Borrar todo	Menú → «Borrar historial», con diálogo	`ScanStore.clear`	`_store.delete`	—
Retención automática	Ajustes → «Retención del historial»	`_updateRetention`	`pruneOlderThan` por `scannedAt`	—
Recorte a 5 000	Automático tras escribir	`_trim`	Borra los más antiguos	—
Cargar al arrancar	Automático	`ScanStore.initialize`	`HistoryRepository.load`	—
Actualizar sobres antiguos	Automático al cargar	`inspect` + `upgradeEnvelope`	Reescribe el `payload`	—
Aislar registros dañados	Automático	`catch` en `load`	Incidencia en `recovery_issues`	—

---

7 · Importación y exportación

Funcionalidad	Interfaz	Orquestación	Validación	Prueba
Importar historial	Menú → «Importar respaldo JSON»	`_importHistory`	6 puertas en `ImportService`	`import_service_test.dart`: 4 casos
Vista previa	Diálogo con válidos, duplicados y rechazados	`HistoryImportPreview`	Nada se escribe hasta elegir	«history import previews valid, duplicate, and rejected records»
Estrategia «Omitir duplicados»	Botón	`preview.apply(ids, skipDuplicates)`	—	Misma prueba
Estrategia «Combinar»	Botón	`ScanStore.addAll`	—	—
Estrategia «Reemplazar»	Botón	`HistoryRepository.replaceAll`	Transacción atómica	`history_repository_test.dart`: «replaceAll atomically replaces the stored history»
Rechazar esquema futuro	Automático	`schemaVersion > 2`	`FormatException`	«future schema is rejected before modifying data»
Colapsar duplicados internos	Automático	Mapa por `id`	—	«duplicate records inside the same file are collapsed before import»
Rechazar JSON profundo	Automático	`_validateJsonShape`	`FormatException`	«excessively deep JSON is rejected before persistence»
Exportar historial JSON	Menú	`ExportService.shareHistoryJson`	—	—

Exportar historial CSV	Menú	`shareHistoryCsv` → `compute(_encodeCsv)`	—	—
Exportar historial XLSX	Menú	`shareHistoryXlsx` → `compute(_encodeXlsx)`	—	—
Importar inventario	Botón «Importar JSON»	`_importInventory`	`maxInventoryItems`	—
Exportar inventario	Menú de la sesión	Tres métodos de `ExportService`	—	—

---

8 · Inventario

Funcionalidad	Interfaz	Orquestación	Persistencia	Prueba
Crear una sesión	Campo + «Iniciar inventario»	`InventoryStore.createSession`	`inventory_sessions`	—
Contar leyendo	Cámara del inventario	`_onDetect` → `addScan`	Cifrado, con cola serial	—
Antirrebote de 1200 ms	Automático	`_lastCode`, `_lastAt`	Ninguna	—
Cambiar cantidad	Botones ±	`setQuantity`	Cifrado	—
Quitar un producto	Menú, o restar hasta cero	`setQuantity(code, 0)`	Elimina del mapa	—
Nota de producto	Menú → «Editar nota»	`updateItemNotes`	Cifrado	—
Cerrar la sesión	Botón «Cerrar», con diálogo	`closeActive`	Escribe `closedAt`	—
Reabrir	Menú → «Reabrir sesión»	`reopenSession`	Limpia `closedAt`	—
Eliminar la sesión	Menú → «Eliminar»	`deleteSession`	Borra el registro	—
Importar como copia	«Importar JSON»	`importSession`	**Siempre `id` nuevo**	—
Serializar el modelo	—	`toJson`/`fromJson`	—	`inventory_session_test.dart`: «calcula unidades y conserva datos al serializar»
Escrituras serializadas	Automático	`AsyncWriteQueue`	—	`async_write_queue_test.dart`: 2 casos

El inventario no filtra contenido sensible (R-03) y **no usa `ScannerEngine`** (R-04).

---

9 · Generador

Funcionalidad	Interfaz	Lógica	Salida	Prueba

Nueve tipos de carga	Desplegable	`_payload`	Cadena	—
Diez formatos visuales	Desplegable	`_barcode`	`Barcode` de `barcode_widget`	—
Corrección de errores	Interruptor, solo QR	`BarcodeQRCorrectionLevel`	—	—
Escapar el Wi-Fi	Automático	`_escape`	Inverso de `_segments`	—
Normalizar la URL	Automático	`_normalizeUrl`	Antepone `https://`	—
Vista previa	`BarcodeWidget`	—	Pantalla	—
Error de contenido inválido	`errorBuilder`	—	Mensaje explícito	—
Copiar los datos	Botón	`ClipboardService.copy`	Portapapeles	—
Compartir los datos	Botón	`SharePlus`	Texto	—
Exportar PNG	Botón	`RepaintBoundary.toImage(pixelRatio: 3)`	Archivo PNG	—
Exportar SVG	Botón	`Barcode.toSvg`	Archivo SVG	—

Nada del generador se persiste.

---

10 · Cifrado y datos

Funcionalidad	Módulo	Detalle	Prueba
Cifrar una carga	`PayloadCipher.encryptJson`	AES-256-GCM, sobre versionado	`payload_cipher_test.dart`: «versioned envelope round-trips without exposing clear values»
Descifrar	`decryptJson`	Acepta sobre actual y heredado	Misma prueba
Leer un sobre heredado	`decryptJson` con `legacy`	Sin campo `version`	«legacy envelope remains readable and can be upgraded»
Actualizar un sobre	`upgradeEnvelope`	Solo si hace falta	Misma prueba
Inspeccionar sin descifrar	`inspect`	Versión, algoritmo, `keyId`	Usada en varias pruebas
Rechazar versión futura	`decryptJson`	`FormatException`	«future cipher envelope versions are rejected explicitly»
Rechazar manipulación	`decryptJson`	Fallo del MAC	«tampered authenticated ciphertext is rejected»
No regenerar llave ausente	`_missingKeyIds`	Bloquea cifrado **y** descifrado	«a missing decryption key is never replaced silently»

Guardar la llave	`SecureStorageKeyProvider`	`ucs_database_key_<keyId>`	—
Llave en memoria	`MemoryEncryptionKeyProvider`	Modo temporal y pruebas	Usada en todas
Rotar la llave	`DataMaintenanceService`	Transacción única	`data_maintenance_test.dart`: «key rotation updates records and active metadata in one database transaction»
Abortar rotación incompleta	Idem	`StateError`	«rotation aborts before changing metadata when a payload is incomplete»
Limpiar llave huérfana	Idem	`forgetKey` en el `catch`	«failed rotation removes a newly generated orphan key»
Guardar la llave activa	`EncryptionMetadataRepository`	`_security_meta`	Cubierta indirectamente
Migrar el metadato desde preferencias	Idem	Lee, escribe y borra	—
Migrar el esquema	`SchemaMigrator`	Idempotente y transaccional	`schema_migrator_test.dart`: «schema migration is idempotent and non-destructive»
Migrar el historial heredado	`HistoryRepository._migrateLegacyHistory`	Respaldo, verificación, borrado	— (R-34)

---

11 · Recuperación y diagnóstico

Funcionalidad	Interfaz	Módulo	Persistencia	Prueba
Inicio seguro	`_StartupRecovery`	`BootstrapHost`	Ninguna	`app_launch_test.dart` (integración)
Reintentar arranque	Botón «Reintentar»	`_retry()`	Ninguna	—
Modo temporal	Botón	`initialize(temporary: true)`	**Memoria**	—
Restablecer preferencias	Botón	`resetNonSensitive()`	Borra 14 de 17 claves	—
Copiar diagnóstico	Botón	`AppDiagnostics.exportJson`	Ninguna	`diagnostics_test.dart`: «diagnostic export excludes error messages and sensitive values»
Registrar errores globales	Automático	`installGlobalErrorHandlers`	Memoria, 100 entradas	Misma prueba
Listar incidencias	`RecoveryScreen`	`RecoveryService.load`	`recovery_issues`	—

Reintentar recuperación	Menú	`RecoveryService.retry`	Reinserta el registro	—
Descartar	Menú, con diálogo	`RecoveryService.discard`	Borra el registro	—
Marcar revisado	Menú	`repository.mark`	Cambia el estado	—
Limpiar resueltas	Botón	`clearResolved`	Borra las no pendientes	—
Exportar paquete	Icono de la barra	`exportBundle`	Ninguna	`recovery_service_test.dart`: «recovery bundle preserves ciphertext but never includes the key or clear payload»
Accesibilidad de la pantalla	—	—	—	`accessibility_test.dart`: texto al 200 %

---

12 · Ajustes

Ajuste	Clave	Consume	Prueba
Tema	`theme_mode`	`MaterialApp.themeMode`	—
Idioma	`language`	`MaterialApp.locale`	`localization_test.dart`: 4 casos
Alto contraste	`high_contrast`	`AppTheme`	—
Controles grandes	`large_controls`	`AppTheme`	—
Reducir movimiento	`reduce_motion`	Tema, barra y marco	`scan_status_bar_test.dart`
Marco de lectura	`use_scan_window`	Ambas pantallas de cámara	—
Linterna al iniciar	`auto_torch`	**Solo `ScannerScreen`**	—
Sonido	`sound_enabled`	`ScanFeedback.success`	`scan_feedback_test.dart`
Vibración	`vibration_enabled`	Idem	Idem
Guardar historial	`save_history`	`_persistAndShow`	—
Sesión privada	`private_mode`	Idem	—
Ocultar sensibles	`hide_sensitive_values`	`ScanRecordCard`	—
Confirmar antes de abrir	`confirm_before_open`	`_openRecord`	—
Bloqueo de la aplicación	`biometric_lock`	`BiometricLockGate`	—
Borrar portapapeles	`clear_clipboard_seconds`	`ClipboardService.copy`	—

Retención	`history_retention_days`	`pruneOlderThan`	—
Banderas	`feature_flags`	**Nadie** (R-02)	—
Ancho de teléfono	—	`HandheldFrame`	`handheld_frame_test.dart`: 3 casos

---

13 · Garantías públicas y su verificación

Cada afirmación que el proyecto hace de cara al público, con el código que la implementa y el estado real de su verificación.

Garantía	Fuente	Código	Prueba	Estado
«Nada se abre solo»	`README.md`	`_openRecord` · un solo `launchUrl`	—	**Verificado por lectura**
«Historial cifrado AES-256-GCM»	`README.md`	`PayloadCipher`	5 casos	**Verificado**
«Llave ausente ≠ llave nueva»	`README.md`	`_missingKeyIds`	1 caso	**Verificado**
«Rotación segura»	`README.md`	`rotateEncryptionKey`	3 casos	**Verificado**
«OTP y Wi-Fi fuera del historial»	`README.md`, `docs/SECURITY.md`	`_persistAndShow`	`isSensitive` sí; el filtro **no**	**Parcial** · falso en el inventario (R-03)
«Diagnóstico sin datos»	`README.md`	`AppDiagnostics`	1 caso	**Verificado**
«Telemetría cero»	`README.md`	Sin cliente de red ni permiso `INTERNET`	—	**Verificado por `grep`**
«16 señales locales»	`README.md`, `CHANGELOG.md`	`analyze`	9 de 22 casos	**Verificado por conteo**
«57 pruebas en verde»	`README.md`, `VALIDATION.md`	—	`flutter test`	**Verificado ejecutando**
«`analyze --fatal-infos`: 0 hallazgos»	`README.md`, `VALIDATION.md`	—	`flutter analyze`	**Verificado ejecutando**
«Modo temporal en memoria»	`README.md`	`openTemporary`	—	Verificado por lectura
«Centro de recuperación por registro»	`README.md`	`RecoveryService`	1 caso	**Verificado**
«Generador de 10 simbologías»	`README.md`	`_CodeFormat`	—	**Verificado por conteo**
«Lotes de 20 imágenes y 50 páginas»	`README.md`	`limit: 20`, `maxPages: 50`	—	**Verificado por lectura**
«Catálogo de regresión de 8 imágenes»	`CHANGELOG.md`	`test_assets/manifest.json`	**Ninguna**	**Existe, no se ejecuta** (R-01)

«`ScannerEngine` aísla el motor»	`docs/ARCHITECTURE.md`	Solo en `ScannerScreen`	—	**Parcial** (R-04)
«`FeatureFlags` reserva capacidades»	`docs/quality/FEATURE_FLAGS.md`	Persistido, sin lector	—	**Reserva de forma** (R-02)
«Interfaz solo en español»	`README.md`	Delegado rechaza `en`	4 casos	**Verificado**
«APK con clave de depuración»	`README.md`, `docs/RELEASE.md`	—	—	**Declarado**
«Sin pruebas en dispositivos físicos»	`IMPLEMENTATION_STATUS.md`	—	—	**Declarado**

---

14 · De un archivo a su cobertura

Vista inversa: qué archivo prueba qué.

Archivo de prueba	Cubre
`test/core/payload_cipher_test.dart`	`PayloadCipher`, `MemoryEncryptionKeyProvider`
`test/core/data_maintenance_test.dart`	`DataMaintenanceService`, `EncryptionMetadataRepository`
`test/core/recovery_service_test.dart`	`RecoveryService.exportBundle`, `RecoveryRepository.record`
`test/core/schema_migrator_test.dart`	`SchemaMigrator`
`test/core/diagnostics_test.dart`	`AppDiagnostics`
`test/core/async_write_queue_test.dart`	`AsyncWriteQueue`
`test/core/cancellation_token_test.dart`	`CancellationToken`, `BatchProgress`
`test/core/localization_test.dart`	`AppLocalizations` y su delegado
`test/core/handheld_frame_test.dart`	`HandheldFrame`
`test/core/accessibility_test.dart`	`RecoveryScreen` con texto al 200 %
`test/features/scan_status_bar_test.dart`	`ScanStatusBar`, `ScannerOverlay`
`test/services/import_service_test.dart`	`ImportService`, `HistoryImportPreview`
`test/services/history_repository_test.dart`	`HistoryRepository.replaceAll` y `load`
`test/services/scan_feedback_test.dart`	`ScanFeedback`, `ScanTonePlayer`
`test/services/content_parser_registry_test.dart`	`ContentParserRegistry`, `ContentParser`
`test/content_interpreter_test.dart`	`ContentInterpreter`, 6 de 20 ramas
`test/scan_security_analyzer_test.dart`	`ScanSecurityAnalyzer`, 9 de 22 casos
`test/scan_record_test.dart`	`ScanRecord` y su serialización

`test/inventory_session_test.dart`	`InventorySession`, `InventoryItem`
`integration_test/app_launch_test.dart`	`BootstrapHost`, arranque completo

Archivos de `lib/` sin ninguna prueba que los toque

`app.dart` (salvo `HandheldFrame`), `bootstrap.dart`, `bootstrap_host.dart`, `main.dart`, `core/app_info.dart`, `core/feature_flags/`, `core/theme/`, `core/utils/barcode_labels.dart`, `models/app_settings.dart`, `models/parsed_content.dart`, `services/biometric_service.dart`, `services/clipboard_service.dart`, `services/export_service.dart`, `services/inventory_repository.dart`, `services/settings_repository.dart`, `services/pdf_page_renderer*.dart`, `state/scan_store.dart`, `state/inventory_store.dart`, `state/settings_store.dart`, y **todas** las pantallas de `features/` salvo los dos widgets del escáner.

Vuelve al índice.